Modgen Version 10.1.0

Developer's Guide

By Statistics Canada

This manual was produced using *Doc-To-Help*®, by ComponentOne™.

Table of Contents

Getting Started

Introduction

This document introduces the topic of dynamic longitudinal microsimulation models and how these models can be developed in the Modgen language. The Modgen language is, essentially, a superset of the C++ programming language in which the application developer simulates the lifetimes of a set of linked actors (e.g. a core individual and related family members). Modgen provides a variety of ways of reporting model simulation output. These include a cross-tabulator which writes results in MS Access format, which can then be exported to MS Excel or text file formats, and a tracking facility which produces a relational database of detailed actor information in MS Access format.

At present, this manual fills both the role of a Developer's Guide to the Modgen modeling language and a reference manual to some of the finer details of this product. As Modgen is still under development, the focus of this guide is to provide a basic understanding of what the language is, and the type of models which can be developed in this framework.

The current version of this guide can be used by a range of application developers. Users who wish to run existing models written in the Modgen language and adjust some of the exogenous model parameters should refer to a separate document entitled "Guide to the Modgen Visual Interface". However, users who wish to alter the tabular output and the algorithms which dictate the behavior of the model actors should read this entire document. Since Modgen is an extension of the C++ language, model developers should have some understanding of structured programming practices. Those who do not have this background will be unfamiliar with many of the concepts described herein. This document makes some attempt to educate the reader in these areas but it is not intended as an introduction to structured programming.

The Modgen language supports the development of three types of longitudinal microsimulation models. The first two are discrete and continuous time models in which a specified number of cases are simulated sequentially. Examples of these model types are the LifePaths model and the POHEM model. This documentation is meant to describe the various aspects of creating and using these types of models. Another type of microsimulation model, a time-based model, has been developed at Statistics Canada in the Modgen language. Modgen also supports simpler cell-based models. (The cell-based model is not described further in this guide.)

For more information on Modgen and Modgen models, please visit the Modgen page on the Statistics Canada web site (www.statcan.gc.ca/microsimulation/modgen-eng.htm).

Status of the Documentation

Modgen is being continuously improved, and new functionality added. This version of the Modgen Developer's Guide was written for version 10.1.0 of Modgen. Users who wish to learn about new or proposed new features of Modgen are encouraged to contact microsimulation@statcan.gc.ca

System Requirements

Modgen has been tested on Windows XP. No general statements can be made about CPU, disk, or memory requirements, since individual Modgen models vary greatly. Modgen does have an intrinsic ability to use multiple processors, and for some models additional processors can enhance performance significantly. For other models, physical memory is the limiting factor.

Modgen models require that the appropriate version of Modgen Prerequisites be installed on the workstation. Modgen Prerequisites is available from the Modgen page on the Statistics Canada web site: (www.statcan.gc.ca/microsimulation/modgen-eng.htm)

An Overview of Microsimulation

Overview of Longitudinal Microsimulation Models

This section provides an overview of the types of microsimulation models which can be developed in the Modgen language. This section provides a list of the key modeling elements which are used in all classes of Modgen models. These elements are described in more detail in subsequent sections of this document. In addition, this section will describe a simple example model which is used throughout the guide in order to provide examples of how the key model elements are fashioned into a working model. Note that the source code for the simple model is also provided later in the document.

In its most general terms, microsimulation models attempt to represent the interactions and relationships amongst the micro-level elements in a system. Examples of microsimulation models in the field of economics can be decomposed into cross-sectional models or longitudinal models.

The first set of cross-sectional models in the field of economic policy was related to the personal income tax system. Income tax simulators, which provided estimates of the expected revenue and distributional impacts of tax reform, have a long history in the public service. More recently, social affairs ministries have often developed analogous models for analyzing welfare-like transfer programs. These models, while technically complex, are conceptually rather straightforward. They are based on data for a representative sample of the population at a point in time (typically with annual income data). The rules and provisions of the income tax and/or transfer system are carefully codified and parameterized as algorithms. A simulation then consists of computing for each individual or family in the sample their tax liability and transfer benefits according to either the status quo rules, or the rules under some hypothesized reform. The software then allows comparisons of these 'base' and 'variant' simulations – for example with respect to aggregate costs and revenues, and gainers and losers by such characteristics as income group and family size.

These kinds of cross-sectional models, however, are completely inappropriate for issues in an area like pension policy. The reason is that both public and private pension systems typically pay benefits for many years after retirement (e.g. at age 65), where the amount of benefits depends in a complex way on earnings over the span of decades prior to retirement. Moreover, pension benefits can also depend significantly on marital and fertility histories, as for survivor

and orphan benefits, and also health histories as for disability pensions. The value of applying longitudinal microsimulation models methods to study pension policy questions is obvious. By design, longitudinal model frameworks incorporate both the rules of the pension systems to be analyzed and have the means to generate a dynamic view of a representative population to whom these pension rules (both base and variant) can be applied.

Demographic projections are another area where longitudinal microsimulation modeling has a large potential role. Clearly one key set of inputs to a pension analysis is one or more demographic projections, in turn based on various fertility, mortality, migration and other scenarios. Demographic models are similar to pension models in that relatively simple modeling approaches dominate in comparison to the richness afforded by a longitudinal microsimulation approach.

Microsimulation models in which hazards compete to determine the time of the next event in a person's life are referred to as Continuous Time Models (CTM) since the time interval between events is not fixed. A more rigid type of model which fixes the interval between events is referred to as a Discrete Time Model (DTM). Both of these modeling approaches can be specified in the Modgen language.

The Fundamental Components of any Modgen Model

All models developed in the Modgen language require the specification of the actors in the model and the relationships between the model actors. Model actors are defined by their respective sets of characteristics. These characteristics are either states, which contain the descriptive information of the actor, or events, which transform the values of the states. If, for example, one was modeling a pension system, then the model actors would be persons, spouses, children, and families. The set of states for the person actor would include age, labour force status, level of earned income. The values of the individual change through time as events impact on the individual. For example, the event which dictates when people retire would change the value of the labour force status state.

The relationships between model actors are necessary if the information set of one model actor need be accessed by another model actor. For example, the family actor in our pension example would contain a state stating the number of children present in the family. Assume for the purpose of this example that the amount of pension income entitled by a person is a function of the number of his/her dependents. Therefore, in order for Modgen to calculate a person's pension income, it must access states found in the family actor.

In other instances, actors of the same type must be generated to complete the profile of an actor undergoing simulation. For example, consider a model in which a person actor is about to be married. Modgen will create another person actor which will become the spouse of the original actor. The initial person actor is deemed the dominant person and the spouse is deemed the non-dominant person actor. In order to create the non-dominant person Modgen must create a sequence of potential spouses until one with the appropriate profile is generated. Modgen then discards the person actors who were not selected to be the spouse. These other persons are termed tentative actors. It is critical to understand the difference between dominant, non-dominant, and tentative actors since, without proper filtering, they will all appear in

the tabular reports.

The Modgen language ensures that the actor states and the links between actors are continuously updated through the course of the simulation exercise. In this way the causality of states across model actors are always up-to-date. Users should be aware that there can be many active actors of a certain type in the model specification. For example, there can be more than one child in a family, but only one definition of the actor child is necessary. In these instances, however, Modgen keeps track of these like components in a linked set. Therefore, information from each active child can be accessed in an appropriate manner.

Finally, the simulation of the model is performed on a case-by-case basis. A case is defined as the linked set of actors. The simulation of the case is begun with the creation of each model actor. The simulation of the case is concluded when the last model actor ceases to be active. Modgen will then begin the simulation of the next case. The number of cases to be simulated is given by a control parameter at the time of the model's execution.

Glossary of Important Terms

In order to successfully develop model's in the Modgen language, the user must understand how the basic model elements interact. The following is a list of these key elements. Each of these elements are described in more detail in subsequent sections of this guide.

cases

Modgen simulates one case at a time. A case contains a set of actors with established relationships or links. Modgen maintains these links between actors at all times during the simulation. The number of cases to be simulated is specified as an execution control option.

dominant actors

These elements are at the core of any Modgen simulation exercise. Dominant actors are usually persons or households who are created at the beginning of the simulation process and undergo changes to their characteristics as they proceed through their lives. Dominant actors are defined by their characteristics (states) and by the events which transform their states. Dominant actors are similar to class objects in C++ and the states and events can be thought of as class members.

non-dominant actors

Modgen simulates one case at a time where a set of dominant actors undergo changes to their states. One possible change to a person actor's state is a marriage or a common-law union. When this event has occurred, Modgen generates an appropriate spouse. This spouse, another person actor, is termed an non-dominant person actor. Once created, non-dominant actors undergo the same possible events as the dominant actors of the same type. However, their purpose is to complete the profile of the dominant actor. These actors should be filtered out of all tabular reports.

tentative actors

The process of generating a non-dominant actor in Modgen involves generating a sequence of potential candidates. The candidates who are not chosen to be the spouse are termed tentative actors since they have no links to any of the dominant actors in the model. Since, however, these actors are also person actors they must be filtered out of any tabular report.

states

These elements define the characteristics of the actors over the span of their lifetimes. States are created through a variety of assignments which are described later. States can be scalars or arrays and are cast using either standard C++ types or Modgen's own types.

model links

These elements allow the access of states between model actors. They are analogous to symmetric C++ pointers between classes.

model parameters

These elements are exogenous to the model and are found in .DAT files which the model's .exe reads during the execution phase. Model parameters generally specify the components of the simulation exercise. These may include the eligibility rules for a pension scheme, or the estimated participation rate in a new social program. Model parameters can be scalars or arrays and are cast using either standard C++ types or Modgen's own types.

types

These elements define the nature of states, and model parameters. Types can be of the standard C++ variety (such as int, double) or a special Modgen type (such as range, logical, partition, or classification).

events

These elements are special function prototypes in which Modgen alters the characteristics of the actors through modifications to their states.

function prototypes

These elements are standard C++ functions which are called by the event functions. They also form the basis of the simulation control.

execution control options

These elements control the nature of the model simulation NOT the elements of the program being simulated. Examples include the number of cases to be simulated, the starting random number seed, or a weighting factor for the cases to be simulated.

state initializor

This element allows for the initialization of a state to take place within the definition of the actor with which it is associated.

identity expression

This element is used to maintain the identities between states at all times through the simulation exercise. These elements can only appear in the definition of the actor.

special Modgen functions

These elements are sometimes used in identity expressions to establish the relationships between states.

A Simple Model Example

This guide includes many examples of code from a simple longitudinal microsimulation model called “simpex” in order to illustrate many of the Modgen language elements. In this model, a single individual lives in a dwelling. The physical disability status of the individual (one of none, mild, moderate, or severe) is initialized to “none”. This disability status may change once the individual reaches the age of 60. The individual’s annual income is based on his/her level of disability ($10,000 annually of not disabled, $7,500 annually of mildly disabled, $2,500 for moderately disabled, and $0 in years in which the adult is severely disabled). The individual incurs cost associated with the upkeep with his/her dwelling. An annual balance for the dwelling is calculated by subtracting a constant annual shelter cost from the annual income of the adult.

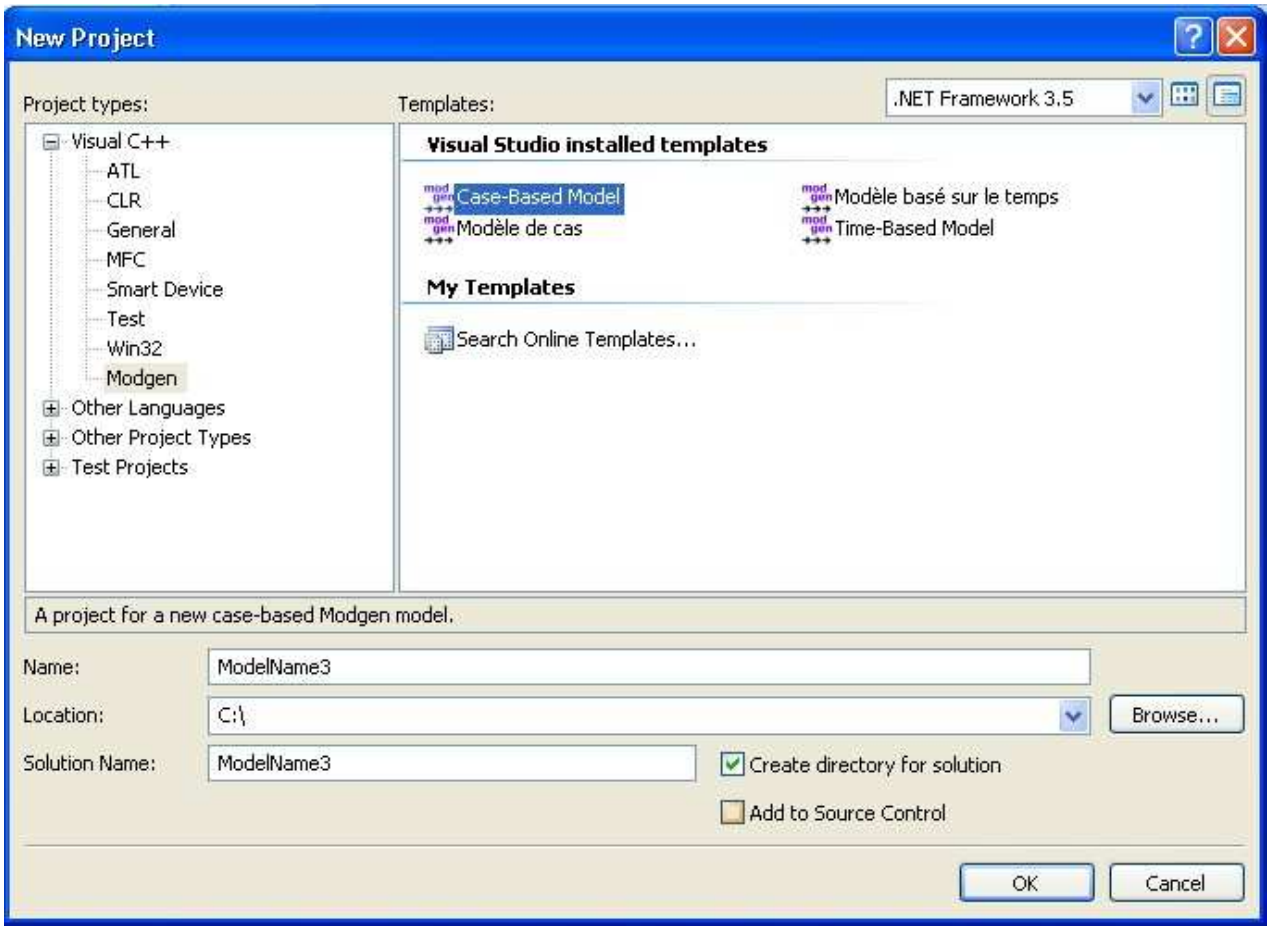
This model is formulated in Modgen as follows. First, there are two model actors: dwelling and person. The dwelling actor contains the states dwelling_cost and a single event, which is invoked annually, calculates the dwelling_balance state. The person actor contains the states for his/her sex, earnings, and disability status. The person never marries and, therefore, no tentative person actors need be generated in this model. The age of the person is handled automatically by Modgen. There are some additional states in the person actor which are used to report other information via the cross-tabulation facility. As with the dwelling actor, the person actor contains one event function, which is called annually, to calculate the disability status and the annual income of the person.

Creating a Model

New Model Wizard

Modgen includes a wizard that helps to set up the underlying foundation or structure for either a new case-based model or a new time-based model. To use the wizard, first open Visual Studio 2008, choose **New** on the **File** menu, and then choose **Project**. (This release of Modgen requires that Microsoft Visual Studio 2008 be installed on the same machine so that models can be compiled. Further details on installing and running Modgen under Visual Studio 2008 are found in "Appendix: Modgen and Visual Studio 2008")

The following ***New Project*** dialog box appears:



To start the wizard for a new Modgen project, you should ensure that 'Modgen' is selected as the project type (under the Visual C++ category) and that either 'Case-Based Model' or 'Time-Based Model' is selected as the corresponding Visual Studio installed template. Next, enter a Name for your model or project, which by default also becomes its Solution Name, and click **OK.**

This in turn leads you to a Welcome screen for your new model on which you can specify the name of the model's main actor, the model's main language (English or French), and an indication of whether or not the model is bilingual. (Actors and the language of a model are discussed further in the topic "Elements of the Modgen Language".) By clicking **Finish** on this latter screen, the underlying structure of the new model is generated, as a .sln file. The .sln file includes another file, readme.txt, which outlines the exact contents of the .sln file.

Organizing the Model Code in .mpp Files

Organizing the model code, model parameters, and model scenarios is crucial to any successful application.

Defining the simulation model in the Modgen language involves specifying actors, model parameters, states, events,

supporting C++ functions, main simulation functions, and report generation. All of these model components are located in .mpp files which are passed to the pre-compiler. An .mpp file, “module_name.mpp”, contains all the necessary definitions of model parameters, logicals, and classes as well as components of the actor definition and event functions associated with a particular module. The module could be demography, mortality, loans calculation, education progression, etc. Remember that Simulation and CaseSimulation, which define the simulation, should also be in one of these modules.

Modgen makes no restrictions on the number of .mpp files which are processed by the pre-compiler, nor are there restrictions governing how these elements of the model are organized in the .mpp files. There is one limitation, however--the name of any .mpp file cannot be the same as an actor, parameter, state, event, etc. For example, if the model has an actor named “Person”, then the model is not allowed to also include a file named person.mpp.

All modules of a model also indirectly include a file named custom.h. If the model developer does not create the file custom.h in the model directory, a default empty version is used during C++ compilation. The file custom.h is useful for models that contain C++ modules coded directly by the model developer, or in models that contain global functions coded directly by the model developer that need to be accessible in all modules of the model. In particular, custom header files that declare global names defined in custom C++ modules should be included in the file custom.h, rather than directly in .mpp files.

Overall, all modules of the model must exist in the same folder and have the .mpp file extension. Modgen in turn parses all files from that working folder which have the extension .mpp.

Organizing the Model Parameters in .dat Files

Modgen has no limit to the number of .dat files it can read during its execution phase. Therefore it is a good practice to group related model parameters together in .dat files. Generally, it is a good idea to create a .dat file containing all model parameters associated with a particular function. For example, if there are model parameters which pertain to education, then a scenario_name(Education).dat file is the natural place to store these parameters. The same argument stands for parameters which are associated with earnings.

Organizing Models and Scenarios

It is critical to work from a copy of the model during any analysis which requires modification to the .mpp files. This is prudent since mistakes will not force the user to re-install the model from the setup program. This also enables the user to compare the results of the user-defined model version with the shipped version. Modgen manages specific scenario inputs and outputs using scenario files. All inputs (.dat files) and outputs (.mdb, .xls, .txt files) are prefixed with the scenario file name.

Organizing Descriptions and Notes for Multilingual Models

Modgen allows for the implementation of multilingual models. Any Modgen description or note can be documented in any number of languages, with the information compiled into the model executable and optionally sent to the Modgen documentation database. Tracking database files have also been designed to hold multiple language desciptors enabling dynamic language switching within the Modgen BioBrowser.

Choose one or more languages by entering the following languages definition statement into an .mpp file (preferably within the .mpp file that contains your model_type statement):

Syntax:

languages

{

langcode, // name

…

langcode // name

};

For example:

```
languages                                                    {
    EN,                                  //                 English
    FR                                   //                 Français
};
```

A label or note entered without the language code will apply to the default language (the first language entered in the languages definition statement above). If a label or note is not supplied in a selected runtime language, the label or note entered for the default language will be used.

For more information on notes, see “Symbol Notes”

Developers who are creating multilingual models can generate the translations they need for Modgen’s internal strings by translating the contents of either the ModgenEN.mpp or ModgenFR.mpp files, both of which form part of the standard Modgen distribution . For a further discussion, see “Using Other Languages” on page 60.

Creating a Model Executable

Creating a new model executable file is necessary when modifications to the underlying model algorithms or table specifications are made. An executable file is created in two steps. In the first step, the Modgen.exe program converts the .mpp files into C++ model code. A corresponding .cpp file is created for each .mpp file. The second step invokes the C++ compiler and transforms the C++ model code into a model executable file.

With Modgen 10, these steps to generate a model's executable file are performed within the Microsoft Visual Studio 2008 programming environment. Details on using Modgen 10 in this environment are discussed in the "Appendix: Modgen and Visual Studio 2008" (Note also that a Modgen 9 model executable would have been generated using Microsoft Visual Studio 2005; the process to migrate such models to Modgen 10 requires the procedures outlined in the "Converting Modgen models to Visual Studio 2008" section of the same Appendix.)

It is also possible to run the Modgen.exe program outside the Microsoft Visual Studio environment. The only arguments allowed are the output directory and the language code:

```
Modgen.exe [-D output_directory] -LANGCODE
```

The output directory is an optional argument but the language code is mandatory. Acceptable language codes are "EN" for English and "FR" for French. The output will be displayed in the corresponding language.

How Modgen Code Becomes a Model Executable File

Modgen requires that a new executable file be built for each modification to a model specification. In this way, the speed in which cases are simulated can be maximized. In comparison with similar models written in interpretive languages, Modgen can simulate 10 to 100 times more cases per second. In order to achieve these speeds, Modgen uses a pre-compile approach to convert Modgen commands into instructions understood by the C++ compiler.

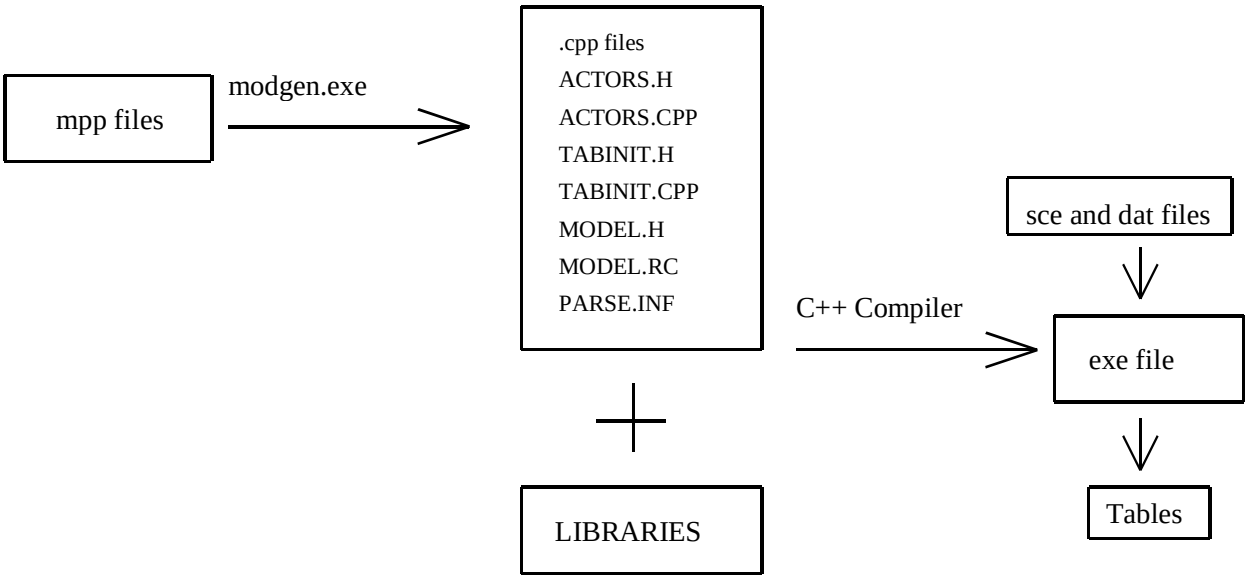
In every specification, the Modgen elements and C++ code which define the specifics of the model are contained in a set of .mpp files which are input to the Modgen.exe pre-compiler. In this pre-compilation phase, the Modgen.exe converts the Modgen elements into standard C++ function calls and instructions. All symbols or elements declared by the developer in these .mpp files are protected via a C++ "namespace" called mm (originally, modgen_model), thus eliminating any conflicts between names of symbols used in a model and names used internally by Modgen or by Microsoft. However, any declarations within the custom.h file are global, i.e. they are not by default contained within this namespace. To include a declaration within the mm namespace, you need to place it within a "namespace mm()" statement.

The output of the pre-compilation process is a set of .cpp files and a set of special files ACTORS.H, ACTORS.cpp,

TABINIT.H, TABINIT.cpp, MODEL.H, MODEL.RC and PARSE.INF. An empty version of file custom.h is also generated if the developer did not create such a file specifically for the model. These files plus a set of C++ and special Modgen libraries are then passed to the C++ Compiler to generate the application's executable file.

All .cpp files and the MODEL.RC file must be added to the project files for compilation to proceed successfully. All .mpp and .dat files can also be inserted into the project although they are not essential for compilation.

The following diagram shows the relationships between various files needed to generate a simulation model executable file for the application. It also shows the inputs and outputs required by the model executable.



Copyright Notices

As a model developer, you can add a copyright notice to your model for each of the languages allowed by the model. Modgen enables this by supplying a string, S_COPYRIGHT1 whose contents can be specified in an .mpp file. Two lines of text are allowed for each notice, and "\n" is permitted as a line separator, as can be seen in the following example showing the definition of both an English and French copyright notice:

string S_COPYRIGHT1; //EN LifePaths™ published by authority of the Minister responsible for Statistics Canada.\n© 1998-2009 Minister of Industry.

string S_COPYRIGHT1; //FR LifePaths™ est une publication autorisée par le ministre responsable de Statistique Canada.\n© 1998-2009 Ministre de l'Industrie

Creating a Custom Icon for your Model

Modgen distributes an APP.ICO file which is copied to the model's directory (if not found there already) when Modgen is run. To create a custom icon, modify the APP.ICO file directly or replace the file with your own customized icon file. Note that the icon is not visible in the Resource View of the project; in fact, none of Modgen's internal resources are visible to the model developer.

After modifying the APP.ICO file, a re-run of Modgen is required. Note that each run of Modgen results in re-compilation of resources and re-linking of the model

Running a Model Simulation

Running the Model Executable

When invoked, a compiled model provides the model user with a generic visual interface for editing parameters, running the simulation and browsing the simulation outputs. This interface is fully described in a separate publication, the Guide to the Modgen Visual Interface; however, for developers, a brief description is provided here. At present, developers must provide model users with an initial set of .dat files necessary for the model to run. This initial set is often refered to as the base case values, which are simply the default values chosen for model users by the model developer.

A compiled model executable will not launch a simulation without a saved scenario file (.sce extension). A Modgen scenario file is an ini style text file containing the execution control options necessary to run a simulation. There are two ways to run a simulation for an existing model. Firstly, from the visual interface by invoking the executable from Explorer (or Start/Run) without any arguments or with a previously saved scenario file. Once in that environment, select Run from the Scenario menu. Use Execution Control within that environment to specifiy simulation options.

Secondly, from a DOS window directly on the command line by supplying the scenario file with the –sc command line argument.

The model can also be run in documentation-only mode as described in syntax 3 below.

Syntax1: Invoking the model's visual interface

modelexe_name <scenario_filename>

no other arguments are permitted.

eliminating the argument altogether will invoke the visual interface with no open scenario file. This option is particularly useful to invoke the model by simply double-clicking on the file name within Explorer.

use Scenario/Run to launch the simulation.

Syntax2: Running the model for batch execution

Runs the model with execution control options taken from scenario_filename and provides a command line override for computational priority. In this mode, the simulation is started in the minimized mode.

modelexe_name –sc scenario_filename <-computational_priority>

the computational priority is optional and can be one of: normal, low or high.

no other arguments are permitted.

Syntax3: Running the model in documentation-only mode

This syntax creates the documentation database described in Modgen Documentation Database. There is no user interface while the model runs in documentation-only mode.

modelexe_name -doc <filename> <-s> (e.g. lpaths -doc MyDoc.mdb)

<> indicates an optional argument. If "filename" is not specified the documentation database is named: modelexe(doc).mdb (e.g. lpaths(doc).mdb).

If "filename" is specified without extension, .mdb will be added.

The run should not take more than a few seconds and then a message box appears indicating successful completion or failure. The message box can be disabled if the "-s" option (silent mode) is specified.

Initializing Model Parameter Values in .dat files

Values for the model parameters are kept in .dat files. The visual interface will name the .dat files according to the scenario name as follows: scenario_name(file_name).dat. The rank and shape of the parameter and the meaning of each of its dimensions, if any, must be defined in both the .mpp file and the .dat file. Typically, the size of the model parameter is defined by classifications or ranges. The values of the parameters must be expressed in the following ways:

Syntax for Model Parameter Initialization in the .dat files

```
parameters
{
        type    name[size_dim1]...[size_dimn] = { value, value, .... , value }
                                                or
        type    name[size_dim1]...[size_dimn] = ( repeater) { value, value, .... , value }
};
```

Repeaters can be nested within blocks of data values. Regardless of the number of parameter dimensions, the values for

parameters are always entered as vectors in row-major order (i.e with the lowest dimension values, the columns, changing fastest and the higher order dimensions changing the slowest). Each value is entered by cycling through the rightmost dimension values first. When these values have been used up, the next rightmost values are entered. This continues until all the values have been entered. For example, suppose your parameter has 2 dimensions (rank 2), has shape 2 by 3 and contains the following values:

```
3    4    5
7    8    9
```

then, your .dat statement would look like:

```
param[dim1_2values][dim2_3values] = { 3, 4, 5, 7, 8, 9};
```

Example1

Trailing commas are allowed (but not required) in the parameter initialization. Therefore, both of these examples are valid.

```
param[dimension] = { 1, 2, 3 };
param[dimension] = { 1, 2, 3, };
```

Example2

Parameter initializers may contain repeated blocks of values. The blocks can be nested.

```
param[dim] = { (15) 0, 1, 2, 3, 4 };
param[dim] = { 0, (2) { 1, 2, (3) { 3, 4 }, 5, 6  }, }
```

The second example is equivalent to:

```
param[dim] = { 0, 1, 2, 3, 4, 3, 4, 3, 4, 5, 6, 1, 2, 3, 4, 3, 4, 3, 4, 5, 6
};
```

This simplest method for parameter editing within an existing model is to use the visual interface.

However, if the parameter consists of one number (scalar) or a short vector of numbers, you can open the .dat file with your preferred text editor and change it directly. Do not modify the parameter definition portion of a .dat file when changing a parameter value.

Example

In this example, we change the value of the scalar parameter, FemaleProp from 0.5 to 1.00 to ensure only females are simulated.

The lines in the person.dat file

```
// parameter specification
parameters {
float    FemaleProp = 0.5;
};
```

get changed to

```
// parameter specification
parameters {
float    FemaleProp = 1.0;
};
```

Changing a range of model parameter values for a higher-dimensional array using a text editor directly on a .dat file is not recommended. With the dimensionality removed, it is difficult to tell which element of the parameter you are changing.

These values are typically taken from an externally created file and pasted in, for example, from a spreadsheet. Create them once in the .dat file syntax or initialize them to zero, then use the visual editor thereafter.

Modgen Model Output Files

Modgen manages model run outputs by using the scenario name followed by a bracketed identifier indicating the type of Modgen output and an extension indicating the type of file. The files can be grouped into 3 categories:

Table outputs for aggregate model results:

scenario_name(tbl).mdb – MS Access output (always produced by a model run and used by the visual interface for table browsing).

scenario_name(tbl).xls – Excel table outputs. This is the default name used by the table export feature of Modgen. Create this file using the menu item Scenario/Export table... from the visual interface. Several formats for this type of output are available.

scenario_name(tbl).txt – Text table outputs. This is the default name used by the table export feature of Modgen. Create this file using the menu item Scenario/Export table... from the visual interface.

Tracking outputs for examining individual life histories:

scenario_name(trk).mdb – MS Access tracking output used by the Modgen BioBrowser. This file is only produced if selected by the user in the Execution control dialog.

scenario_name(trk).txt - Text tracking output. This file is only produced if selected by the user in the Execution control dialog.

Log and error outputs:

scenario_name(log).txt – Modgen log file (always produced, if errors are encountered it will also include the error messages).

scenario_name(err).txt – Modgen error file (produced only if errors are encountered when parsing the input files).

scenario_name(trc).txt – Modgen file containing much extra information about errors in case a model crashes unexpectedly (deleted if no errors are encountered).

For more information on usage of these files, refer to the Guide to the Modgen Visual Interface.

Elements of the Modgen Language

Overview of Modgen Elements

This section describes the elements of the Modgen language which are not part of the standard C++ language. These elements greatly facilitate the representation of a microsimulation model in a structured programming environment. Almost all Modgen elements must be defined before they appear in simulation code. Therefore, the definition of these elements not only allows for the successful compilation of the model, but also serves as a source of documentation of how the elements work together.

Defining the Kind of Microsimulation Model

The first and most critical element to be defined by the application developer is the specification of the type of model being constructed. The modeler defines the type of the model by using one of the following statements (recommended placement: top of the main .mpp file):

```
model_type case_based<,just-in-time>;
model_type time_based<,just-in-time>;
model_type cell_based;
```

The visual interface will vary according to the model type. For more information, refer to the Guide to the Modgen Visual Interface.

The just_in_time keyword is optional and affects the advancement of time when an event occurs. For more information, see the discussion of “Events” on page 97.

For case based models, the time_type statement must be used to distinguish a discrete from a continuous model. The time_type statement which is generally found in the .mpp file containing all model definitions can be any of char, short, int, long, uchar, ushort, uint, or ulong for Discrete Time Models, and double or float for Continuous Time Models. (The types uchar, ushort, uint or ulong are synonyms for unsigned char, unsigned short, unsigned int, and unsigned long). Modgen supplies double as the default for the time_type element if none is specified. For time based models, time_type

must be set to double.

Modgen defines the state age through this definition. Users need not specify an age state in any of the actor definitions even though no microsimulation model would be complete without it. Modgen creates this state automatically.

Modgen also defines the type TIME through this definition. This type is often used to define states which are closely linked to the application's concept of time.

Example

```
// Define the time type for the simple model example.
// Since this is an annual model, or equivalently a discrete time model,
// the model type is int
time_type int;
```

For continuous time models, specify:

```
time_type double;
```

or

```
time_type float
```

Unlike other model types, a cell-based model is considered to be in the simulation stage when processing the user tables (in the UserTables function). The progress update is done by the model's code (either in the Simulation or the UserTables function) by calling the following Modgen function:

```
void CalculationProgress(double dPctDone);
```

Defining the Actors

The actor definition contains the list of all states, events, and supporting functions associated with the actor. As mentioned above, the age state need not be specified by users in this definition as Modgen automatically supplies this state. Although the actor definition lists all events and supporting functions, the source code for these modules, which can make use of all elements of C++, is outside of the actor definition. The syntax of the definition is as follows:

Syntax

```
actor name // label
```

```
{
type  state;                  // state name (simple cast)
type  variable[class];        // array variable name (simple cast)
type  state = identity_expression;  // state name (identity expression)
type  state = {value};  // state name (state initializor)
type  state = supporting_function() // state name (through a C++ function)
type  state = MACRO()   // state name (through a C++ macro)
event function                // event function associated with the actor
function_prototype            // C++ functions which are called by events or the
simulation engine
}
```

For more information on events and supporting functions, see “Events” on page 97.

In our simple worked example, there is only one actor which is the individual. The model simulates the disability status of the individual over the course of his lifetime. As we have information, in the form of a model parameter, on the transitions from one disability status to another by age and sex, we also need to define a state which holds the age, sex, and disability status of the individual. Modgen supplies the age state automatically. Therefore, it is not part of the actor definition below.

```
//   definition of the person actor
// in this simple example model, there is only one event for the person
// actor which occurs once a year to calculate the person's disability status
// and corresponding annual income conditioned on the person being alive.
// This event, is called AgeOneYearEvent.
// To estimate the waiting time between the execution of this event
// we employ the timeAgeOneYearEvent function

actor Person            // Individual actor definition
{
   // states required for this specific simulation
   // note: the age state is automatically generated by Modgen
```

```
   SEX   sex;                          // Sex
   int   earnings = {0} ;              // Lifetime earnings to date
   DFLE_STATE   dfle = {NO_DIS};       // DFLE State
   LIFE_STATE   life_status = {ALIVE}; // Life Status

   // states required for tabular reports
   int   person_year = duration (life_status,ALIVE);   // Person year
   int   age_at_death = value_at_changes ( life_status, age ); // Age at death
   int   death = value_at_entrances ( life_status, NOT_ALIVE, age ); // Age at
death

   // functions called by the Dwelling event function
   event timeAgeOneYearEvent, AgeOneYearEvent;

   // functions required by the simulation engine
   void Start();
   void Finish();
};
```

Default argument values for actor function prototypes

Modgen supports default argument values for actor functions. Functions declared with default argument values can be called with fewer arguments than required by the function prototype.

Example

```
actor Person {

   void   Start( logical bTentative, logical dom,  TIME birth = TIME_INFINITE,
      TIME immig = TIME_INFINITE, SEX eSex = MALE, PROVINCE eProv = ONT );
```

```
};
```

```
prDominant->Start( FALSE, TRUE ); // defaults used for other 4 arguments
```

Actor Sets

Actors can also belong to actor sets. An actor set is a collection of actors dynamically maintained by Modgen, i.e. Modgen internally handles membership in the actor set based on criteria provided by the model developer.

Actor sets can be useful in time-based models with many interacting actors. Actor sets are intended to facilitate the development of such models by eliminating the need for the developer to write code to maintain and manipulate the sets. For example, Modgen provides functions that can be used to select an actor randomly from the set, to iterate over all members of the set, etc.

Actor sets are discussed in much more detail in the topic “Actor sets” on page 128.

Defining the States

States describe all of the actor’s characteristics. In C++, they are data members of actor objects. Declare states in the actor definition for each actor object. States are always scalars and must be typed as one of int, long, float, double or one of Modgen's special types: TIME, logical, classification or range. Values contained in the states are assigned either through the evaluation of an identity expression in the actor definition, the evaluation of a C++ expression in an event, or the evaluation of a C++ expression in a C++ supporting function called by an event.

States are defined by a simple cast, state initializer, an identity expression, a call to a supporting function, or a C++ macro expression. States should always be initialized using braces. If braces are not used, the implication is that the state is an expression state whose value is maintained by Modgen at all times. Any statement in the model code that attempts to modify an expression state is ignored, although Modgen does provide a warning in such situations.

When used, the identity expression can contain a simple assignment, an arithmetic identity, a C++ boolean operation, the C++ conditional-expression operator (?:), or the result returned by a Modgen special function. Modgen automatically updates the value of each state every time a supporting function alters a state on which it is conditioned throughout the course of the simulated lifetime.

Note well that any other types (e.g. multi-dimensional arrays) can be declared in the actor definition, but will not be treated by Modgen as a state, these declarations become simple data members of the object class.

Modgen defined states

Modgen creates the following special states for each actor:

actor_id

The actor_id state is a unique identifier for an actor that is generated by Modgen for each actor in the model. It can be read in model code but not modified. Note that while actor_id uniquely identifies each actor in a simulation, it should not be used as an identifier to compare actors across different simulations. Its value is guaranteed to be unique only within a specific simulation.

actor_subsample_weight

The actor_subsample_weight state is equal to actor_weight multiplied by the number of sub-samples. This state indicates the weight applied to sub-sample quantities (actor_weight is applied to total quantities). The “actor_subsample_weight” state may be tracked and displayed with the BioBrowser.

actor_weight

A special state called “actor_weight” is available for every actor. Calling the SetCaseWeight function sets the value of “actor_weight” state for every actor in the case. The “actor_weight” state may be tracked and displayed with the BioBrowser.

age

The age state indicates the age of the actor and is defined by the model type TIME which is dependent on the model type time_type. This state can be set only in the actor's Start() function, if not set there it will be initialized to zero. Thereafter, it can only be referenced in model code.

case_id

The case_id state contains a unique identifier for each case and is of type long. For case-based models, this state identifies the current case number (origin 1). For time-based models, it is not used. The case_id state is always tracked in all case-based models. It can never be set only referenced in model code.

case_seed

The case_seed state indicates the starting seed of the current case and is of type double. Note that two random number generators are used for each subsample, one to generate the starting seed for each case, one to generate the stream of random numbers for the simulation. This results in more consistent model runs, as an extra call to the random number generator in one case will not affect the starting seed for next case. It can never be set only referenced in model code.

time

The time state indicates the current time and is typed by the model type TIME. This state is also used for the state tracking which is part of the model debugging facility. This state can be set only in the actor's Start() function, if not set there it will be initialized to zero. Thereafter, it can only be referenced in model code.

Modgen also generates labels internally for these states in both English and French, as per the following.

	English label	French label
actor_id	Actor identifier	Identificateur d'acteur
case_id	Case identifier	Identificateur de cas
case_seed	Case random seed	Valeur aléatoire d'amorçage de cas
time	Time	Temps
age	Age	Âge
actor_weight	Weight	Poids
actor_subsample_weight	Sub-weight	Poids du sous-échantillon

Model defined states

Any number of states can be declared in the actor's defintion. They must be scalars. Numeric states in Modgen can be declared with the standard C++ types of int, long, float or double. The types char and short can also be used for small signed integers. In addition, the types uchar, ushort, uint and ulong are available for unsigned integer values, where uchar is a synonym for the C++ type unsigned char, ushort is a synonym for the C++ type unsigned short, etc. Modgen allows the following additional type declarations for states:

classification

Categorical states are defined through a classification. Such states can only be initialized using a value from the appropriate classification (otherwise, a C++ compiler error will be generated that refers to the mpp source file and line where the classification state is defined and initialized.) The type declaration contains the classification name as in the example below.

Example

```
// The state gender is declared using the classification SEX
SEX gender; // Gender
```

logical

Boolean states used in the definition of an actor are defined using the logical **t**ype. The values supported for logical states are FALSE (zero) and TRUE (non-zero).

Example

```
// The state, resident, is true when the actors age is
// greater than the age of immigration
logical resident = age >= age_imm;
```

range

States which hold a value defined by specific numeric interval are defined through a range**.**

Example

```
range LIFE  { 0, 99 };  // duration of life
```

real, counter, and integer

These types can be used to declare actor states that contain floating point values (real), non-negative integral values (counter) or integral values (integer). The actual C++ types used to represent these states are controlled using the real_type, counter_type, or integer_type statements, respectively. These types and type statements can be useful in models where memory utilization is important, such as time-based models with many actors. For such models, developers may wish to consider declaring all floating point actor states as real, certain non-negative integral states as counter, and all integral actor states as integer. The corresponding type statements can then be used to optimize memory usage. The following table lists further characteristics for each of these three types

	type real	**type counter**	**type integer**
C++ types allowed in type statement (with storage used by each type)	float (uses 4 bytes of memory, has 6 decimal digits of accuracy) double (uses 8 bytes of memory, has 15 decimal digits of accuracy)	char or uchar (1 byte) short or ushort (2 bytes) int or uint (4 bytes) long or ulong (4 bytes)	char (1 byte) short (2 bytes) int (4 bytes) long (4 bytes)
Default C++ type if model has no type statement	double	int	int

Example of type statement and type declaration in actor definition	`real_type float;`
	`actor Person {`
	`  real earnings;`
	`};`
		`counter_type ushort;`
		`actor Person {`
		`  counter jobs;`
		`};`
			`integer_type short;`
			`actor Person {`
			`  integer gains_and_losses;`
			`};`

Comments on optimal use by model developer	Use double for maximum accuracy or float to minimize memory use	Developer must ensure that maximum values attained by actor states fall within the C++ limits for the data type specified in the counter_type statement	Developer must ensure that maximum and minimum values attained by actor states fall within the C++ limits for the data type specified in the integer_type statement

TIME

The TIME type has special meaning in Modgen. This model type takes its value from the time_type cast which sets the type of simulation model being specified. States which are associated with the time concept of the model are defined with the TIME type.

Derived State Expressions Used in Actor and Table Definitions

Modgen includes a collection of special derived states which record various conditions of states at every point in the simulation. These special states are used for report generation and state assignments in Modgen expressions.

The arguments to these expressions may be a state which is a classification type, a range type, or a regular state of type int, or long. The state argument associated with this must be of the corresponding type.

Examples

```
undergone_entrance (dfle, SEVERE_DIS)
undergone_entrance (children, 1)
```

Derived states associated with changes to actor states

Listed in alphabetic order:

changes(state)

The number of times the classification state has changed so far in the simulation.

e.g. changes (dfle) records the number of times the disability status has changed. Consider a person who survived to age 77 and whose disability status changed every 10 years from the age of 15 onwards. The value of *changes* would be zero until the person turned 15 years old, 1 until the person turned 25 years old, 2 until the person's 35 birthday, and so on. At the age of death *changes* would equal 7.

entrances(variable**,** state)

The number of times the variable has entered the state to that point in time in the simulation.

e.g. entrances (dfle, SEVERE_DIS) records the number of times an individual has entered the severe disability state to that point in the simulation. If a person survived to age 75, and his disability status entered the severe state at age 25 and 60, then the value returned by *entrances* is 0 until he turned 25 , 1 until he turns 60, and 2 for the remainder of his life.

exits(variable**,** state)

The number of times the variable exited the state so far in the simulated lifetime.

e.g. exits (dfle, SEVERE_DIS) records the number of times an individual has exited the severe disability state to that point in the simulation. If a person survived to age 75, and his disability status entered the severe state at age 25 and 60 and exited the severe status at 31, then the value returned by *exits* is 0 until he turned 31 and 1 for the remainder of his life.

undergone_change(variable)

The value of this function is 1 if the variable has changed its state by that point in the simulation and 0 otherwise.

e.g. Consider a person who survived to age 75 and who became disabled at age 68. Then, the value of undergone_change(dfle) is 0 until he turns 68 year old and 1 for the remainder of his life.

undergone_entrance(variable**,** state)

The value of this function is 1 if the variable has entered the state by that point in the simulation and 0 otherwise.

e.g. Consider a person who survived to age 75 and who became severely disabled at age 68. Then, the value of undergone_entrance(dfle, SEVERE_DIS) is 0 until he turns 68 year old and 1 for the remainder of his life.

transitions(variable, from state, to state)

The number of times the variable has entered the "to state" from the "from state" so far in the simulated lifetime.

eg. transitions (dfle, MILD_DIS, MEDIUM_DIS) records the number of times an individual left the mild disability state to enter the medium disability state so far in the simulation.

value_at_changes (variable, recorded variable)

The sum of the values of the recorded variable when the variable changes. This function is used, in conjunction with *changes*, to calculate the average value of a recorded variable as the variables has changed so far in a simulation.

e.g. *value_at_changes* (dfle, age) records the sum of each age at which the disability status changes. If a person survived to age 77, and his disability status changed every 10 years from age 15 onwards, then the value returned by the function at the age of death is 15+25+35+45+55+65+75 = 260. The average age of change would be given by dividing this result by the number of changes: 260/7=37.1

value_at_entrances (variable**,** state**,** recorded variable)

The sum of the values of the recorded variable at which the value of variable entered the state. This function is used, in conjunction with *entrances*, to calculate the average value of a recorded variable for entrances into a state over the

simulated lifetime.

e.g. value_at_entrances (dfle, SEVERE_DIS, age) records the sum of each age at which the disability status entered the severe state. If a person survived to age 75, and his disability status entered the severe state at age 25 and 60, then the value of *value_at_entrances* is 0 to his 25th birthday, 25 to his 60 birthday, and 85 until the time of his death.

value_at_exits (variable**,** state**,** recorded variable)

The sum of the values of the recorded variable at which the value of variable exited the state. This function is used, in conjunction with *exits* to calculate the average value of a recorded variable for exits out of a state over the simulated lifetime.

eg. value_at_exits (dfle, SEVERE_DIS, age) records the sum of each age at which the disability status left the severe state. If a person survived to age 75, and his disability status entered the severe state at age 25 and 60 and exited the severe status at 31, then the value of *value_at_exits* is 0 to his 31st birthday, and 31 until the time of his death.

value_at_first_change(variable, recorded variable)

The value of the recorded variable at which time the value of variable first changed. If the variable has not yet changed its the state at that point in the lifetime, then the function has a value of zero.

e.g. value_at_first_change (dfle, age) records the age of a person at which the disability status first changed. If a person survived to age 70, and his disability status changed at age 25 and 60, then the value of *value_at_first_change* is 0 until he turned 25 and 25 for the remainder of his life.

value_at_first_entrance(variable**,** state**,** recorded variable)

The value of the recorded variable at which time the value of variable first entered the state. If the variable has not yet entered the state at that point in the lifetime, then the function has a value of zero.

e.g. value_at_first_entrance (dfle, SEVERE_DIS, age) records the age of a person at which the disability status first entered the severe state. If a person survived to age 70, and his disability status entered the severe state at age 25 and 60, then the value of *value_at_first_entrance* is 0 until he turned 25 and 25 for the remainder of his life.

value_at_transitions (variable, from state, to state, recorded variable)

The sum of the values of the recorded variable at which the value of variable left the from state and entered the to state. This function is used, in conjunction with *transitions*, to calculate the average value of a recorded variable as the variables has changed so far.

eg. value_at_transitions (dfle, MILD_DIS, MEDIUM_DIS, age) records the sum of each age at which the disability status left the mild state and entered the severe state.

value_at_latest_change(variable**,** recorded variable)

The value of the recorded variable at which the value of variable last changed. If the classification variable never changed, then the function returns a zero.

e.g. value_at_latest_change (dfle, age) records the age of a person at which the disability status last changed its state. If a person survived to age 75, and his disability status changed age 25 and at 60, then the value returned by

value_at_latest_entrance is 0 to his 25 birthday, 25 to his 60th birthday, and 60 until the time of his death.

value_at_latest_entrance(variable**,** state**,** recorded variable)
The value of the recorded variable at which the value of variable last entered the state. If the classification variable never entered the classification state, then the function returns a zero.
e.g. value_at_latest_entrance (dfle, SEVERE_DIS, age) records the age of a person at which the disability status last entered the severe state. If a person survived to age 75, and his disability status entered the severe state at age 25 and at 60, then the value returned by value_at_latest_entrance is 0 to his 25 birthday, 25 to his 60th birthday, and 60 until the time of his death.

Derived states associated with durations of actor states

The following expressions accumulate the values of a variable over the range of time an actor was in a specified state. When these expressions are used inside a table specification, then Modgen duration is only calculated for the time the actor is in the domain of the table. For example, if a table filter included residency status, then the duration of immigrant actor would only count the time the actor was a resident.

A subset of these functions or expressions operate on states over "spells" (i.e. periods of time over which a state assumes a specific value). These spell functions fall into two classes, those that operate on a "current" or "active" spell, and those that operate on a "completed" or "previous" spell. These two classes of functions behave very differently. In particular, the "active" functions are saw-tooth in form, returning to zero at the end of each spell, whereas the "completed" functions are step-like in form.

active_spell_delta (variable, state, analysis variable)
This reports on the change that the analysis variable has undergone since the variable has been in the state.
eg. active_spell_delta (dfle, MILD_DIS, earnings) records the sum of earnings over the time an individual has currently been in the mild disability state in the simulation.

active_spell_duration (variable, state)
The length of time the variable has currently been in the state from its last transition from a previous state in the simulation. If the actor has been in the same state for its entire lifetime to date, then the value of this function is zero.
eg. active_spell_duration (dfle, MILD_DIS) records the length of time an individual has currently been in the mild disability state in the simulation. Consider a person who has survived to age 75, his disability status entered the mild state at age 20, exited the mild state at 25, returned to it at aged 60, and stayed in this state until the time of his death. The value of active_spell_duration is 0 until he turned 20, 1 when he is 21, 2 when he is 22, and so on until his 25th birthday. At that point active_spell_duration would return to zero, until age 60 where it would be incremented by 1 for each year until he died.

active_spell_weighted_duration (variable, state, weighting variable)

This is a weighted version of active_spell_duration. It could be used to determine, for example, the income earned so far during the current (on-going) spell of employment. This would be done by using the earnings rate variable as the weighting variable.

completed_spell_delta (variable, state, analysis variable)

This reports on the change that the analysis variable has undergone in the last completed spell in which the variable was in the state.

eg. completed_spell_delta (dfle, MILD_DIS, earnings) records the sum of earnings over the previous completed time an individual was in the mild disability state in the simulation.

completed_spell_duration(variable, state)

The length of time the variable was last in the state. If the actor has been in the same state for its entire lifetime to date, then the value of this function is zero.

eg. completed_spell_duration (dfle, MILD_DIS) records the length of time an individual was last in the mild disability state. Consider a person survived to age 75, and his disability status entered the mild state at age 20, exited the mild state at 25, returned to it at aged 60, and stayed in this state until the time of his death. The value of completed_spell_duration is 0 until he turned 25, and 5 until the time of the actor's death.

completed_spell_weighted_duration (variable, state, weighting variable)

This is a weighted version of completed_spell_duration. It could be used to determine, for example, the income earned during a previous spell. This would be done by using the earnings rate variable as the weighting variable.

duration(variable, state) or duration()

The length of time the value of the actor variable has been in the state so far in the simulated lifetime. If the variable and state are not specified, then duration cumulates duration during the existence of the actor. It performs the same function as duration(life_status, ALIVE).

eg. duration (dfle, MILD_DIS) records the length of time an individual has ever been in the mild disability state. Consider a person who has survived to age 75, his disability status entered the mild state at age 20, exited the mild state at 25, returned to it at aged 60, and stayed in this state until the time of his death. The value of duration is 0 until he turned 20, 1 when he is 21, 2 when he is 22, and so on until his 25th birthday. At that point duration would keep the same value, 5, until age 60 where it would be incremented by 1 until the year of his death.

duration_trigger(state, value, elapsed_time)

This function is a logical state informing if the duration of a specific state at a specific value reached a specified time period.

e.g. logical worked6months = duration_trigger(employed, TRUE, 0.5) states the value of the derived state worked6months is TRUE if the individual has been working for at least 6 months, FALSE in any other case. It also implicitly creates an event with the waiting time being the specified time. In the above example, the waiting time would

be 0.5.

duration_counter(state, value, size of time interval);

or

duration_counter(state, value, size of time interval, maximum value);

This function is like duration_trigger(), except that it counts the number of intervals of specified size that have elapsed since a particular state reached a particular value.

e.g. int curtate_age = duration_counter(alive, TRUE, 1)

will count the number of years elapsed since the person is alive, i.e. the person's integerized age. Implicit events will also be created at each birthday.

Due to the creation of implicit events, using duration_counter() can be expensive, since it may continue to be updated (perhaps monthly) long after its usefulness is at an end (e.g. after retirement, or beyond the largest value needed by a time function). For that reason, it is possible to use an optional fourth argument: the maximum count it's allowed to reach. Once it reaches that maximum, Modgen will cease updating the value.

e.g. int curtate_age_below_50 = duration_counter(alive, TRUE, 1, 50)

will count the number of years elapsed since the person is alive as long as that number is lower than or equal to 50. At any point during a person's life, curtate_age_below_50 would be either the age (integer) of the person, or 50.

weighted_cumulation(state1,state2)
"state1" is the state whose cumulation is observed. Whenever the value of "state1" changes the difference is mutiplied by the value of "state2" and added to the derived state. The new state is useful for creating a constant dollar version of some state cumulated in current dollars.

weighted_duration (variable, state, weighting variable)

or

weighted_duration (weighting variable)
This function uses the values of the weighting variable, which is always a rate variable (i.e. earnings rates, wage rates, etc.) to the sum of this variable over the duration of time the variable has, so far, been in the state. This function, essentially, converts a rate variable into a accumulating variable. If the variable and state are missing from the specification, then the value of the function is the sum of the weighting variable to the current point in the actor's lifetime.
eg. weighted_duration (dfle, MILD_DIS, earnings_rate) records the total earnings of an actor while it has, so far, been in the mild disability state. Consider a person who survived to age 75, and his disability status entered the mild state at age 60, left it at aged 65, and his annual earnings are $10,000. The value of weighted_duration is, at age 60, is

10000*1, at aged 61 is 10000*2, and so on to age 65 where the value of this function is 10000*5 = 50000. This value is maintained for the duration of the actor's lifetime.

Other special functions

split(state, partition_name)

This function partitions the state by the classification groups defined in partition_name. See the Tables section of this document for examples.

sqrt(state)

This function returns the square root of the value of the state. It can only be used in table expressions.

Defining the Model Variable Types

The model variable types which, in turn, define the states and model parameters in Modgen, must also be defined. Each model variable type is described below.

Definition of the TIME type

The special definition, TIME, is used by Modgen to distinguish between discrete and continuous models. It is used to declare states that store time values, and to define the return type of several functions. It takes its value from the definition of the time_type model type.

real, counter, and integer types

These three Modgen data types can be used to declare actor states that contain floating point (real type), non-negative integral (counter type), and integral (integer type) values. The actual C++ types that get used in the model are controlled by the real_type, counter_type, and integer_type statements, respectively These type and type statements are described further in the section "Model defined states" on page 26. Note however that none of these types is permitted in a parameter declaration; they may only be used in the declaration of actor states.

Ranges

A range is defined by a range of integer values. It is used for defining the size of array states or model parameters.

Syntax

range name { min, max }; // label

Examples

```
// the range, LIFE, is defined by the integers 0 through 100
```

```
range LIFE      { 0, 99 };      // duration of life
// the range, DFLE_AGE, is defined by the integers 15 through 100
range DFLE_AGE   { 15, 99 };   // age at dfle changes
```

Partitions

A partition defines a set of breakpoints for a continuous state. It is typically used in the specification of model parameter dimensions (as shown below) or in tables and identity expressions as an argument to the split function. You can not use partitions to declare states.

Syntax

partition name { number, ... , number };

where number is a double.

Examples

```
partition AGE_GROUP1 { 15, 20, 25, 30, 35, 40, 45, 50 };
parameters {
    float    AvgIndWage[AGE_GROUP1];  // average industrial wage
};
```

Classification types

A classification type is defined as a collection of levels together with their labels. The classification type may be used to define the dimensionality of a state or model parameter.

Syntax

classification name { // <langcode1> label1 <//langcode2 label2…>
    level, //<langcode1> label1 <//langcode2 label2…>
    level, //<langcode1> label1 <//langcode2 label2…>
    level   //<langcode1> label1 <langcode2 label2…>
};

Examples

```
// the classification, SEX, is defined by its two gender types
classification SEX { // Gender
    FEMALE,    // Female
```

```
    MALE       // Male
};


// the classification, DFLE_STATE, is defined by four disability status
// categories
classification DFLE_STATE { // DFLE State
    NO_DIS,          // No Disability
    MILD_DIS,        // Mild Disability
    MODERATE_DIS,    // Moderate Disability
    SEVERE_DIS       // Severe Disability
};
```

Aggregations of classifications in case-based and time-based models

A special function converts values in one classification to values in a second classification provided the second is a strict aggregation of the first. To perform this transformation a map between the first and the second classification state must have been provided in the mpp source code using the "aggregation" statement, which has the following syntax:

```
    aggregation aggregated_classification, detailed_classification {
        aggregated_item, detailed_item,
        aggregated_item, detailed_item,
        …
        aggregated_item, detailed_item
    };
```

"**aggregation"** is a Modgen keyword, "aggregated_classification", "detailed_classification", "aggregated_item" and "detailed_item" refer to previously defined aggregated and detailed classifications and their items. Note that there is no comma before the closing '}'.

If an aggregation has been defined from classification `XXX` to classification `YYY` and if `stateXXX` is a classification state of type `XXX` then

```
aggregate(stateXXX, YYY)
```

will return the corresponding state value from the classification YYY. Derived states using aggregate() can be used

directly as dimensions of tables, in expressions, and in user functions.

Modgen also creates functions that can be used to map one classification to another. For every pair of classifications XXX and YYY for which an aggregation has been defined, Modgen will create a function XXX_To_YYY(eLevel) that takes as argument a level of classification XXX and returns a level of classification YYY. These functions can be used in PreSimulation() and UserTables() as well as in expressions and in actor functions, but cannot be used as dimensions of tables.

The following example illustrates how this works. Sample results are shown after the code.

```
classification PROVINCE {        //EN Province
      NFLD,  //EN Newfoundland
      PEI,   //EN P. E. I.
      NS,    //EN Nova Scotia
      NB,    //EN New Brunswick
      QUE,   //EN Quebec
      ONT,   //EN Ontario
      MAN,   //EN Manitoba
      SASK,  //EN Saskatchewan
      ALTA,  //EN Alberta
      BC,    //EN British Columbia
      YUK,   //EN Yukon
      NWT    //EN N. W. T.
};

classification REGION {   //EN Region
      R_ATLANTIC,  //EN Atlantic
      R_QUEBEC,    //EN Quebec
      R_ONTARIO,   //EN Ontario
      R_PRAIRIES,  //EN Prairies
```

```
      R_ALBERTA,   //EN Alberta,
      R_BC         //EN British Columbia
};

aggregation REGION, PROVINCE {      //EN Province to region
      R_ATLANTIC, NFLD,
      R_ATLANTIC, PEI,
      R_ATLANTIC, NS,
      R_ATLANTIC, NB,
      R_QUEBEC,   QUE,
      R_ONTARIO,  ONT,
      R_PRAIRIES, MAN,
      R_PRAIRIES, SASK,
      R_ALBERTA,  ALTA,
      R_BC,       BC,
      R_BC,       YUK,
      R_ALBERTA,  NWT
};

actor Person {
      PROVINCE province;      //EN Province
      REGION region = aggregate( province, REGION ); //EN Region
      REGION region2 = PROVINCE_To_REGION( province ); //EN Region
};

table Person TB_PROV
[ dominant ]
```

```
{
        province+
   * { unit, duration() }

};

table Person TB_REGION
[ dominant ]
{
       aggregate(province,REGION)+  //EN Region
  * { unit, duration() }

};
```

The two resulting tables from a test model are shown below. Note how the value of 'unit' using the REGION classification is not the sum of the corresponding values in the PROVINCE classification. This is because within-region transitions are not measured in the table that uses REGION as its classification dimension.

Table: TB_PROV

Province	unit	Duration()
Newfoundland	6	6.0
P. E. I.	6	6.0
Nova Scotia	6	6.0
New Brunswick	7	6.5
Quebec	7	7.0
Ontario	8	7.5
Manitoba	8	7.5

Saskatchewan	8	7.5
Alberta	8	7.5
British Columbia	7	7.0
Yukon	7	6.5
N. W. T.	6	6.0
All	84	81.0

Table: TB_REGION

Region	unit	Duration()
Atlantic	7	24.5
Quebec	7	7.0
Ontario	8	7.5
Prairies	9	15.0
Alberta,	14	13.5
British Columbia	7	13.5
All	52	81.0

Defining the Model Parameters

At least one user-specified model parameter file must exist in order for the simulation of the model to occur. The model parameters must be defined in both the Modgen model code (in an .mpp file) and the parameter data file (a *.dat* file) in order for the model executable to make use of them successfully. If a parameter is missing from the .dat input files, Modgen indicates an error. If there is an unknown parameter in an input file, Modgen will give an error message when the parameters are read but will continue. The user will still be able to see parameters. If a simulation was running, the error message will also be written in the log file. In such a case, the simulation will continue.

Modgen has the capability of generating a model parameter from two user-specified parameters. This facility can speed up execution if the endogenous parameter is used over and over again in the simulation. The size of all the model

parameters are defined by classifications, ranges or partitions.

Syntax for Model Parameter Definition in .mpp files

```
parameters  <group_name>
{
        type      name[size]...[size];
};
```

or:

Syntax for Model Parameter Definition in .dat files

```
parameters  <group_name>
{
        type      name[size]...[size] = <( repeater)>{ value, value, .... , value }
};
```

Modgen recognizes numbers stored in scientific notation (with mantissa and exponent) in the .dat files. Some examples, 1E-10, 2.5e100, 0.3E+101

Parameter descriptions can be entered on a separate line before the parameter definition or may follow on the same line as the definition. The syntax is (the <…> are optional):

//<langcode1> label1 <decimals=n> <//langcode2 label2 …>

Single-line comments in parameter input files are allowed to have a maximum of 4096 characters.

For more information on parameter decimals, see “Parameter Decimals” on page 47.

numeric

Numeric model parameters are defined as one of int, long, float or double.

Example definition in the .mpp file

```
// annual earnings scale by disability status
float       annual_earnings_scale[DFLE_STATE];
```

Example definition in the .dat file

```
// annual earnings scale by disability status

float annual_earnings_scale[DFLE_STATE] = { 10000.0, 7500.0, 2500.0, 0.0 };
```

logical

Modgen supports the definition of logical parameters. The values of logical parameters in .dat files can only be TRUE or FALSE (capitalized).

Example

```
logical Alive = {TRUE};
```

cumrate

There are some instances in simulation modeling where the state of an actor may change one state to one of many new states. For example, a person's health status may change from good to one of poor, fair, or good. , It is typical to express these transitions as probabilities. If these model parameters require a comparison to a random number to determine whether or not a transition occurs, they should be defined as type cumrate.

Parameters of type cumrate [i] are probabilities which sum to one over the last i dimensions of the parameter. Therefore, the probabilities are conditioned on any previous dimensions in the table. Finally, the model parameters are not input as cumulative probabilities. Modgen cumulates the values when accessing them via the Lookup_ function.

Examples

```
// this example is not present in simpex

// the model parameter, CumImmig, is a three-dimensional array whose

// size is determined by the classification SEX, the range YEAR,

// and the range IMM_AGE.

// the values of CummImmig express the probabilities of a persons age

// at immigration conditioned on their AGE and simulation YEAR.

cumrate         CumImmig[SEX][YEAR][IMM_AGE];


// this example is not present in simpex
```

```
// the model parameter, CumBirth, is a three-dimensional array whose
// size is determined by the classification SEX, the range YEAR_OF_BIRTH,
// the classification PROVINCE.
// the values of CumBirth express the probability of being born in a given
// year and province conditioned on the sex of the person
cumrate [2]      CumBirth[SEX][YEAR][PROVINCE];
```

piece_linear

The model parameter of type piece_linear stores a set of N points (their X and Y values) in 2-dimensional space: x[1], y[1], x[2], y[2], ..., x[N], y[N]. The X values must be given in increasing order (i.e. x[1] < x[2] < ... < x[N]). This set of points defines a piece-linear function y(x) in the following way:

- for x <= x[1] : y(x) = y[1]
- for x[K] < x <= x[K+1]: y(x) = y[K] + (y[K+1] - y[K]) / (x[K+1] - x[K]) * (x - x[K])
- for x > x[N]: y(x) = y[N]

The parameter of this type must be one-dimensional and must contain an even number of values. The dimension of the parameter must be a range.

Example

```
// The following example is not found in simpex
piece_linear WagePoints[WAGE_POINT_VALUES]
```

file

A parameter of type *file* provides a file name that can be varied by the model user from scenario to scenario. For example, the end user could vary a starting microdata file used in a simulation with a *file* parameter.

A parameter of type *file* is declared in a model source code file (.mpp file) as in the following example:

```
parameters {

      file my_file;       //EN File for testing
```

```
};
```

The value for a *file* parameter is given in a parameter input file (.dat file) as a sequence of characters enclosed in double quotation marks:

```
parameters {

        file  my_file = "C:\StatCan\tmp\thing.txt";

};
```

A Modgen model sets the current working directory to the directory of the scenario. A file name specified using a relative path is therefore interpreted relative to the scenario directory. For example, “thing.txt” (with no leading path) means the file “thing.txt” located in the scenario directory, and “..\thing.txt” means the file thing.txt located in the parent directory of the scenario directory.

Arrays of *file* parameters are not permitted.

Model-generated file parameters are not permitted.

Model generated parameters and the PreSimulation function

Model generated parameters are declared just like regular user-supplied parameters. The only difference in the definition of the parameter is the keyword model_generated before the parameter type. Model generated parameters need no entries in the data (*.dat)* files. All such parameters are initialized to zero at the moment of their creation.

Syntax for Model Generated Parameter Definition in .mpp files

```
model_generated type  name[size]...[size];
```

Each model can define a special function, called PreSimulation, which takes no argument and returns no result. This function should be used to assign the values of model-generated parameters, based on input parameters. The function supports all C++ language types and is executed after input parameters have been read, but before the simulation starts. The use of PreSimulation is optional.

Parameters of all types can be used in the definition of model generated parameters. Like input parameters, their values assigned in PreSimulation() should be their original values. Modgen will perform all required transformations (e.g. cumrate) after PreSimulation() has been called. The parameters will also be validated at that point.

Examples

```
// The following example is not taken from the simple example
```

```
parameters {
...
    int DiseasesModeled[DISEASES];
    float MortHazards[DISEASES];
    model_generated cumrate NonModCauseOfDeath[DISEASES];
...
}
...
void PreSimulation()
{
    int nIndex;

    for ( nIndex = 0; nIndex < SIZE( DISEASES ); nIndex++ ) {
        NonModCauseOfDeath[nIndex] = DiseasesModeled[nIndex]
            ? 0 : MortHazards[nIndex];
    }
}
```

Parameter Groups

Parameter groups allow the model builder to group parameters for visual selection in the Group View tab of the interactive option. Parameter groups can only be defined in .mpp files. One parameter can belong to any number of groups. A parameter group may consist of parameters and/or other parameter groups. A group must not be a member of itself, directly or indirectly. If this happens, Modgen will generate a "circular dependency" error.

Syntax

parameter_group groupname {
parameter_name|groupname, ..., parameter_name|groupname
};

Example

```
// This is not a simpex example

parameter_group GDP_DEFLATOR { // GDP deflator parameters

BasketUpdateTime, DeflatorComputeTime, DeflatorCommodThresh,

DeflatorStoreThresh, PaascheFactor

};
```

Model generated parameters & parameter groups

The keyword model_generated_parameter_group operates just like parameter_group, but applies only to model-generated parameters. Model generated parameters are not allowed as members of a parameter group. Instead, you must use a model-generated parameter group for such parameters. The only parameters allowed as members of a model-generated parameter group are model-generated parameters.

Parameter Dependencies and Extensions

Parameter extensions allow the model builder to initialize a parameter with fewer datapoints than its definition requires (however completely filling a number of first dimension slices). The parameter can then be extended along the first dimension using one of the 2 extension definitions supported by Modgen:

Syntax

extend_parameter parameter_name ;
extend_parameter parameter_name related_parameter_name ;

Extension definitions are entered in .mpp files. The first method extends the data by repeating the last stored slice.

The second method uses a one-dimensional related parameter and requires that both the original parameter and related parameter be of type ‘range’. In addition, the range corresponding to the original parameter that is being extended must either match the range or be completely contained within the range corresponding to the related parameter The original parameter is then extended using the ratios between the corresponding cells of the related parameter as multiplying factors on the stored values. This extension method is most useful for timeseries parameters which include historical and projected years (the time must be the leading dimension). Only parameters of type double can be extended using another related parameter.

Examples:

extend_parameter UniTuitionFees ;

extend_parameter MaxLoan AvgIndWage ;

The following is an example where the range of the state being extended is completely contained within the range of the related parameter.

```
range MODELED_YEAR            { 1985, 2099 };
range CCTUT_MODELED_YEAR      { 1980, 2101 };

double      CCTuitionFees[MODELED_YEAR][PROV_OF_STUDY][CC_TUITION_FOS];
double      CCTuitionFeeIndex[CCTUT_MODELED_YEAR];

extend_parameter  CCTuitionFees CCTuitionFeeIndex ;
```

Parameter extensions are useful for parameters containing dollar-denominated values. The values beyond the stored historical years may, for example, be projected using an index based on the Average Industrial Wage as above. The visual editor indicates such values by displaying them in light gray within the grid.

Parameter Decimals

The model user may wish specify the number of decimals stored with a numeric parameter. Unlike the decimals for the table expression, parameter decimals are used for both the formatting of the displayed value and the rounding of a stored value (in case of the table expressions decimals are only used for formatting the displayed value). Parameter decimals prove particularly useful for extended parameters (especially those containing dollar-denominated values).

To specify the number of parameter decimals include “decimals=” string in parameter’s label (the same convention is used in table expressions).

Example

```
 //EN Income exemption decimals=0 //FR Exemption du revenu
double  IncomeExemption[MODELED_YEAR] = {
    (7) 600,
};
```

The decimals are specified in the .mpp files. The default value for parameter decimals is -1 which means no decimal

limit and no formatting on display. In general, all negative values have this meaning.

Hiding Parameters

The function hide is used to prevent parameters from appearing in the visual interface to a Modgen model. Its syntax is:

hide (parameter_name {parameter_name, parameter_name …});

The arguments are the names of the parameter(s) or parameter group(s) that are to be hidden, with each parameter name or parameter group name separated by a comma. (Note that the same function, hide, can also be used to hide tables in the visual interface; however, parameters and tables cannot be combined in the arguments.) Using the name of a parameter group is equivalent to using the name of each member of the group. A parameter group will be shown in the model only if it contains at least one parameter that is not hidden. Hidden parameters will also not be included in the database exported to Excel when requested through the visual interface (even if the user asked to include the model's parameters).

Parameter Validation Routines and Parameter Normalization

Modgen provides support for parameter validation. The model designer may write a parameter validation routine called "ValidateParameters" and Modgen will call it in appropriate situations. More than one definition of "ValidateParameters" may be included in the .mpp files, e.g. different .mpp files could have validation routines for parameters that they contained.

Here is the prototype of "ValidateParameters":

```
BOOL ValidateParameters( SCENARIO_EVENT eEvent );
```

where "eEvent" may be one of the following constants:

- SCENARIO_SAVE_EVENT, // the routine is called when saving the scenario.
- CELL_MOVE_EVENT, // the routine is called when a parameter cell changes.
- BLOCK_CHANGE_EVENT. // the routine is called when one or more parameter cells are modified.
- SCENARIO_RUN_EVENT // the routine is called just before the simulation starts in the interactive mode (the event is not handled in the batch mode).

- SCENARIO_MENU_EVENT // the routine is called from the Scenario/Validate menu item.

The function returns TRUE or FALSE. TRUE indicates that the validation was performed succesfully (possibly with some adjustments or with user approval to go ahead in case of problems) and Modgen proceeds wih the event that called the function (Scenario Run, Scenario Save, Data Change or Cell Movement). If FALSE is returned Modgen cancels the event (e.g. Scenario is not saved and user is returned to editing the parameter).

There are also some Modgen-supplied utility functions that may be called by "ValidateParameters". Some routines (GetParameterValue, SetParameterValue, CheckParameterTotals, NormalizeParameter) only handle parameters of simple numeric types (int, long, float, double).

BOOL **ParameterChanged**(const char *szName);
returns TRUE if parameter "szName" was modified in current edit session.

double **GetParameterValue**(const char *szName, …);
returns the value of the specified parameter cell (indicated by the optional arguments)

void **SetParameterValue**(const char *szName, double dValue, …);
sets the value of the specified parameter cell (indicated by the optional arguments)

BOOL **CheckParameterTotals**(const char *szName, int nDimension, double dDesiredTotal, double dToleranceFactor);
verifies dimension totals for dimension specified by "nDimension" argument. Returns TRUE if all dimension totals match "dDesiredTotal". For parameters of type "int" or "long" the match has to be exact, for parameters of type "float" or "double" the comparison tolerance is determined by "dToleranceFactor" argument. That factor is multiplied by the parameter storage precision which is indicated by the "decimals=" setting in parameter label or default storage precision (6 decimals for "float", 15 decimals for "double"). The comparison tolerance is equal to dToleranceFactor * pow(10, - decimals). If a total along a specified dimension is zero then CheckParameterTotals will not indicate invalid data.

int **ConfirmUndoParameterChanges**(const char *szName);
displays a message box giving the user a chance to cancel all parameter changes if the validation routine failed. The return code indicates which button in the message box was pressed (IDYES, IDNO or IDCANCEL). The modeler may then take appropriate actions depending on the return code.

BOOL **NormalizeParameter**(const char *szName, int nDimension, double dDesiredTotal);
re-scales the parameter by the ratio of the desired total and and real total along indicated dimension. After this operation the parameter totals along "nDimension" should be reasonably close to the desired total. Returns TRUE if the parameter was modified during the normalization process. If a total along a specified dimension is zero then NormalizeParameter will not try to normalize it.

int **ConfirmCallToCorrectionRoutine**(const char *szName);

or

int **ConfirmCallToCorrectionRoutine**();

This function has 2 prototypes. The function may be called when parameter validation failed for a specific parameter (the version with the argument) or a group of parameters. It will display the message box informing the user that parameter validation failed and ask if the correction routine should be called. The return code indicates which button in the message box was pressed (IDYES, IDNO or IDCANCEL). The modeler may then take appropriate actions depending on the return code.

```
void   MessageValidationFailed ( const char *szName );
```

or

```
void   MessageValidationFailed() ;
```

This function simply displays an error message indicating that the parameter validation routine failed. It is similar to the ConfirmCallToCorrectionRoutine function; however, its message box only contains the OK button and no question about calling the correction routine is asked. The function takes an optional argument which is a parameter name if the message is related to one specific parameter.

Here is an example of a parameter validation routine:

```
BOOL ValidateParameters( SCENARIO_EVENT eEvent )
{
      BOOL bProceed = TRUE;
      int nResult;

    switch ( eEvent ) {
        case SCENARIO_SAVE_EVENT:
            if ( ParameterChanged( "Coefficients" ) ) {
                if ( !CheckParameterTotals( "Coefficients", 1, 1.0, 100 ) ) {
                    nResult = ConfirmCallToCorrectionRoutine("Coefficients");
                    if (nResult == IDYES) {
                         NormalizeParameter( "Coefficients", 1, 1.0 );
                    }
```

```
            else if (nResult == IDCANCEL){
                bProceed = FALSE;
            }
        }
    }
    break;
case CELL_MOVE_EVENT:
    break;
case BLOCK_CHANGE_EVENT:
    break;
case SCENARIO_RUN_EVENT:
    if ( !CheckParameterTotals( "Coefficients", 1, 1.0, 100 ) ) {
        nResult = ConfirmCallToCorrectionRoutine("Coefficients");
        if (nResult == IDYES) {
            NormalizeParameter( "Coefficients", 1, 1.0 );
        }
        else if (nResult == IDCANCEL){
            bProceed = FALSE;
        }
    }
    break;
case SCENARIO_MENU_EVENT:
    if ( !CheckParameterTotals( "Coefficients", 1, 1.0, 100 ) ) {
         nResult = ConfirmCallToCorrectionRoutine( "Coefficients" );
         if ( nResult == IDYES ) {
             NormalizeParameter( "Coefficients", 1, 1.0 );
         }
```

```
            }
            break;
        default:;
    }
    return bProceed;
}
```

Symbol Labels

Various symbols defined in mpp files can be labeled using the LABEL statement. The syntax is useful for multilingual models which may label the symbols in just one (default) language in their main set of mpp files and provide labels for additional languages in separate mpp files. The labels may also be added immediately after the symbol definition as shown in Person.mpp below.

The syntax of a symbol label:

```
// LABEL ( symbol ) text
```

or `// LABEL ( symbol, language ) text`

The maximum size per label (i.e. maximum size of "text" in the above syntax statement) is 255 characters.

Examples (from the Simpex model distributed with Modgen setup):

SimpexEN.mpp:

```
classification SEX { //EN Gender
    FEMALE,    //EN Female
    MALE         //EN Male
};
```

SimpexFR.mpp:

```
// LABEL ( SEX, FR ) Sexe
// LABEL ( FEMALE, FR ) Féminin
// LABEL ( MALE, FR ) Masculin
```

Person.mpp:

```
actor Person            //EN Individual
{
…
    int    earnings = {0} ;         //EN Lifetime earnings to date
    DFLE_STATE    dfle = {NO_DIS};  //EN Disability State
    LIFE_STATE    life_status = {ALIVE};  //EN Life Status
    …
    };
```

PersonFR.mpp:

```
// LABEL ( Person, FR ) Individu
// LABEL ( Person.earnings, FR ) Gains totaux à ce jour
// LABEL ( Person.dfle, FR ) Situation-handicapé
// LABEL ( Person.life_status, FR ) Situation–vie
```

StandTabEN.mpp:

```
table Person X08_AvgEarnByAge //EN Cases and their average earnings by age group
{
    split( age, AGE_GROUPS )+  //EN Age Group
    * {
        unit,                  //EN Persons
        earnings / duration()  //EN Avg earnings
    }
};
```

StandTabFR.mpp:

```
// LABEL ( X08_AvgEarnByAge, FR ) Cas et gains moyens par groupe d'âge
// LABEL ( X08_AvgEarnByAge.Dim0, FR ) Groupe d'âges
// LABEL ( X08_AvgEarnByAge.Expr0, FR ) Personnes
// LABEL ( X08_AvgEarnByAge.Expr1, FR ) Gains moyens
```

To label modules, classifications, ranges, partitions, actors or tables, use their names as the symbol name in the LABEL statement.

To label actor members (e.g. states) use their names prefixed by the actor name and a dot (e.g. Person.earnings).

To label a classificatory dimension of a table use a dimension name (Dim0, Dim1,) prefixed by the Table name and a dot (e.g. X08_AvgEarnByAge.Dim0).

To label a table expression use the expression name (Expr0, Expr1, …) prefixed by the Table name and a dot (e.g. X08_AvgEarnByAge.Expr0).

Symbol Notes

Symbol notes allow the model developer to encapsulate the Modgen model documentation into the compiled model for use by model users.

Any notes entered in .mpp and .dat files are pre-formatted into paragraphs and lists before being displayed in the model's interactive environment. The following rules apply when formatting a note:

- All leading white-space characters (spaces and tabs) are removed from each line of note text.
- A set of non-empty lines delimited by CRLF is considered as a paragraph and formatted as such
- An empty line (\n\n) introduces a new paragraph.
- A line that starts with a hyphen ('-') is considered to be an element of a list and is formatted as such.

Example:

```
/* NOTE (MortalityModule, EN )
     Here is an imaginary paragraph
     which has been indented for illustrative
     purposes.  It is identified as a paragraph because it
     is terminated by a blank line, i.e. \n\n.


    - This is item 1 of an ordered list,
      which has been wrapped because
      it is long.
    - This is item 2 of the list

     Here we have another paragraph.  Had this paragraph started
     immediately following the list, an empty line should still have been
```

```
    inserted since paragraphs should have one blank line between themselves
    and preceding or following text.
*/
```

Displayed as:

```
Here is an imaginary paragraph which has been indented for illustrative
purposes.  It is identified as a paragraph because it is terminated by a blank
line, i.e. \n\n.

 - This is item 1 of an ordered list, which has been wrapped because it is
long.

 - This is item 2 of the list

Here we have another paragraph.  Had this paragraph started immediately
following the list, an empty line should still have been inserted since
paragraphs should have one blank line between themselves and the preceding or
following text.
```

When saving editable notes (parameter value note, parameter file note, scenario note) in their source files the paragraphs are broken into lines of approx. 80 characters and blank lines are inserted between paragraphs if not already there.

Also, it is possible to indicate additional lines where carriage returns should be preserved without a requirement for inserting blank lines. To do this, a greater-than character ('>') must be entered as the first character (other than spaces and tabs) on the line of text. Such text will then be displayed at the beginning of the new line and not appended to the previous paragraph. The '>' character itself will not be displayed. The white-space characters (spaces and tabs) following the '>' character are preserved on display to enable indentation of text.

Example (part of scenario note from COPS Demand Model):

```
    ...

Coefficients were generated using the same methodology as for the 1998 Spring
Update scenario.  However, coefficients have been revised to reflect NOC coding
of occupations on the Labour Force Survey to the end of 1998.
```

```
Year Range: 1984-2008


The scenario was originally generated by:
>    Occupational Projections and Macroeconomic Studies
>    Applied Research Branch, HRDC
>    …
```

Symbol notes can be attached to any Modgen symbol used in a model (module, classification, range, partition, actor, state, table, parameter, parameter group etc.). There are 2 types of symbol notes. Notes in .mpp files should describe the usage of a particular symbol; notes in .dat files should describe particular parameter values and are labelled as value notes in property windows.

The syntax of a symbol note (the langcode is optional) is:

/* NOTE (symbol <,langcode>)
text
*/

The symbol names for table dimensions and the table expressions axis are special:

<tablename>.Dim0, <tablename>.Dim1….
<tablename>.Expr0, <tablename>.Expr1…

If the expressions axis, for example, is the last dimension of your table (i.e. in the columns) this allows detailed notes for every column of the table.

Example

```
/* NOTE ( WagePoints )
The wage rates are relative earning rates, that will be scaled (to have mean 1.0
).
These rates are applied to Firms' overall costs and profits
(divided by the number of individuals ) to calculate Consumer's income.
*/
```

When writing notes for actor's objects (states, events etc.) the symbol name consists of actor name followed by a dot and the object name, e.g. "Person.mar_status".

Documentation

Help menu

The Help Menu available through the Visual Editor can have up to five sub-items enabled:

- Help on <ModelName>
- Help on Modgen (provides online access to the Guide to the Modgen Visual Interface)
- <ModelName> Release Notes
- Modgen Release Notes
- About:

The third sub-item, <ModelName> Release Notes, is enabled only if the release notes file is found in the model directory. The name of a model release notes file must follow the standard:<ModelName>Notes_<LanguageCode>.htm. (eg. SimpexNotes_EN.htm). If there is no known language, then the file name becomes <ModelName>Notes.htm.

The fifth sub-item, About:, will display both the Model and Modgen version numbers, and provide buttons to display the actual license for either the model or Modgen. The model license button, however, is only enabled if the corresponding license file exists in the same directory as the model. The license file name must follow the standard:

<ModelName>Licence_<LanguageCode>.htm (eg. SimpexLicence_EN.htm)

Automated model documentation facility

Modgen has the ability to produce and use model documentation in CHM form. The following sketches what occurs when an end user of a Modgen model requests model help from the menu.

First, Modgen detects whether the model has an associated up-to-date CHM help file, and whether the current machine environment supports CHM help files. If so, the CHM help file is displayed. Modgen will search for and use the file <ModelName>1_<LanguageCode>.chm if present, allowing the model developer to transparently substitute a different help system. Generally, the help project defining <ModelName>1_<LanguageCode>.chm should refer to the CHM help file generated by Modgen, i.e. to <ModelName>2_<LanguageCode>.chm.

Modgen model developers are advised to invoke the "**Help on <ModelName>**" menu item to generate an up-to-date CHM file that can be shipped with a model. This should be done for each language supported if multiple languages are supported by the model. The generated CHM file is named <ModelName>2_<LanguageCode>.chm, where <ModelName> is the name of the model, and <LanguageCode> is the string identifying the language for models that support multiple languages. For example, the English version of the generated documentation for the LifePaths model is named LifePaths2_EN.chm.

For Modgen models which do not have any languages specified, the CHM files are called <ModelName>1.chm and <ModelName>2.chm.

Help mode

A model can be run in help mode, similar to the manner in which it can be run in documentation mode. Instead of running the simulation, the following command will create the help files for all the languages known to the model. The syntax to be used (using the model Simpex as an example) is:

```
D:\Simpex.exe -help
```

Language choice

The languages defined in a particular model are actually grouped to form a language set. Each language set has a preferred language ordering specific to each user.

Example:

Simpex "knows" two languages: English and French. If a user opens a Simpex model and then sets the language to English, then the preferred language for the language set `{English, French}` will be English.

More than one model may use the same language set and will therefore have the same preferred language for a particular user. We have assumed that if a user was given the choice of French and English and chose English, then the user would always prefer English to French. Thus, when a user changes the language in a model, it will affect all models having the same language set.

Furthermore, if another language set contains at least the languages of that model, and if the current preferred language is the same as that of the model's language set, the same principle of rational choice ordering will apply and the preferred language will be affected.

Example:

Let's say we have a language set 1:

```
LanguageSet = {English, French}
```

with the preferred language English

And another language set 2

```
LanguageSet = {English, French, Spanish}
```

with the preferred language English

Then if a user opens a model using language set 1 and changes to **French**, it will be interpreted to mean that **English** is no longer the preferred language, and **French** is the new one. Modgen will go through the list of language sets, and whenever both **English** and **French** are in the language set, if the preferred language was **English**, it will be changed to **French**. In this case, both language sets would change the preferred language to **French**.

The known language sets and preferred languages will be kept in the registry for each user. Whenever a model which contains an "unknown" language set is opened in interactive mode, the user will see the following dialog asking him to choose a language before the model opens. The language set will then be created.

This is also the dialog that is shown when the user wants to change the language (**View menu, Set Language**). Once again, a given user will be prompted with this dialog box only once in interactive mode for English/French models. The response for one English/French model will be remembered and will apply to all English/French models.

When installing Modgen, the user is asked to choose a language for the setup. The two languages of the setup program (English and French) are used to create a language set, and the language chosen by the user becomes the preferred language of that language set for that user.

Using Other Languages

Both English and French language support are fully built in to Modgen. However, support for other languages can also be built in, with the assistance of the files ModgenEN.mpp or ModgenFR.mpp, both of which are part of the Modgen distribution and which can be found in the Modgen installation directory. These files contain all of the character strings for the Modgen interface and are intended to be used as a starting point by a model developer wishing to add support for additional languages to a Modgen model. For example, a model developer could translate the contents of ModgenFR.mpp from French to Spanish to produce the file ModgenES.mpp, and include this file (along with an appropriate 'languages' statement defining 'ES' as 'Espanol') in a model project to provide a Spanish language interface.

Using the Table Facility to Report Model Results

Overview of the Table Facility

The standard means of reporting model results is through the cross-tabulation facility. This facility enables users to specify a wide variety of tables on the simulation population. Generally, the tabulations summarize information on the following (usually categorized by any number of classificatory states):

- Values of continuous states such as earnings or costs.
- Durations of actor states.
- Counts of actors or events (state changes).

Unlike normal cross-tabulators, Modgen tabulation has the added richness of time to take into account. To maximize computational efficiency and minimize overhead, Modgen tabulations are accumulated "on the fly". Think of the actors moving through time journeying through the cells of a table as states change. Modgen builds the table outputs with little knowledge of the actor's past and no knowledge of the future. As an actor enters a cell in the table, the opening values of various dependent states are put aside and a cell entry count (called unit) is incremented by 1. When the actor exits a table cell and a request to move time forward is detected by Modgen, a table update is triggered using the exiting state values, the opening state values and the setting for the cumulative operator. The cumulative operator tells Modgen how to summarize the values within the cell. The most common such operator is a sum but Modgen also allows minimums and maximums. The default operator (called delta) is a bit more interesting. It subtracts the opening values from the exiting values and adds the result to the running total already within the cell. The delta method of cumulation only makes sense for states which can be expressed as cumulative lifetime values (an important point to remember when implementing new states for actors).

Continuous states such as earnings meet this requirement nicely with a minimum amount of overhead. Suppose a person

actor has been defined with a state containing cumulative lifetime earnings and we wish to tabulate average earnings by age in 1 year intervals. Each case has one such actor and the simulation has been launched with 100 cases. When the first actor say turns 30, his cumulative lifetime earnings are put aside, unit is incremented by 1 for that table cell and time moves forward with any number of intervening events until age 31. At that time, his cumulative earnings at 30 are subtracted from his cumulative earnings at 31 to obtain the earnings for that interval. When the second case comes along, the same calculations take place on cell entry and exit, the result for the numerator being added to the running total from the previous cases. And so on for all 100 cases. When the last case is completed, the division for the average takes place. Note that with this technique, if the underlying behavioural model updates the earnings state every 2 weeks, no computational overhead is added to the tabulation process.

You could use unit as the denominator for the average but it would be a poor choice. Use of durations is essential for the tabulation process. Suppose you wish to tabulate average annual earnings within 10 year age intervals rather than the 1 year interval described above. Using unit as the denominator would be incorrect. You really want to divide by the interval duration (in this case 10 but which just happened to be 1 above) not the number of actors entering the cell. To get the annual average for the 10 year interval, you need to sum the earnings over the 10 year interval for all actors and divide by the sum of the interval durations for all actors. Modgen supplies derived states for this purpose, in this case the duration() state. Refer to “Derived states associated with durations of actor states” on page 31 for a complete list of such states.

This works nicely for earnings but what if you would like to count events within an age interval. The events can be viewed as changes to a specific value of a state. The tabulation is an interval delta of the cumulative lifetime changes to that state value. Again, Modgen provides derived states which can do the lifetime accumulations for you. For example, suppose you want to tabulate the total number of marriages at age 30 for all 100 actors. In this case, tabulate with a derived state called entrances, with arguments specifying the state marital_status with setting MARRIED. When combined with the delta cumulative operator, the cell will contain the marriages during that 1 year interval. Refer to “Derived states associated with changes to actor states” on page 28 for a complete list of such states.

The table specifications, which are defined by the table command, are contained in the StandTab.mpp file. Generally, classification, partition or logical states define the table’s dimensionality while table expressions specify the cell values in a table. All tables are associated with a model actor (typically a person, i.e. core individual). An optional filter allows users to report on subsets of the simulated population. Labeling and naming each table is especially useful in conjunction with writing output to Excel spreadsheets and Access output databases.

Model developers can also define table groups for use in the Group View option of the model's visual interface. In addition, should a user wish to exclude certain tables at run-time, the visual interface allows you to select the subset of tables to be accumulated and sent to output. Excluding unneeded tables for a specific run can speed up the simulation considerably. For more information on table outputs, refer to the Guide to the Modgen Visual Interface.

Table Definitions

The syntax of the table command is as follows. The < > brackets indicate features of the table command which are optional.

Syntax

```
table <sparse> actor_name <table_name> <[filtering_criteria]>  <// label>
{  < dimension  <+> <// dimension label>
          *>
   < dimension <+> *>
          …
          {
                    analysis dimension expression,  <// expression label > <expression options>
                    <analysis dimension expression,  <// expression label > <expression options>>
                    …
                    <analysis dimension expression  <// expression label > <expression options>>
          }
          <*
   dimension  <+> < // dimension label> >
          …
} ;
```

If the table is very large and very sparse (contains many zeroes), you may save space in your output database by adding the optional sparse keyword to the table statement.

There is no limit to the number of dimensions in a table. Use "*" to delimit the dimensions. The dimension label is optional. However, if no label has been explicitly provided, Modgen will construct a language-specific label by using the language-specific label of the underlying state. Also, if the label is absent, the "*" can be placed on the same line as the dimension. Use an optional "+" to indicate that a dimension total be computed for that classificatory dimension.

Each table can contain only 1 analysis dimension with any number of expressions separated by commas before the expression label. The last expression can not be followed by a comma. The expression options specify decimals and scaling factors for the data in the body of the table. If the analysis dimension, for example, appears in the columns of your table (i.e. no dimension statements appear after it), you can optionally specify different decimals and scaling factors for each column of your table.

Use of table_name is optional, default names (i.e., Table X1, Table X2, etc…) will be generated by Modgen. Use of table_name is recommended however for the Excel output option and/or the Access output option used by the interactive

mode.

Once the simulation is complete for all cases, the table expressions are evaluated with the aggregate values of these states. Therefore, expressions which are ratios of states report the aggregate value of the numerator over the aggregate value of the denominator and not the average of the actor-specific ratios in the simulation.

Table Groups

Table groups allow the model builder to group tables for visual selection in the Group View tab of the interactive option. Table groups can only be defined in .mpp files. One table can belong to any number of groups. A group may consist of tables and/or other table groups. A group must not be a member of itself, directly or indirectly. If this happens, Modgen will generate a "circular dependency" error.

The syntax of table group definition is similar to the parameter group definition:

table_group groupname { // label
table_name|groupname, ..., table_name|groupname
};

Example:

```
table_group LOAN_TABLES { // Loan-related Tables
LOAN, FORGIVEN, INTRELIEF, LOANNEEDS
   };
```

Table Expressions

Table expressions, which define the contents of the table cells, may consist of the tabulation_operator, states, cumulation keywords, special functions, the unit keyword, and numeric constants in conjunction with the arithmetic operators + - * / and parenthesis (). Expressions are separated by a comma in the table specification.

The unit keyword

The unit state stores the number of actors which entered each cell of a table. Therefore, it is useful for reporting the number of actors who had a specific set of characteristics at some point in their simulated lifetime, or for providing a denominator for calculating average values for continuous states being analyzed. The unit keyword should be used with caution. Unexpected double-counting is common particularly in population tables.

Example 1

In this table we report the total number of person actors in a simulation. In this example, the special state unit is the only table expression being specified. There are no classification states in the table specification. Note how the use of comments in the table specification are carried through to the report.

Table Specification

```
table Person // Number of cases simulated
{
    {
        unit // Persons
    }
};
```

Table Result

```
Table X1: Number of cases simulated

+--------+-----+
|Persons |Value|
+--------+-----+
|Persons | 5000|
+--------+-----+
```

States

The values of continuous states can be reported using the table facility. Generally, states which are accumulating over the lifetime of an actor are best suited for tabular reporting.

Example 2

In this example, the table reports the total population's accumulated lifetime earnings (scaled to millions of dollars), the average accumulated lifetime earnings of each person, and the average annual earnings of each person.

Table Specification

```
table Person // Earnings summary
{
...{
    earnings,    // Total population lifetime earnings(millions) scale=-6
```

```
    earnings / unit,        // Avg lifetime earnings
    earnings / duration ()  // Avg annual lifetime earnings
. ..}
};
```

Table Result

```
Table X2: Earnings summary

+-------------------------------------------+------+
|Selected Quantities                        |Value |
+-------------------------------------------+------+
|Total population lifetime earnings(millions)|  3294|
|Avg lifetime earnings                      |658881|
|Avg annual lifetime earnings               |  8588|
+-------------------------------------------+------+
```

Cumulative operators

The cumulation keywords control the way states for all actors are summarized within table cells. They can be interpreted as running totals or running minimums/maximums. In all cases, the state can be a simple state or a derived state expression.

delta(state)

This is the default operator. This keyword allocates the difference between the value of the state when the actor first enters a table cell and the value of the state when the actor leaves a table cell. This is the default cumulative operator.

delta2(state)

Similar to the delta () operator but reports the squared value. This may be useful for calculating the standard errors.

max_delta(state)

Similar to the delta() operator but reports the maximum value.

max_value_in(state)

This keyword returns the maximum value of the state over all cell entries when these actors first enter the cell of the

table.

max_value_out(state)

This keyword returns the maximum value of the state over all cell entries when these actors exit a cell of the table.

min_delta(state)

Similar to the delta() operator but reports the minimum value.

min_value_in(state)

This keyword returns the minimum value of the state over all cell entries when these actors first enter the cell of the table.

min_value_out(state)

This keyword returns the minimum value of the state over all cell entires when these actors exit a cell of the table.

nz_delta(state)

This keyword returns a 1 in instances where the delta () keyword would produce a non-zero value and 0 otherwise.

nz_value_in (state)

This keyword returns a 1 in instances where the value_in() keyword would produce a non-zero value and 0 otherwise.

nz_value_out (state)

This keyword returns a 1 in instances where the value_out () keyword would produce a non-zero value and 0 otherwise.

value_in(state)

This keyword returns the value of the state when the actor first enters the cell of the table.

value_in2(state)

This keyword returns the squared value of the state when the actor first enters the cell of the table.

value_out(state)

This keyword returns the value of the state when the actor exits a cell of the table.

value_out2(state)

This keyword returns the squared value of the state when the actor exits a cell of the table.

Tabulation operators

The table facility distinguishes between two types of tabulation: interval tabulation, and event tabulation. These operators control the tabular output differently when an analysis variable and a classification variable in a table specification change simultaneously. The new values of the analysis variables are tabulated in the cell previous to the change under the interval() operator. The values of the analysis variables are tabulated in the cell after the event has occurred under the event() operator. The default tabulation is the interval() operator. It is the operator of choice for situations where duration-related variables are being analyzed. Sometimes however, one needs to count certain events

by their type. The count and the type could change at the same time. In such case event() tabulation should be used. The Tabulation operators are used similarly to cumulation operators (delta, value_in, etc.) in table definitions.

Syntax

tabulation_operator(state)

where tabulation_operator is either event or interval.

If both a tabulation operator and a cumulation operator are used, then this is the syntax:

cumulation_operator (tabulation_operator (state))

Examples

```
// the following is not part of the simple example
table Person
{
    type
    * {
        event ( count ),
        interval ( duration () ),
        duration ()  // same as the previous line
    }
};

table Person
{
    type
    * {
        delta ( event ( count ) )
    }
};
```

Table expressions with derived states

A rich set of derived states are available for the purpose of table generation. For more information, refer to:

“Derived states associated with changes to actor states” on page 28, and

“Derived states associated with durations of actor states” on page 31.

Example 3

This table consists of an expression which gives the number of persons who were ever classified as severely disabled. The results indicate that of the 5000 simulated persons, 3921 were severely disabled at least once in their life.

Table Specification

```
table Person // Selecting the severely disabled
{
    {
    entrances ( dfle, SEVERE_DIS ) // Persons who were ever severely disabled
    }
};
```

Table Result

```
Table X3: Selecting the severely disabled

+---------------------------------------+-----+
|Persons who were ever severely disabled |Value|
+---------------------------------------+-----+
|Persons who were ever severely disabled | 2768|
+---------------------------------------+-----+
```

Example 4

In this example we calculate the average age at death 5 separate ways for the population as a whole using some of Modgen’s special functions and the special state unit.

Table Specification

```
table Person // different ways to calculate age at death
{
    {
    value_at_entrances (life_status,NOT_ALIVE,age)// Avg Age at Death 0
decimals=2
    / unit,
    value_at_exits ( life_status, ALIVE, age )  // Avg Age at Death 1 decimals=2
    / unit,
    value_at_transitions (life_status,ALIVE,NOT_ALIVE,age)//Avg Age at Death 2
decimals=2
    / unit,
    value_at_changes ( life_status, age ) // Avg Age at Death 3 decimals=2
    / unit ,
    duration ( life_status, ALIVE ) // Avg Age at Death 4 decimals=2
    / unit
    }
};
```

Table Result

Table X4: different ways to calculate age at death

Selected Quantities	Value
Avg Age at Death 0	76.72
Avg Age at Death 1	76.72
Avg Age at Death 2	76.72
Avg Age at Death 3	76.72

```
|Avg Age at Death 4 |76.72|
+-------------------+-----+
```

Expression labeling

Each expression may be followed by the label (starting with a //). If no label is provided, Modgen will construct a language-specific label by using the language-specific label of the underlying state. When provided, the label may contain the special formatting strings which control the presentation of the tabular results. They are as follows.

decimals= n

indicates that the value of the expression is to be reported to *n* decimal places.The table decimals may be changed dynamically for viewing by using the Decimals tab of Table Properties (refer to the User Guide for more information). If the table has more than one expression in its analysis dimension a different number of decimals may be specified for each expression. To enter a new value simply type it in or press <F2> or double click on the grid cell to edit the existing value.

scale= [-]m

indicates that the values of the expression are to be multiplied by 10**_m_. A negative sign can proceed m to, effectively, dividing the result by 10**_m_.

Empty Cells

It is possible to specify tables where some cells remain empty (or zero) at the completion of the simulation. In such situations, an undefined table expression is reported as a * (or a blank cell in Interactive Mode).

Example 14

In this example, the table will report results for the females population under the age of 0. The average earnings expression is undefined since the value for unit is zero.

Table Specification

```
table Person[sex==FEMALE && age < 0] // perverse example: females under the age
of zero
{
    {
        unit,    // persons
        earnings / unit // ave earnings
    }
```

```
};
```

Table Result

```
Table X14: perverse example: females under the age of zero


+------------------+-----+
|Selected Quantities|Value|
+------------------+-----+
|persons           |    0|
|ave earnings      |    *|
+------------------+-----+
```

Reporting Descriptive Statistics on Table Expressions

Standard errors and coefficients of variation can be produced for all table cells by specifying a number, **n**, greater than 1 for the sub-sample control parameter **s**. These statistics are produced by creating **n** parallel sub-samples for each simulation and then calculating the values from these observations.

Table Dimensions

The dimensions of a table are specified through the use of classification variables, special functions which return discrete values, or nested special functions which return discrete values. There is no limit to the number of dimensions in a table. Table dimensions are linked together with the * to define the total rank of the table. To specify the sum over a particular dimension, the user can place a + after the table dimension.

Table dimensions whose values are constant for each actor

These type of tables are the easiest to interpret as each actor is present in only one cell of the table for his/her entire lifetime. For example, persons are either male or female, and have only one age at death.

Example 5

In this table we stratify the total number of persons in the simulation by their sex. The special state, , is only analysis variable being specified. The column table dimension contains the single classification state sex. Note that a + follows the classification state indicating that the table includes a column for the sum of both sex types.

Table Specification

```
table Person // stratify persons by gender (in columns)
{
    {
        unit // persons
    }
    * sex+
};
```

Table Result

```
Table X5: stratify persons by gender (in columns)


                Sex
+-------+------+------+------+
|persons|Female|Male  |All   |
+-------+------+------+------+
|persons|  2535|  2465|  5000|
+-------+------+------+------+
```

Example 6

In this table we stratify the total number of persons in the simulation by their sex along the row dimension.

Table Specification

```
table Person // stratify persons by gender (in rows)
{
    sex+ *
    {
        unit // unit
    }
};
```

Table Result

```
Table X6: stratify persons by gender (in rows)


       unit
+------+----+
|Sex   |unit|
+------+----+
|Female|2535|
|Male  |2465|
+------+----+
|All   |5000|
+------+----+
```

Example 7

In this example the age at death is reported by age class. The age breakdown is defined by the AGE_GROUPS.

Table Specification

```
table Person // cases stratified by age at death
{
    split (age, AGE_GROUPS )+ // age_at_death groups
    *
    {
    entrances (life_status,NOT_ALIVE) // persons
    }
};
```

Table Result

Table X7: cases stratified by age at death

	persons
age_at_death groups	persons
(min,20)	86
[20,30)	37
[30,40)	66
[40,50)	121
[50,60)	348
[60,70)	658
[70,80)	1284
[80,max)	2400
All	5000

Table dimensions whose values change for each actor

There are many classification states whose values change over the course of an actor's simulated lifetime. Examples of such states are a person's age or disability status. If such a state is used to define a table's dimension, then an actor may be present in more than one table cell over his/her lifetime. These types of tables can be useful, but require careful thought in their formulation.

Example 8

In this example, the first column reports the population classified into one of five age groups using the special function split and the special state unit. The column sums to more than the 5000 persons since each individual in the simulation increments each row of the table as he/she passes from one age class to the next. Therefore, this table shows the survivorship pattern of the simulated individuals. The results can be interpreted as follows: of the 5000 cases, 4928 of

them survived to their 20th birthday, 4871 survived to their 30th birthday, 4720 survived to their 50th birthday, and 4434 survived to their 60th birthday. This column can also be interpreted as the number of persons who ever entered the respective age group.

The second column of the table reports the average annual earnings of the population. The special function duration() is used as a denominator to correctly estimate the years of a persons life.

Table Specification

```
table Person // cases and their average earnings by age group
{
    split ( age, AGE_GROUPS )+  *
    {
        unit,        // persons
        earnings / duration () // ave earnings
    }
};
```

Table Result

```
Table X8: cases and their average earnings by age group

                                                                Selected Quantities
+-----------------------------------------------------------+------------+----------
--+
|Breakdown of Person.age for Individual actor definition|persons     |ave
earnings|
+-----------------------------------------------------------+------------+----------
--+
|(min,20)                                                   |        5000|
10000|
|[20,30)                                                   |        4920|
10000|
|[30,40)                                                   |        4881|
```

```
10000|
|[40,50)                                                |        4823|
10000|
|[50,60)                                                |        4708|
9952|
|[60,70)                                                |        4406|
6870|
|[70,80)                                                |        3766|
3311|
|[80,max)                                               |        2557|
657|
+-------------------------------------------------------+------------+----------
--+
|All                                                    |       35061|
8588|
+-------------------------------------------------------+------------+----------
--+
```

Example 9

In this example, the population is classified into one of five age groups using the special function split. The first column of the table reports the average earnings of the population. The second column of the table reports the accumulated earnings of the population over their lives to date using the value_in () operator.

Table Specification

```
table Person // average and cumulative earnings by age group
{
    split ( age, AGE_GROUPS )+  *
    {
        earnings / duration () , // ave earnings
        value_in (earnings) / unit // cum ave earnings (value_in)
    }
```

```
};
```

Table Result

```
Table X9: average and cumulative earnings by age group


                                                                Selected
Quantities
+------------------------------+--------------------+--------------------------
--+
|Breakdown of age for Individual|ave earnings        |cum ave earnings
(value_in) |
+------------------------------+--------------------+--------------------------
--+
|(min,20)                      |               10000|
0|
|[20,30)                       |               10000|
200000|
|[30,40)                       |               10000|
300000|
|[40,50)                       |               10000|
400000|
|[50,60)                       |                9952|
500000|
|[60,70)                       |                6870|
599504|
|[70,80)                       |                3311|
667531|
|[80,max)                      |                 657|
699249|
+------------------------------+--------------------+--------------------------
--+
|All                           |                8588|
390029|
+------------------------------+--------------------+--------------------------
```

```
--+
```

Note that for each table declared in a model, Modgen creates internal actor states that record the location of the current cell in the table. The C++ type used to represent these internal states can be controlled using the *index_type* statement. Allowed values for *index_type* are the C++ signed integer types *char*, *short*, *int*, *long*, and the C++ unsigned integer types *uchar*, *ushort*, *uint*, *ulong* (synonyms for unsigned char, unsigned short, unsigned int and unsigned long, respectively) If a model contains no *index_type* statement, these internal states are of type *int*.

The *index_type* statement can be useful in models where memory utilization is important, such as time-based models with many actors. It is the responsibility of the model developer and model user to ensure that the maximum number of cells in selected tables does not exceed the limit of the C++ data type specified in the *index_type* statement. If more than one replicate is specified in a model scenario, the number of cells in each table needs to be multiplied by the number of replicates plus 1 when calculating the limit.

The following is an example of using index_type:

```
index_type ushort;
```

Table Filtering

The table command contains a facility to select subsets of the simulated population through a filtering option. This feature recognizes the usual set of C++ Boolean operators ==, > ,>=, <, <=, !=, &&, ||. The filtering of a table also controls the initialization of a duration state in a table specification. Therefore, users filtering on a disability state and specifying a duration analysis variable will have durations which begin at the point the individual entered that disability state.

Example 10

In this example, the table will only report for females in the population.

Table Specification

```
table Person[sex==FEMALE] // filtering on the females
{
    {
        unit // persons
    }
```

```
};
```

Table Result

```
Table X10: filtering on the females

+-------+-----+
|persons|Value|
+-------+-----+
|persons| 2535|
+-------+-----+
```

Example 11

In this example, the table erroneously reports the average age at death of persons who were ever disabled over the course of their life. The error was made to show how the results of a duration special function are conditioned on the table filtering. What the result actually reveal are the average number of years an individual is in a disability state before death.

Table Specification

```
table Person[dfle!=NO_DIS] // Filtering on the disabled
{
    {
    duration ( life_status, ALIVE ) // Avg Age at Death 4 decimals=2
    / unit
    }
};
```

Table Result

```
Table X11: Filtering on the disabled

+------------------+-----+
|Avg Age at Death 4|Value|
```

```
+------------------+-----+
|Avg Age at Death 4|16.46|
+------------------+-----+
```

Example 12

In this example, the table contains the results for persons who were ever under 18 and over 65.

Table Specification

```
table Person[age<18 || age >=65] // persons younger than 18 or older than 65
{
    {
        unit // persons
    }
    * split (age,AGE_GROUPXS)*sex
};
```

Table Result

```
Table X12: persons younger than 18 or older than 65
```

	Sex	
Breakdown of Person.age for Individual actor definition	Female	Male
(min,18)	2535	2465
[18,65)	0	0
[65,max)	2238	1879

Example 13

In this example, the table is restricted to females who were ever older than 18 and younger than 65.

Table Specification

```
table Person[sex==FEMALE && (age > 18 && age <=64) ] // females older than 18
younger than 65
{
    {
       unit // persons
    }
    * split (age,AGE_GROUPXS)*sex

};
```

Table Result

Table X13: females older than 18 younger than 65

	Sex	
Breakdown of Person.age for Individual actor definition	Female	Male
(min,18)	0	0
[18,65)	2507	0
[65,max)	0	0

Case Weights and Population Scaling

Modgen allows the use of case weights for tabulation purposes. Case weights are used to scale all additive tabulated quantities (including “unit”) but not the “min” and “max” type quantities (min_value_in, max_delta, etc.). They are also used for population scaling (the sum of case weights is used instead of the number of processed cases). The default weight of a case is 1 but it may be modified using the SetCaseWeight function described below. Each case starts with a weight of 1, which remains in effect until SetCaseWeight is called. Case weights have no utility in time-based models

(e.g. XEcon).

The following Modgen-supplied functions are used to get and set the case weights:

```
void SetCaseWeight( double dCaseWeight, double dCaseSubsampleWeight );
void GetCaseWeight(double *pdCaseWeight, double *pdCaseSubsampleWeight );
```

A special state called “actor_weight” is available for every actor. Calling the SetCaseWeight function also sets the value of “actor_weight” state for every actor in the case. The “actor_weight” state may be tracked and displayed with the BioBrowser.

Another special state called actor_subsample_weight can also be tracked by the BioBrowser. This state is equal to actor_weight multiplied by the number of sub-samples. This state indicates the weight applied to sub-sample quantities (actor_weight is applied to total quantities).

An example application of case weights could be the cloning of interesting cases (currently used in the POHEM model). Another use would be the adjustment of case weights when some probabilities are boosted to increase the number of interesting cases.

Hiding Tables and Establishing Table Dependency

The function **hide** is used to prevent tables from appearing in the visual interface to a Modgen model, while the function **dependency** allows a dependency relationship to be established between two tables or between one and several other tables. The syntax for each function is as follows:

hide(table_name, {table_name, table_name, …});

dependency(table_name1, table_name2, {table_name3, … });

The arguments for **hide** are the names of the table(s) or table group(s) that are to be hidden, with each table name or table group name separated by a comma. (Note that the same function, **hide,** can also be used to hide parameters in the visual interface; however, parameters and tables cannot be combined in the arguments.) Using the name of a table group is equivalent to using the name of each member of the group. User table names (defined in the next chapter) are also valid choices for arguments.

The **dependency** function establishes a dependency relationship of the first named table upon the remaining tables specified in the arguments; that is, the first named table depends on the remaining tables. The first named table can be either a regular table or a user table, but not a table group. The other arguments can be regular tables, user tables, or table groups. Again, the use of a table group is equivalent to including each of the table group’s members.

The following rules apply to both the **hide** and **dependency** functions.

- If a table is hidden and no other tables (included) depend on it, then it will not be included in the simulation.
- If a table (A) is hidden, but another table (B) which is included depends on A, then A will be included (calculated) but will not be shown in the list of tables, nor in the database, and it will be impossible to export A. However, should the user remove B from the list of tables included in the simulation, then A will not be included either
- A table group will be shown in the model only if it contains at least one table that is both included and not hidden.

User Tables

User tables are tables built from other tables defined in .mpp files. User tables are more powerful than spreadsheet calculations done on the output tables because they also produce the standard errors and CVs which are not possible to obtain just by working on the output tables.

A user table definition is similar to a regular table definition, but there are a few differences. The user table must always be named, it does not contain filters, it is not related to any actor, it uses the classification, range or partition symbol as the classificatory dimension, it uses names for the table expressions. Table expressions may contain decimals and/or scale specification just like in regular tables.

Modgen allows dimension totals in user tables. To request the dimension totals add a '+' after the dimension name (just like in the regular tables). Unlike in regular tables, the request for dimension totals only results in a creation of storage space (the extra cells) for the total values. The totals are not automatically computed as a result of adding a '+' to the dimension name. The reason for this is that various quantities can be stored in the user tables and just adding them up doesn't always make sense (e.g. for the ratios). The user may want to compute the totals himself if they are more complicated than just the simple sums.

If the totals are simply the sums of cell values you may call a new Modgen-supplied function to compute all dimension totals for a specific user table. The function is called "TotalSum" and has the following prototype:

```
void TotalSum( const char *szTableName );
```

The proper way to use the TotalSum function is to fill the non-total cells of the user table and then call the function to compute the sums.

A special Modgen constant called UNDEF_VALUE is available to all models. It is meant for user tables in which the modeler wants to indicate that some cells contain a null (no data) value as opposed to a proper zero. User tables cells are initialized to UNDEF_VALUE (null value). The modeler does not need to assign the UNDEF_VALUE to cells that

contain no observations and also the dimension totals will contain UNDEF_VALUE if there were no values found in the entire dimension (else the null values are treated as zeroes for total calculation).

If the user table is very large and very sparse (contains many zeores), you may save space in your output database by adding the optional sparse keyword to the user_table statement as follows:

user_table sparse table_name ….

If the table is marked as sparse, then the cells containing zeroes are not saved in the output database.

Here is an example of a user table:

```
user_table ALLPS_Grads    // test table (combines ALLPS and Grads)
{
REP_SCHOOL_YEAR    //EN School year //FR Année scolaire
    * PS_EDLEV     //EN Level of study //FR Niveau de scolarité
    * {
        ENROL_F,  //EN Enrolments - female //FR Inscriptions - féminin
        GRAD_F,   //EEN Graduates - female //FR Diplômés - féminin
        ENROL_M,  //EN Enrolments - male //FR Inscriptions - masculin
        GRAD_M    //EN Graduates - male //FR Diplômés - masculin
    }
};
```

“user_table” is a Modgen keyword. To fill the cells of the user table the model must use a function called “UserTables”. Here is the prototype:

```
void UserTables();
```

Modgen supplies 2 functions called “GetTableValue” and “SetTableValue” to manipulate the contents of the tables. Here are the prototypes:

```
double GetTableValue( CString szExprName, ... );
void SetTableValue( CString szExprName, double dValue, ... );
```

The optional arguments (indicated by ‘…’) are the indices of the table’s classificatory dimensions. “szExprName” is the string that identifies both the table and the table expression. It consists of the table name followed by a dot (‘.’) and the expression name, for example "ALLPS.Expr0". The expressions of the regular table are always named “Expr0”,

“Expr1”, etc. The expressions of the user table are named in the table definition.

“GetTableValue” can read a cell of a regular table or a user table, “SetTableValue” can only set a cell of a user table (using “dValue” as a new cell value). Modgen calls “UserTables” for each sub-sample so that the standard errors and coefficients of variation can be calculated.

Modgen also supplies two functions, **GetUserTableReplicate()** and **GetUserTableSubSample()**, to make it possible, in the function UserTables of a model, to know in which subsample or replicate the function was called.

Here is an example of the “UserTables” function from the LifePaths model:

```
void UserTables()
{
    double dValue;
    int    nLevel;
    int    nYear;

    for ( nYear = 0; nYear < SIZE( REP_SCHOOL_YEAR ); nYear++ ) {
        for ( nLevel = 0; nLevel < SIZE( PS_EDLEV ); nLevel++ ) {
            dValue = GetTableValue( "ALLPS.Expr0", nYear, nLevel, FEMALE );
            SetTableValue( "ALLPS_Grads.ENROL_F", dValue, nYear, nLevel );
            dValue = GetTableValue( "ALLPS.Expr0", nYear, nLevel, MALE );
            SetTableValue( "ALLPS_Grads.ENROL_M", dValue, nYear, nLevel );
        }
    }
    for ( nYear = 0; nYear < SIZE( REP_SCHOOL_YEAR ); nYear++ ) {
        dValue = GetTableValue( "Grads.Expr1", nYear, FEMALE );
        SetTableValue( "ALLPS_Grads.GRAD_F", dValue, nYear, PSE_TVOC );
        dValue = GetTableValue( "Grads.Expr1", nYear, MALE );
        SetTableValue( "ALLPS_Grads.GRAD_M", dValue, nYear, PSE_TVOC );

        dValue = GetTableValue( "Grads.Expr2", nYear, FEMALE );
```

```
        SetTableValue( "ALLPS_Grads.GRAD_F", dValue, nYear, PSE_PI );
        dValue = GetTableValue( "Grads.Expr2", nYear, MALE );
        SetTableValue( "ALLPS_Grads.GRAD_M", dValue, nYear, PSE_PI );

        dValue = GetTableValue( "Grads.Expr3", nYear, FEMALE );
        SetTableValue( "ALLPS_Grads.GRAD_F", dValue, nYear, PSE_CC );
        dValue = GetTableValue( "Grads.Expr3", nYear, MALE );
        SetTableValue( "ALLPS_Grads.GRAD_M", dValue, nYear, PSE_CC );

        dValue = GetTableValue( "Grads.Expr4", nYear, FEMALE );
        SetTableValue( "ALLPS_Grads.GRAD_F", dValue, nYear, PSE_BA );
        dValue = GetTableValue( "Grads.Expr4", nYear, MALE );
        SetTableValue( "ALLPS_Grads.GRAD_M", dValue, nYear, PSE_BA );

        dValue = GetTableValue( "Grads.Expr5", nYear, FEMALE );
        SetTableValue( "ALLPS_Grads.GRAD_F", dValue, nYear, PSE_MA );
        dValue = GetTableValue( "Grads.Expr5", nYear, MALE );
        SetTableValue( "ALLPS_Grads.GRAD_M", dValue, nYear, PSE_MA );

        dValue = GetTableValue( "Grads.Expr6", nYear, FEMALE );
        SetTableValue( "ALLPS_Grads.GRAD_F", dValue, nYear, PSE_PHD );
        dValue = GetTableValue( "Grads.Expr6", nYear, MALE );
        SetTableValue( "ALLPS_Grads.GRAD_M", dValue, nYear, PSE_PHD );
    }
}
```

Computed tables

Computed tables represent a third different category of table that can be generated by Modgen. Unlike regular tables or user tables, however, computed tables are created after a model has been run or executed, and not "on the fly" as the model is being run. Since they are created after the completion of a model run, and because they are generated, modified, or removed through the visual interface to a Modgen model, the techniques for using computed tables are described in the Guide to the Modgen Visual Interface.

As basic background, however, computed tables can be produced to calculate either run differences or run ratios. When computed tables are being defined, the following needs to be specified:

- New table name (which must consist of alphanumeric characters or underscores only)
- New table description
- Original table name (existing table name from the current Modgen output database which thus must first be generated and present)
- Reference database name (output database generated by a different scenario)
- Reference table name (existing table name from the reference database)
- Computation Algorithm ("Table Difference" or "Table Ratio")

The original and reference tables must satisfy a few criteria before they can be compared:

- they must be generated with the same number of sub-samples (or replicates for time-based models),
- the rank must be identical,
- the position of the analysis dimension must be identical,
- the shapes of all dimensions must be identical.

Typically, computed tables are used to calculate differences or ratios of the same table in two different runs, which thus makes it very easy to satisfy these criteria. The dimension labels of the original table are used in the resulting computed table.

Each computed table definition is stored in an MS Access file named scenario(CTBL).MDB (e.g. BASE(CTBL).MDB). Note that this file is created only when truly needed, thus reducing overall file clutter. It is also opened only when needed, thus reducing the risk that a long-running simulation fails if at some point the computed table database cannot be accessed, perhaps due to network problems.

When a computed table is opened, the creation of a temporary database takes place first. For large tables (e.g. in the Demand model) this may take a while so the progress is reported in the status bar. This is also the case when such tables are being refreshed. Additional windows may be opened for a computed table, just as for run-generated tables. When additional windows are opened, the existing temporary database is re-used. The temporary database is deleted when all windows corresponding to the particular computed table are deleted. Make sure you have enough space on the temporary drive (usually C:) before creating computed tables.

Table aggregations for cell-based models

For cell-based models, aggregations are permitted to provide an on-the-fly aggregation capability during table viewing. To define an aggregation, first define two classifications: one containing the detailed list of items and one containing the aggregated list, e.g.:

```
classification OCCUPATION { //EN Occupation //FR Profession
      OCC_0012, //EN 0012 Snr Gov't Mgrs&Officials   //FR 0012 Cadres sup. de l'admin. publ.
      OCC_0013, //EN 0013 Snr Mgrs-Fin.,&Bus. Srvc   //FR 0013 Cadres sup. - serv. fin., télécomm. et serv. aux entr.
      OCC_0014, //EN 0014 Snr Mgrs-Hlth, Educ.       //FR 0014 Cadres sup. - santé, ens., serv. comm. et soc.
…
};

classification OCCUPATION3 { //EN Occupation ( 3-digits ) //FR Occupation ( 3-digits ) (FR)
      OCC3_001, //EN 001 Legislators and Senior Management
      OCC3_011, //EN 011 Administrative Services Managers
      OCC3_012, //EN 012 Managers in Financial and Business Services
…
};
```

Then the aggregation may be defined by using the following syntax:

```
aggregation aggregated_classification, detailed_classification {
    aggregated_item, detailed_item,
    aggregated_item, detailed_item,
    …
    aggregated_item, detailed_item
};
```

"**aggregation"** is a Modgen keyword, "aggregated_classification", "detailed_classification", "aggregated_item" and "detailed_item" refer to previously defined aggregated and detailed classifications and their items. Note that there is no comma before the closing '}'.

Example:

```
aggregation OCCUPATION3, OCCUPATION {
    OCC3_001,    OCC_0012,
    OCC3_001,    OCC_0013,
    OCC3_001,    OCC_0014,
…
};
```

The current aggregation for the Table window may be changed by using the Dimensions tab of Table properties. It contains an Aggregations column if any dimension can be aggregated. The dimensions which may be aggregated display the current aggregation in the grid cell and will display the drop-down boxes when the cell receives the input focus. At that point, a different aggregation may be selected from the drop-down list.

Note also that computed tables generated in cell-based models as run differences can be aggregated, but computed tables generated as run ratios cannot.

Examining the Individual Lifetimes of Actors

Examining Actor Histories

The life progression of each simulated case can be examined either visually, through the use of the Biographical Browser, or through the results reported in the debug ASCII text files. The purpose of these two tools is to aid in uncovering possible algorithmic errors in the model, or to study some particularly interesting cases with respect to the specified model. To create the actor histories use the track command within a .mpp file, re-compile the model and launch the simulation using either the MS Access tracking or Text tracking Output Options from the Execution Control dialog of Modgen's visual interface. Then, examine the MS Access tracking output using the Modgen Biographical Browser software, documented in a separate User's Guide. Examine the text tracking output using your preferred text editor.

The contents of the MS Access tracking database and the rules used when creating the tracked data are described in detail in the Reference Material section entitled “Modgen Tracking Database” on page 187. Tracking is expensive in terms of both storage space and simulation time. For efficiency reasons the tracking database is initially generated in the TEMP directory and then copied to its final requested location. Before attempting to generate a large tracking file make sure that you have enough space available on your temporary drive (usually C but it can be changed by modifying the TEMP environment variable) and the output drive.

The Track Command

The activation of case by case results is activated through the use of the Modgen track command. Only one track definition is allowed for each actor in the model. A filter can be specified in order to examine only a subset of all simulated cases. The filter is a logical expression. For case-based models, the filter can contain an optional final filter. Initial filtering is used for tracking actors while they are in a certain state. Final filtering is used for tracking actors who were ever in a certain state.

Time-based models should generally arrange to track actors from a single replicate to avoid possible confusion when viewing the results in BioBrowser.

The syntax of this command is as follows:

Syntax

track actor_name [initial_filter<,final_filter>] { state_or_link , ... , state_or_link } ;

Instead of listing each state or link to be tracked, Modgen allows two alternative keywords: "all_derived_states" or "all_base_states". The former adds all derived states of the actor to the tracking database; the latter adds all simple states, excluding internally generated states associated with table filters, the 'unit' keyword, or actor set filters.

Example1

```
// This example is not part of the simpex model. In this example,
// individual cases of the child actor are stored once they are
// attached to a set of parents.  Two states are being
// reported over the course of each child's simulated lifetimes. These
// are the child's sex and date of birth.  The multiple link to its
// parents is also being stored for the entirety of the child's lifetime.
track Child
[ !tentative ]
{
    sex,
    date_of_birth,
    mlParents
};
```

Example2

```
// Both filters used
track Person
[dominant && cohort, ever_borrow]
{
```

…

```
};
```

Example3

```
// Final filter only
track Person
[,ever_borrow]
{
…
};
```

The actor_id State and the Modulus (Remainder) Operator

The actor_id state can be used either in identity expressions or in filters. The actor_id state pertains to the value of the object identifier. Its primary use is to track a subset of actors with identical characteristics in conjunction with the modulus operator %. The modulus operation produces the remainder when a number is divided by another. The value of the operation is zero when the numerator is exactly divisible by the denominator.

Example

```
// The following example is not part of the simple example code.
// in this example every 20th consumer actor is being tracked.
// Note that the actor_id state is used in conjunction with the modulus
// arithmetic operator.
track Consumer
[ 0 == actor_id % 20 ]
{
    savings,
    last_quantity,
```

```
    last_price,
    lLastStore
};
```

Modgen-generated states for tables

For each table, Modgen generates a state that implements the table's filter expression and a state that implements the table's 'unit' state (if the table uses 'unit'). These states have predictable names that can be used to facilitate exploration of tabulation by specifying them using the `track` statement for export to BioBrowser. Filters have names of the form `table_filter_X` and units of the form `table_unit_X`, where `X` stands for the name of the associated table. It is also possible to use these states in subsequent tables, i.e. tables that occur further on in the source code module. This use is recommended for debugging or exploratory purposes only and should not be used in production code. Use of these states is illustrated in the following example:

```
track Person
[ dominant ]
{
      province,
      region,
      table_unit_CENSUS_COUNTS,
      table_filter_CENSUS_COUNTS
};

table Person CENSUS_COUNTS //EN Census counts
[ dominant && IntIs_CENSUS_YEAR( year) ]
{
      census_year+ * { unit, duration() }
};

table Person CENSUS_COUNTS_CHECK //EN Debug the Census table
```

```
[ dominant ]
{
      { duration() }
      * sim_year+
      * table_filter_CENSUS_COUNTS //EN Filter for CENSUS_COUNTS
};
```

The tables that result from a test model are shown below.

Table: CENSUS_COUNTS

Table Description: Census counts

Data: Value

	Selected Quantities	
Census Year	unit	duration()
1971	3	2.5
1976	3	3
1981	2	2
1986	2	2
1991	1	1
1996	1	1
2001	0	0
All	12	11.5

Table: CENSUS_COUNTS_CHECK

Table Description :Debug the Census table

Data: Value

	Filter for CENSUS_COUNTS	
Simulation year	False	True
1970	29	0
1971	0	2.5
1972	3	0
1973	3	0
1974	3	0
1975	3	0
1976	0	3
1977	3	0
1978	2.5	0
1979	2	0
1980	21	6
All	69.5	11.5

Advanced Topics in Modgen

Events

Actors evolve through time through the use of user-supplied event functions. An event function has two parts. The first part returns the time to wait for the given event to occur, conditional on no other event occuring first. The second part of the event function consists of code that implements the given event by changing the values of states. An example of an event is the loss of a job. In this example, a hazard of job loss as a function of an individual's characteristics, such as duration of current job, could be used in the waiting time portion of the event function. The implementation part of the event function would consist of code that set states that describe the individual's new employment status and earnings rate after job loss. Modgen, by analyzing the implicit dependencies in the waiting time portion of all event functions, calls the various waiting time event functions and event implementation functions to move the actors in a simulation through time.

As stated above, two C++ functions must be coded by the developer to correspond to each model event. All elements of the C++ language can be used in these functions. Both of these functions are declared in the *event* definition which is a component of the actor definition.

Syntax

```
event time_function, implement_function <, priority_code>;
```

time_function must return the *absolute* time of the next occurrence of the event. It has either no argument or an optional argument that is a pointer to an int variable, as described in the next section, Events with Memory. There are two rules which must be followed when coding a time_function. Firstly, you can not set actor states, you can only reference them. Secondly, you can not use pointers within these functions. The Modgen compiler will end with errors if these rules are broken.

implement_function generally has no coding rules. It returns no result, and it has either no argument or an optional argument that is an int variable, as described in the section “Events with memory” on page 99.

priority_code is optional. When present, it must be an integer in the range of 0 to 250, or it can instead be the keyword

“time_keeping”. The time_keeping keyword, however, should only be used in the declaration of an event whose purpose is to keep track of time, such as the “Clock” event in the LifePaths model and the “Ticker” event in some time-based models. Events with this priority will automatically be assigned a higher priority than the events for which the priority was declared by the model developer; however, events with the time_keeping priority will have a slightly lower priority than that assigned to the event created for updating self-scheduling states .

Priority codes are checked when there is a tie between two or more events. In such case, the event with the higher priority code is implemented first. If priority_code is absent, Modgen assigns a priority code of 0 (i.e. the lowest priority) to the event. In case of a tie between priorities, the events are executed in alphabetical order of event name. In case of a tie between identically-named events of different actors, the order of creation of actors is used, which means the event for the first actor created will be executed first. Priorities should be assigned to events when there is a possibility of having simultaneous events. In particular, the "Clock" or "Ticker" event should always have the highest priority, which will automatically be the case if they are declared with the time_keeping priority. Note also that in some cases, it would give better results to use a hook rather than multiple simultaneous events.

Example1

```
// The following line appears in the actor definition.
// The timeAgeOneYear function estimates the waiting time to the event.
// The AgeOneYear function implements the appropriate actions on the actor.
// In our simple example, they belong to the Dwelling actor

event timeAgeOneYearEvent, AgeOneYearEvent;
```

Example2

```
   event timeUniStartEvent, UniStartEvent, 1;
   event timeMonthlyEvent, MonthlyEvent;
```

In the above example, in case of a tie, “UniStartEvent” will be implemented before “MonthlyEvent” because of the higher priority code.

However, if the events are in fact simultaneous events, they are treated as distinct during tabulation, rather than being "amalgamated" into a single tabulation "super-event" in which tables would only be updated immediately before the time changed, rather than after each event. This means that tables are updated before the next event starts regardless of the time. Thus, if two events occur simultaneously, the tables are updated twice.

Note that tables are insensitive to "intermediate" variable changes that occur during the implementation of an event. Only the values of states at the end of the implementation of an event have an effect on tables.

Events with memory

Modgen also includes the ability to record auxiliary information in the "time" function of an event that can later be retrieved and used by the associated "implement" function of the event. This capability can be useful in situations where the waiting time is determined together with numerable particularities about the eventual event implementation. An example is the migration event of LifePaths, where a potential migration event has a waiting time as well as an associated destination that is inextricably associated with the waiting time.

It is possible for the "time" function of the migration event to record the associated destination, so that the destination can be set by the migration "implement" function if and when the migration occurs. This ability to save and retrieve event-specific auxiliary information can be activated as desired on an event-by-event basis.

To activate this capability for a particular event, the "time" and "implement" functions of the event need to be coded to take an argument . Specifically, the "time" function needs to take as argument a pointer to an "int", and the "implement" function needs to have an argument of type "int". The following example illustrates the process.

```
actor Person
{
      event timeClockEvent, ClockEvent, 250;
};

TIME Person::timeClockEvent(int *pnEventInfo)
{
      // Get year portion of next gong.
      int nGongYear = (int) next_gong_time;
      // The following records the calendar year of the next ClockEvent.
      *pnEventInfo = nGongYear;
      return next_gong_time;
}
```

```
void Person::ClockEvent(int nEventInfo)
{
      // The following retrieves the previously stored nGongYear.
      int nGongYear = nEventInfo;
      // code that uses nGongYear
}
```

just_in_time

This keyword enables a different algorithm for advancing time that can result in dramatic improvements in computational efficiency, especially for time-based models with many actors.

In all models, time advances when an event occurs. By default, the time for **all** actors is always updated when **any** event occurs. The optional 'just-in-time' keyword, however, causes time to only be updated for the actor for whom the event occurs, as well as for any other actor affected by the event, i.e. whose states will change as a result of the event. The model developer enables this capability by including the 'just_in_time' keyword in the model type declaration, as follows:

```
model_type  case_based,just_in_time;
model_type  time_based,just_in_time;
```

The use of 'just_in_time', however, imposes some rules on the use of continuously updated states in the model. A "continuously updated state" is a state that falls into one of the following three categories:

- The state is continuously updated
- The state is derived from a continuously updated state or another state which is derived from a continuously updated state, etc.
- The state has the same name (member name) as another state in another actor which is in one of the previous two categories

(Modgen produces a list of the continuously updated states, as well as how they are used, in the help file of the model. The list is found in the link "Continuously updated states and their dependencies" in the help page for each type of actor

in the model.)

The rules that must be respected by the model are as follows:

- The use of continuously updated states in the **time** function of an event is not allowed with just_in_time.
- When just_in_time is specified, a state cannot be derived from a continuously updated state **from another actor**.
  Examples:

  ```
  double test = lSpouse->age ;
  ```

  would not be allowed with just_in_time but

  ```
  double test = age;
  ```

  would. This also means that sumover, etc. are not allowed to have a continuously updated state as an argument.
- In functions that are not member of actors, reading continuously updated states is not allowed with just_in_time.

Any violation of these rules is detected by Modgen during compilation and leads to an appropriate error message. The easiest way of being sure your model respects these rules is to avoid using continuously updated states except in the analysis dimensions of tables

Additional Functions Related to Events or Advancement of Time in Secondary Loops of Events

The function GetCurrentEvent allows the model to peek at the event being currently implemented, while the function GetNextEvent allows a peek at the next event in the queue (but without actually implementing that next event). Such functions can be useful when implementing secondary loops of events, such as in the spouse generation portion of the LifePaths model. Together, they can be used to determine if the next event is to be implemented in the secondary loop or if it is time to stop simulating events in the secondary loop and return to the main loop of events.

The function WaitUntilThisActor allows Modgen to advance the time for only one actor in models without the just_in_time algorithm. Such a function should be used only in secondary loops of events, again such as in the spouse generation portion of the LifePaths model. Advancing time for all actors in secondary loops can lead to inconsistencies in tabulation, and thus this function helps to avoid such problems.

Defining and using function hooks

Modgen allows for the splitting of a single actor function among more than one file by providing function *hooks*. This is especially useful for certain standard functions common to all model type such as Start() and Finish(). Different modules of a model may need to add some code to these functions. They can do so by defining function hooks. A function hook is a member function of an actor which takes no parameters and return no result. The prototype and definition of such functions is similar to the other member functions. One additional definition (hook definition) is required inside an actor definition. The syntax is as follows.

Syntax

hook HookFrom , HookTo ;

where *HookFrom* must be a member function which takes no arguments and returns no result and *HookTo* can be any member function or event function other than the event time function. One function can be hooked to more than one other functions and more than one function can be hooked to the same function. The following is an example from the LifePaths model.

Example

```
file: Education.mpp

actor Person

{

...

    void    StartEducation();

    hook    StartEducation, Start;

...

};


void Person::StartEducation()

{

    prov_of_study = (PROV_OF_STUDY) min( prov_of_res, SIZE( PROV_OF_STUDY )  - 1
);

    school_year_start = next_year + (TIME) ( 5 + 8.0 / 12.0 );
```

```
    school_year_end = school_year_start + (TIME) ( 10.0 / 12.0 );
}
```

If the function to which other functions are hooked has a preferred place where all hooks should be inserted, then it can include the following line in its definition. Only one such line can exist in a function definition.

IMPLEMENT_HOOK();

If IMPLEMENT_HOOK is not encountered then all hooks will be inserted at the end of the function. Note that the order in which the hooks are implemented upon encountering this command is completely arbitrary.

Example from LifePaths model

```
void Person::BirthdayEvent()
{
    IMPLEMENT_HOOK();
    if ( sex == FEMALE && curtate_age >= MAX( FERT_AGE ) ) {
        if ( fecundstat == FERTILE || fecundstat == POSTPARTUM ) {
            fecundstat = MENOPAUSE;
        }
    }
    curtate_age ++;
}
```

Run-time Functions and Macros

The following run-time C++ functions and C++ macros are used extensively in the functions of a microsimulation model. They can be used anywhere in Modgen code. The following functions fall into no particular category and are presented in alphabetic order. The remaining functions are categorized by use. The run-time macros are also presented separately.

int **EmptyDistributions_**<parameter_name>()

Modgen generates EmptyDistribution_() functions for every parameter of type cumrate. The model developer can use these functions to identify cumrate parameters with empty distributions and to take appropriate action, such as terminating the simulation or writing a warning message using “WriteLogEntry()”.

“EmptyDistributions_<parameter_name>()” returns the number of empty distributions contained within the cumrate parameter parameter_name. For model-generated cumrate parameters, the function will return 0 if called in “PreSimulation()”, but it will return the actual number of empty distributions if called in simulation code or in “UserTables()”.

int **GetThreadNumber**();
This function returns the thread number of the current simulation thread (in origin 0, i.e. the first thread will return 0, the second thread will return 1 etc.).

bool **Lookup_**<parameter_name>(long number, int index1, ,,,, int indexK, int *dest1, ..., int *dest)
Modgen generates Lookup_() functions for every parameter of type cumrate. They draw values from discrete probability distributions specified through the parameter identified by parameter_name. The index variables contain values of indices in non-cumulated dimensions (they must be of type int or classification) and dest variables are destination variables that receive the values of cumulated dimensions (they must be of type int).

It is not unusual for cumrate parameters to contain empty distributions (all zeros), either for structural or sampling reasons. In order to allow model developers to detect and handle empty distributions in cumrate parameters, the “Lookup__<parameter_name>()” function returns a value of TRUE if the underlying distribution is non-empty, and a value of FALSE otherwise. This allows a model developer to take appropriate fall-back action if an attempt is made to draw from an empty distribution during the simulation. Note also that “Lookup__<parameter_name>()” returns (through its trailing arguments) the first index of the distribution, if the distribution is empty.

Example:

```
Lookup_CumBirth( RandUniform(), eSex, &nYear, &nProv );
```

double **PieceLinearLookup**(double dNumber, double *pdXY, int nSize);

double **PieceLinearLookup**(double dNumber, double *pdX, double *pdY, int nSize);
This Modgen supplied function used for linear interpolation is available in two forms. In both cases “dNumber” is the x-value for which the corresponding y-value is returned as a result. In the first case, “pdXY” points to the vector of value pairs (x1, y1, x2, y2, ...) and “nSize” contains the length of that vector (i.e. 2 * number of pairs). In the second case, “pdX” points to the vector of x-values (x1, x2, ...), “pdY” points to the vector of y-values (y1, y2, ...) and “nSize” contains the length of both vectors (i.e. number of pairs). The X and Y points can come from different parameters and for parameters of higher dimension will be taken from the last dimension.

int **sgn**(double dValue);
Returns the sign of dValue (-1, 0, 1).

void **SignalCase**();

Notifies Modgen that the case has completed. Updates the simulation progress bar. It should be called in the case loop of "Simulation" function, just after "CaseSimulation" function is called.

void **StartCase**();
Notifies Modgen that the next case is about to start. It should be called in the case loop of "Simulation" function, just before "CaseSimulation" function is called.

void **TotalSum**(const char *szTableName);
For user tables, to insert the sum in the total columns of the table.

Functions which modify the progress window

void **CalculationProgress**(double dPctDone);
For cell-based models, the progress update is done by the model's code (either in the Simulation or the UserTables function) by calling this function.

void **ModelExit**(const char *szMessage);
This function is used to exit the model cleanly when errors are detected during the run. It takes a string argument which will be added to the "**Errors and Warnings**" box in the Model Run Progress window and the model will exit cleanly (no outputs will be generated in such cases).

CString **ModelString**(CString szName);
ModelString may be used to output multilingual strings from the model. It will return the label associated with string szName in the language of the simulation. It is particularly useful with ProgressMessage function.

Example

```
string S_GENERATING_SPOUSES; // Generating spouses
```

…

```
sprintf( szMsg, "%s: %ld / %ld", ModelString( "S_GENERATING_SPOUSES" ),
lSpouseInd, SpouseMarketSize );
ProgressMessage( szMsg );
```

ProgressMessage(const *char);
This function is used to display a message in the "**Progress**" window when running a model, especially if there is a time-consuming activity happening within a case or at the beginning of the simulation e.g. ProgressMessage**("Generating spouses: 20 / 5000**");

void **TimeReport**(*double Time*);
Updates the simulation progress bar of time-based models. The first call to the function TimeReport() is used only to

specify the time at the beginning of the simulation, but this call to TimeReport() is mandatory. (The call should take place in the function Simulation() before the loop that does the simulation.) The progress bar will use the time given in the argument on that first call to TimeReport() to calculate the percentage done.

WarningMsg(const *char);
Displays a warning message in the progress window.

Functions associated with the random number generator

void **GetRandState**(CRandState &rRandState);
Returns by reference a complete set of seeds and deviates.

double **RandLogistic()**;
This function returns a Logistic deviate using the inverse cdf where cdf = « cumulative density function » (F(x) = 1/(1 + exp(-x))). A model developer may use this function either with no argument or with a unique constant integer argument; however, if no argument is provided, Modgen actually alters the .mpp file by assigning an available integer as the argument.

double **RandNormal()**;
This function returns a normal random deviate. A model developer may use this function either with no argument or with a unique constant integer argument; however, if no argument is provided, Modgen actually alters the .mpp file by assigning an available integer as the argument.

double **RandUniform()**;
This function returns a Uniform floating point random number in the range 0-1.A model developer may use this function either with no argument or with a unique constant integer argument; however, if no argument is provided, Modgen actually alters the .mpp file by assigning an available integer as the argument.

void **SelectRegularRandomGenerator**();
Switches back to the random number generator assigned to the current case, after using one of the special generators below.

void **SelectSpecialRandomGenerator1**();

void **SelectSpecialRandomGenerator2();**
Allows the model to use different random number generators for different activities (e.g. generation of spouses).

void **SetRandAuxState**(long lRandSeed);
Sets the auxiliary random generator seed to the new value.

void **SetRandState**(CRandState &rRandState);
Sets a complete set of seeds and deviates.

void **SetRandState(**long lSeed,bool bResetDeviates = true);
Used to modify the random number generator state based on a single seed argument, which may be useful for modifying clones and/or at case initialization. The second argument is optional, by default all random deviates will be reset i.e. no leftover deviates will be used.

Functions used for case weighting and population scaling

long **GetAllCases**();
Returns the number of requested cases.

int **GetCaseSample**();
Returns the sub-sample number associated with current case.

void **GetCaseWeight**(double *pdCaseWeight, double *pdCaseSubsampleWeight);
Returns the case weight and case sub-sample weight through pointer aguments.

int **GetDistributedSubSamples**();
Returns the number of subsamples that will be simulated on the current machine. If the simulation is not distributed, that number is the total number of subsamples specified in the scenario settings. If the simulation is distributed, that number is the number of subsamples by machine specified in the scenario settings..

int **GetFirstSample**();
Returns the number of the first subsample simulated on the machine. Subsamples are numbered starting with 0. If the simulation is not distributed, the number of the first subsample is always 0. If the simulation is distributed, the number is equal to the slave number multipled by the number of subsamples by machine. For example, if there are 8 subsamples per machine, the number of the first subsample simulated by the first slave (slave 0) is 0, the number of the first subsample simulated by the second slave (slave 1) is 8, the number of the first subsample simulated by the third slave (slave 2) is 16, etc.

long **GetPopulation**();
Returns the target population used for population scaling.

bool **GetPopulationScalingRequired**();
Returns TRUE if the user has checked the box “Population Scaling” on the Scenario Settings dialog of the visual interface, and FALSE otherwise.

int **GetReplicates**();
Returns the number of replicates for a time-based model.

int **GetSubSamples**();
Returns the number of sub-samples.

long **GetSubSampleCases**(int nSample);

Returns the number of cases in the specified sub-sample.

int **GetThreads**();

Returns the number of simulation threads.

void **NoteCaseWeight**(double dCaseWeight, double dSubsampleWeight);

Notes the case weight and sub-sample weight. The case weight used for population scaling is determined in the following order:

1.The last weight supplied by NoteCaseWeight.

2.If NoteCaseWeight was not called then the weight in effect at case end is used.

The sub-sample weight is optional. If the second argument is not used then the sub-sample weight noted will be equal to case weight multiplied by the number of sub-samples, which is appropriate for most cases. However if sub-sample weight is assigned a non-default value (e.g. if using post-stratification in LifePaths model) and cloning is used then NoteCaseWeight should be called with 2 arguments.

void **RestoreSubSampleSeeds**();

Restores starting seeds in all sub-samples.

void **SetCaseWeight**(double dCaseWeight, double dCaseSubsampleWeight);

Sets the case weight and case sub-sample weight.

void **SignalSubSampleCase**(int nSample);

Signals (ends) the next case in the specified sub-sample.

void **StartSubSampleCase**(int nSample);

Starts the next case in the specified sub-sample.

The following 2 constants are also available:

MAX_SUBSAMPLES

Maximum number of sub-samples that can be used by a model (currently 42).

MAX_SUBSAMPLES_PLUS1

MAX_SUBSAMPLES+1, useful for creating sub-sample totals.

Functions associated with date and time

Modgen supplies a number of functions that assist in the handling of sub-annual calendar time. These functions use three different ways of expressing time, referred to below as "**model time**", "**day number**" and "**calendar**". Most of the functions in this section allow users to convert from one way of expressing time to another.

"**Model time**" is time expressed as a continuous real number where a "year" has a value of 1.0.

"**Day number**" is an integer counter of days. It uniquely identifies and sequentially counts elapsed days starting from a fixed day in the distant past.

"**Calendar time**" re-expresses the current day into the corresponding year, month, and day of month.

There are two different calendars which have been incorporated in these functions. The Gregorian calendar years vary in length between 365 and 366 days, making them unsuitable as a unit of measurement. By "Gregorian Standard Year" we mean 365.2524 days, the average length of a year in the Gregorian calendar used today. The other calendar is the Leapless calendar. It has a constant year length of 365 days which means that February 29th never occurs in this calendar. Both calendars are split into years, months, and days of the month. The day of the week (e.g. Monday to Sunday) is also provided.

Note that in the Gregorian calendar, the integer part of model time is not necessarily the associated calendar year, although it will always be very close. For example, January 1, 1988 is day number 2447162, and has an associated model time of 1987.9997535884.

In the functions described below, note that month of the year, day of the month, and day of the week are expressed using 0 as origin. In other words, January is month 0 of the year and December is month 11. Similarly, Monday is day 0 and Sunday is day 6. Weeks always start on Monday. The first day of the month is numbered 0.

Certain other date-related quantities can be derived using these functions. For example, a continuously increasing counter of weeks can be constructed with an expression like nDay/ 7. A continuously increasing counter of months can be constructed with an expression like nYear * 12 + nMonthOfYear.

Gregorian functions:

double **GregorianDayLength** ();

Returns the length of a day in "Gregorian Standard Years". This is the reciprocal of 365.2524, appropriately rounded for

precision.

int **GregorianCalendarToDay** (int nYear, int nMonthOfYear, int nDayOfMonth);

Returns the day number associated with a given date (4-digit year, month of year [0-11] and day of month [0-30]) on the Gregorian calendar.

double **GregorianCalendarToTime** (int nYear, int nMonthOfYear, int nDayOfMonth);

Returns the model time associated with a given date (4-digit year, month of year [0-11] and day of month [0-30]) on the Gregorian calendar.

double **GregorianDayToTime** (int nDay);

Converts a day number to the corresponding model time.

bool **GregorianIsLeapYear** (int nYear);

Given a 4-digit Gregorian calendar year, indicates if it is a leap year.

int **GregorianTimeToDay** (double dTime);

int **GregorianTimeToDay** (double dTime, *pnIsDayBoundary);

Converts a model time to a day number. Any fractional part of the day is ignored, i.e. the time is rounded down to the nearest day. The second argument, which is optional, returns a flag which is set to true if dTime corresponds to an exact day boundary. Otherwise the flag is set to false.

void **GregorianDayToCalendar** (int nDay, int *pnYear, int *pnMonthOfYear, int *pnDayOfMonth, int *pnDayOfWeek);

Given a day number, returns the corresponding year, month of year [0-11], day of month [0-30], and day of week [0-6] in the Gregorian calendar.

void **GregorianTimeToCalendar** (double dTime, int *pnYear, int *pnMonthOfYear, int *pnDayOfMonth, int *pnDayOfWeek);

Given a model time, returns the corresponding year, month of year [0-11], day of month [0-30], and day of week [0-6] in the Gregorian calendar.

Leapless functions:

double **LeaplessDayLength** ();

Returns the length of a day in "Leapless Standard Years". This is the reciprocal of 365, appropriately rounded for precision.

int **LeaplessCalendarToDay** (int nYear, int nMonthOfYear, int nDayOfMonth);

Returns the day number associated with a given date (4-digit year, month of year [0-11] and day of month [0-30]) on the Leapless calendar.

double **LeaplessCalendarToTime** (int nYear, int nMonthOfYear, int nDayOfMonth);

Returns the model time associated with a given date (4-digit year, month of year [0-11] and day of month [0-30]) on the Leapless calendar.

double **LeaplessDayToTime** (int nDay);

Converts a day number to the corresponding model time.

bool **LeaplessIsLeapYear** (int nYear);

Always returns true. This function is only present so that the “Leapless” family of functions contains an analogous function for each member of the Gregorian family of functions.

int **LeaplessTimeToDay** (double dTime);

int **LeaplessTimeToDay** (double dTime, *pnIsDayBoundary);

Converts a model time to a day number. Any fractional part of the day is ignored, i.e. the time is rounded down to the nearest day. The second argument, which is optional, returns a flag which is set to true if dTime corresponds to an exact day boundary. Otherwise the flag is set to false.

void **LeaplessDayToCalendar** (int nDay, int *pnYear, int *pnMonthOfYear, int *pnDayOfMonth, int *pnDayOfWeek);

Given a day number, returns the corresponding year, month of year [0-11], day of month [0-30], and day of week [0-6] in the Leapless calendar.

void **LeaplessTimeToCalendar** (double dTime, int *pnYear, int *pnMonthOfYear, int *pnDayOfMonth, int *pnDayOfWeek);

Given a model time, returns the corresponding year, month of year [0-11], day of month [0-30], and day of week [0-6] in the Leapless calendar.

Functions used for transformations between integers and classifications

Modgen has functions that facilitate the use of classifications which implicitly map to integer values. An example of such a classification is a selected subset of years or a subset of ages.

For each classification having at least one level whose name ends in a number, Modgen will create three functions. These functions are named IntIs_xxx(integer), IntTo_xxx(integer), and IntFrom_xxx(level), where “xxx” is the name of the classification. For the purposes of these functions, each level of the classification is associated with a numeric value corresponding to the trailing numeric digits of the level’s name. For example, the classification level YR_1991 would be associated with the integer 1991. The function IntIs_xxx(integer) will return TRUE if integer is found among the levels of classification xxx. The function IntTo_xxx(integer) will return the level of classification xxx that corresponds to integer. If integer is not found in classification xxx, IntTo_xxx(integer) will return the last level that has no associated numeric value (e.g. YR_NA). If no such non-numeric level exists in classification xxx, the last level of the classification is returned. If multiple levels share the same integer value, the first such level is returned. The IntTo_xxx(integer) function returns a value of type xxx, which means that no casting of the return value is normally necessary. Finally, the IntFrom_xxx(level) function converts a level from the classification xxx to the corresponding integer value. If the level does not have an associated numeric value (e.g. YR_NA), the value –1 is returned.

The use of these functions is most easily understood through examples. In the code that follows, note how the IntIs_CENSUS_YEAR() function is used in the table filter to restrict the table to report on Census years only, and how the IntTo_CENSUS_YEAR() function is used to map the integer state ‘year’ to the classification CENSUS_YEAR.

```
classification CENSUS_YEAR { //EN Census Year
YR1971, //EN 1971
YR1976, //EN 1976
YR1981, //EN 1981
YR1986, //EN 1986
YR1991, //EN 1991
YR1996, //EN 1996
YR2001 //EN 2001
};

actor Person {
CENSUS_YEAR census_year = IntTo_CENSUS_YEAR( year ); //EN Census Year
};

table Person CENSUS_COUNTS //EN Census Counts
[ dominant && IntIs_CENSUS_YEAR( year ) ]
{
census_year * { unit }
};
```

The following example shows how the IntFrom_ function can be used to generate a table using levels selected from a larger classification.

```
classification FOS { //EN Field of study
FOS1, //EN Mathematics
FOS2, //EN Economics
FOS3, //EN Sociology
FOS4, //EN Anthropology
```

```
FOS5, //EN Psychology
FOS6, //EN Kinesiology
FOS7 //EN Ontology
};

classification FOS_SEL { //EN Field of study (selected)
FOS_SEL2, //EN Economics
FOS_SEL3, //EN Sociology
FOS_SEL5 //EN Psychology
};

actor Person {
FOS fos; //EN Field of study
FOS_SEL fos_sel = IntTo_FOS_SEL( IntFrom_FOS( fos ) ); //EN Field of study
};

table Person FOS_ALL //EN Graduates
[ dominant ]
{
fos * { unit }
};

table Person FOS_SELECTED //EN Graduates
[ dominant && IntIs_FOS_SEL( IntFrom_FOS( fos ) ) ]
{
fos_sel * { unit }
};
```

Run-time Macros

CASES( )

Returns the total number of cases to be simulated by the current thread (when a case-based model is being executed).

COERCE(range_name**,** state_name)

Forces the value of the state to fall within the lower and upper limits of a range.

Example

```
range REP_YEAR70 { 1970,1979 };

COERCE(REP_YEAR70, year);
```

Returns the actual year if the state, year, is between 1970 and 1979 inclusive; returns 1970 if year is less than 1970; returns 1979 if year is greater than 1979. The COERCE statement in this example is equivalent to:

```
(year < MIN(REP_YEAR70)) ? MIN(REP_YEAR70) : (year > MAX(REP_YEAR70)) ?
MAX(REP_YEAR70) : year;
```

MAX(range_name)

Returns the maximum value of the range.

MIN(range_name)

Returns the minimum value of the range.

POINTS(partition_name)

Returns the values defining a partition as an array of floating point numbers. Thus POINTS(partition)[0] gives the first point in the partition, POINTS(partition)[1] the second point, etc.

Example:

If the following partition is defined:

```
partition AGE_PARTITION //EN Age group
{
      15, 45, 65
};
```

Then **POINTS**(AGE_PARTITION)[0] is equal to 15, **POINTS**(AGE_PARTITION)[1] is equal to 45 and **POINTS**(AGE_PARTITION)[2] is equal to 65.

The following code computes the lower and upper bounds of the age group nAgeGroup.

```
int nAgeGroup;
double dLower;
double dUpper;

// lower limit of age group
if ( nAgeGroup == 0 ) {
    // set lower to be minimum age (zero)
    dLower = 0.0;
}
else {
    dLower = POINTS(AGE_PARTITION)[nAgeGroup - 1];
}

// upper limit of age group
if ( nAgeGroup == SIZE(AGE_PARTITION) - 1 ) {
    // set upper to be maximum age in model
    dUpper = MAX(LIFE_SPAN);
}
else {
    dUpper = POINTS(AGE_PARTITION)[nAgeGroup];
}
```

Note that the macro *SIZE*(partition) gives the number of intervals in the partition, and not the number of points that define the partition. This is consistent with the use of *SIZE* for ranges and classifications: *SIZE* gives the number of distinct values in the range, classification, or partition. The number of intervals in a partition is always one more than the number of points defining the partition. In other words, the number of elements in the array *POINTS*(partition) is always equal to *SIZE*(partition) – 1. Note that because index values of arrays in C++ always start at 0, the maximum allowed index for the array *POINTS*(partition) is *SIZE*(partition) – 2

RANGE_POS(range_name**,** state_name)

Converts range values to 0-origin indices. This is particularly useful for indexing the array of parameters where range is one of the dimensions. RANGE_POS also handles out of range values, forcing them to fall within the range.

Example

```
// the following example is not taken from the simple example
```

```
range CAL_YEAR   { 1899, 1999 };

nYearInd = int( current_time );

fte_earnings = 52 * weekly_hours * earn_rate*

AvgIndWage[RANGE_POS( CAL_YEAR, nYearInd )];
```

SIMULATION_END()

Returns the time when simulation of time-based models ends (it starts at time 0).

SIZE(symbol)

Returns the size of the range, classification or partition indicated by symbol.

SPLIT(dValue, partition_name)

Partitions dValue by the classification groups defined in partition_name . This function is typically used in the "PreSimulation" function to initialize model-generated parameters which use a partition as one of the dimensions.

Note the SPLIT uses the same arguments as "split" which is used to define a derived state. Thus, SPLIT can only be used in simulation code while "split" can only be used in actor or table definition.

WAIT(wait_time)

Converts the waiting time wait_time to the absolute time.

WITHIN(range_name**,** state_name)

Returns a logical value indicating whether a given value of the state falls within the upper and lower bounds (inclusive) of the given range.

Example

```
range REP_YEAR70 { 1970,1979 };

WITHIN(REP_YEAR70, year);
```

The code in the WITHIN statement is equivalent to

```
year >= MIN(REP_YEAR70) && year <= MAX(REP_YEAR70);
```

The WITHIN macro can be used to simplify model code used that restricts tabulation to certain ranges. For example, the code

```
actor Person {

REP_YEAR70 rep_year70 = year; //EN The seventies
```

```
logical rep_year70_filter = year >= MIN(REP_YEAR70) && year <= MAX(REP_YEAR70);
};

table Person DECADE70
[ dominant && rep_year70_filter ]
{
rep_year70 * {unit}
};
```

can be replaced by

```
actor Person {
REP_YEAR70 rep_year70 = year; //EN The seventies
};

table Person DECADE70
[ dominant && WITHIN(REP_YEAR70, year) ]
{
rep_year70 * {unit}
};
```

Accessing Model Parameters in C++ Functions

There are several ways of accessing model parameters in the Modgen language. Each way is specific to the model parameter type.

numeric type parameters

Access to numeric model parameters is done through simple assignments.

Example

```
// the state, tfem, is being assigned the value in the scalar
// model parameter, FemaleProp.
tfem = FemaleProp;
// the state, eSex, is being assigned the value of zero
// if the random number, RandUniform(), is less than or equal to
// the value of FemaleProp.
// the state, eSex, is assigned the value of one otherwise
eSex = ( RandUniform() <= FemaleProp )
// the state, z, is being assigned a value in the model
// parameter matrix, ProbMort. The value corresponds to the location in
// the array defined by the current value of the classification sex and
// age-1
z = ProbMort[sex][age - 1]
```

cumrate type parameters

Access to the cumrate model parameters is gained through the use of the special function Lookup_. The syntax of Lookup_ is described in the topic “Run-time Functions and Macros” on page 103.

Example

```
// the states, Year and Prov, are being defined conditional on
// the comparison of the random number, RandUniform(), with values in the
// CumBirth parameter. The lookup is conditional on the sex of the person.
Lookup_CumBirth( RandUniform(), eSex, &nYear, &nProv );
```

piece_linear type parameters

To obtain the value of a piece_linear function for a given argument *dArg* calls the special function Lookup_.

Example

```
range WAGE_POINT_VALUES    { 0, 7 };
```

```
...
parameters {
...
    piece_linear   WagePoints[WAGE_POINT_VALUES] =
        { 0, 40, 0.3, 85, 0.7, 115, 1, 205 };
...
};
dLowerWage = Lookup_WagePoints( .33 );
dUpperWage = Lookup_WagePoints( .66 );
```

In the above example, the X values contain the cumulative probabilities for the corresponding wage interval and the Y values contain the wage breakpoints. Inside each interval wages are uniformly distributed. For example the probability of the wage being in [40, 85] interval is 0.3, for the [85, 115] it's 0.4 (0.7 - 0.3) and for the [115, 205] it's 0.3. The tertile wage breakpoints for the WagePoints parameter can be obtained with the following calls to "Lookup_WagePoints" function:

Note that the preferred and more flexible method for linear interpolation is to use the second form of the PieceLinearLookup function. With this method, you do not define a parameter as type piece_linear, you simply use parameters containing the X and Y points with a function call to PieceLinearLookup.

file type parameters

Modgen represents a parameter of type *file* as a C++ string (more specifically as a CString). A parameter of type *file* can be used directly as an argument to a function that takes a string as an argument, for example the C++ function fopen() in the example below. The string value can be accessed directly by name, as in the following example:

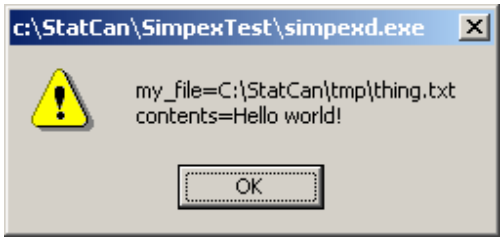
```
void PreSimulation()
{
       CString szWork;
       CString szFileContents;

       FILE *fp = fopen( my_file, "r" );
```

```
        if ( fp != NULL ) {
                char szBuffer[100];
                fgets(szBuffer, 100, fp);
                szWork.Format(" my_file=%s\n contents=%s", my_file, szBuffer);
        }
        else {
                szWork.Format(" my_file=%s\n Unable to open file", my_file);
        }


        AfxMessageBox(szWork);
}
```

When the simulation is run, the above example would display a message box such as the following:



Actor Links: How Model Objects Access each other's States

There are many instances in the course of the model simulation when one actor must access a state of another actor. For example, consider the simple example in which the dwelling actor must access the person actor's earned income in order to calculate the dwelling balance. Typically, the application developer would associate pointers between the data structures (or classes) to perform this task. Modgen provides links which are, essentially, pointers between model actors. These links must be defined symmetrically between the model actors so that Modgen can continually update the dependencies between states from different actors.

Modgen also allows for linked variables to define a stratification of a table report. Therefore, the table makes use of the link facility.

Since the links types are derivations on the C++ pointer, the declaration, definition, initialization, and syntax follow the much of the C++ convention for these members.

Link definition

The links in a model may connect one actor to another, one actor to many other actors, or many actors to many other actors. In all instances the reciprocal links are automatically generated by Modgen. This means that only one side of the link need be assigned by the application developer. The link name must be unique within the actor. The same link name cannot be used for two different connections. The link definition normally appears in the part of the .mpp file which defines the model parameters. The syntax is as follows:

Syntax for a one-to-one link

link actor_name.link_name ;

link actor_name1.link_name1 // link label 1

actor_name2.link_name2 // link label 2

;

The first definition specifies a link between two actors of the same type (actor_name). In the second definition actor_name1 must be different from actor_name2. In the second instance, labels can be associated with the links. Generally, the first letter following the '.' should be an 'l' indicating that this is a single link.

Examples

```
// Note these links do not appear in the simple example
// in the first example, a single link between the dominant person
// and his/her spouse is established.
// in the second example, a single link is established between
// a second marriage spouse and ancillary spouse
link Person.lSpouse;   // Spouse
link Person.lSecSpouse   // Second-Marriage Spouse
        Ancillary.lSpouse;   // Spouse
```

Syntax for a one-to-many link

link actor1_name.link_name1 actor_name2.link_name2[];

The definition specifies a link between two actors where the first actor is of a single type and the second actor is of a multiple type. Generally, the first letter following the '.' in the first actor should be an 'l' indicating that this is a single link and a 'ml' in the second actor indicating the multiple type.

Examples

```
// Note these links do not appear in the simple example
// a link is established between the spouse and the children
link Person.lSpouse   Person.mlChildren[ ];
```

Syntax for a many-to-many link

link actor1_name.link_name1[ ] actor_name2.link_name2[];

The definition specifies a link between two actors where both actors are of multiple type.

Examples

```
// Note these links do not appear in the simple example
// a link is established between the children and the parents
link Person.mlChildren[ ]   Child.mlParents[ ];
```

Link declaration

The links are declared as pointers in the C++ manner. The declaration remains the same regardless of whether the link is a single or multiple type.

Syntax

Actor_name *prActor_name;

Example

```
// Note these links do not appear in the simple example
// a link is established between the children and the parents
```

```
Child *prChild;

Person *prDominant;

Person *prNonDominant;
```

Link initialization

The link initialization follows the C++ convention pointer initialization for the single links. However, for the initialization of multiple links, Modgen relies on a special function. Remember that the both the actor and the pointer must be initialized before the link can be initialized

Syntax for a single link initialization

link_name = prPointer_name;

Example

```
// Note these links do not appear in the simple example

// a link is initialized for the spouse

lSpouse = prNonDominantPerson;
```

Syntax for a multiple link initialization

link_name->Add(prPointer_name);

Example

```
// Note these links do not appear in the simple example

// a link is initialized for the Children

mlChildren->Add(prChild);
```

Special functions acting on multiple links

These special functions allow the application developer to generate statistics over a set of multiple linked objects. For multiple links (with "[ ]") the syntax is one of the following.

Syntax

multilink_function (link_name)

multilink_state_function (link_name, state_name)

where multilink_function is one of

count	- number of all linked actors

and multilink_state_function is one of

sum_over	- sum of the requested state over all linked actors
min_over	- minimum of the requested state over all linked actors
max_over	- maximum of the requested state over all linked actors

Examples

```
actor Household {

    children = count ( parChild );

    age_youngest_child = min_over ( parChild, age );

};
```

Functions for adding, removing, and indexing through the link of actors

For each requested multiple link Modgen defines a special class which includes member functions for adding actors, removing actors and iterating through the link. The following functions are defined:

Add(actor_pointer)

This function adds an actor of the right type to the link.

FinishAll(no parameters, no result)

This function calls the Finish function for all connected objects.

*__GetNext__(int nInitPos, int *pnPos)

The *GetNext function returns a pointer to the next object available through the multiple link starting the search at position specified by "nInitPos". The position of the found element is also returned. Note that the object connected through multiple link may not occupy consecutive positions.

Remove(int nPosition)

This function removes the actor at specified position from the link.

RemoveAll()

This function removes all actors from the link.

Sample prototype

```
Child *GetNext( int nInitPos, int *pnPos );
```

Sample model code using "GetNext"

```
// connect the dominant person to the partner and partner's children
lSpouse = prNonDominant;
if ( bFemale ) {
    prChild = prNonDominant->mlChildren->GetNext( 0, &nIndex );
    while ( prChild != NULL ) {
        mlChildren->Add( prChild );
        children++;
        prChild = prNonDominant->mlChildren->GetNext( nIndex + 1, &nIndex );
    }
}
```

Example of iterating through the multiple link:

```
int nIndex;
nIndex = prFamily->parChild->GetFirst();
while ( nIndex != -1 ) {
    prChild = prFamily->parChild->GetNext( &nIndex );
    if ( prChild->active ) {
        min_time = prChild->NextEvent( min_time );
    }
}
```

Example of adding an actor to the multiple link:

```
Child *prChild;
prChild = prFamily->arAllocChildren[nChild];
```

```
prChild->Start( eChildSex, child_birth, sep_age, child_immig );
prFamily->parChild->Add( prChild );
```

Example of removing actors from the multiple link:

```
prFamily->parChild->RemoveAll();
```

Multilink class doesn't allocate or deallocate pointers to specific actors. It operates on existing pointers only (adding, removing and referencing them). If a model specified in Modgen wants to pre-allocate pointers for specific number of actors, it may do so by defining an array of actor pointers inside the actor definition and then using that array. Modgen pre-defines array types (e.g. ChildArray, PersonArray etc.) for each actor.

Example:

```
actor Household {
...
    ChildArray arAllocChildren;   // array of allocated children
...
}

Household::Household()
{
    int  nIndex;
...
    arAllocChildren.SetSize( SIZE( CHILD_PAR ) );
    for ( nIndex = 0; nIndex < SIZE( CHILD_PAR ); nIndex++ ) {
        arAllocChildren[nIndex] = new Child( this );
    }
...
}
```

```
TIME Person::ChildBirthEvent( CALL_TYPE eCallType )
{
...
    nChild = prFamily->children;
    prChild = prFamily->arAllocChildren[nChild];
    prChild->Start( eChildSex, child_birth, sep_age, child_immig );
    prFamily->parChild->Add( prChild );
...
}
```

Actor sets

An actor set is a collection of actors dynamically maintained by Modgen. "Dynamically maintained" means that Modgen internally handles group membership based on criteria provided by the model developer. Modgen provides functions that the model developer uses to select an actor randomly from the set, to iterate over all members of the set, etc. An example of an actor set might be the set of actors currently susceptible to infection by a particular disease. As the simulation progresses, actors would enter the set at birth or when they lose maternal immunity. They would leave the set through death or by acquiring immunity as a result of infection.

Actor sets are useful in time-based models with many interacting actors. Actor sets are intended to facilitate the development of such models by eliminating the need for the developer to write code to maintain and manipulate such groups. Modgen dynamically maintains groups using efficient internal algorithms and provides a core set of functions to efficiently perform operations on such groups.

Following is a list of attributes of an actor set which should be specified in its definition in developer model code. See the examples immediately below for the syntax.

actor type

Required. An actor set will contain only actors of a given type, e.g. "Person" actors or "Dwelling" actors, but not both types in a single set.

actor set name

Required. A unique name that identifies the actor set and allows operations to be performed on it through function member calls.

dimensions

Optional. Specifies one or more **states** having fixed number of values (of type classification, range, or partition) that subdivide the actor set into distinct subsets, e.g. by city.

filter

Optional. A logical expression that defines whether a given actor is a member of the group, e.g. "marital_status == SINGLE". Similar to a Modgen table filter, although the syntax is slightly different.

order *state* or **order RandomStream(*nStream)***

Optional. Specifies a state whose value is used to order the members of the actor sets. Modgen automatically maintains this order as the simulation progresses. When **RandomStream** is specified instead of a state, a state *dRandom_XXX* will be generated for each actor. Modgen will assign a value to that state using **RandUniform** when an actor enters an actor set, or a cell of a multi-dimensional actor set. If the value of *nStream* is missing, Modgen will generate a unique integer value (as for the existing functions RandLogistic(), **RandUniform()** and **RandNormal()**). The state *dRandom_XXX* will be used to order the members of the actor sets.

Following are some examples that illustrate the syntax used to define an actor set.

```
actor Person {
      SEX sex;          // Sex
      CITY city;        // City of residence
};

actor_set Person asEveryone; // All Persons

actor_set Person asEveryoneByCity[city]; // All Persons, in subsets by city

actor_set Person asSusceptible // Persons susceptible to infection
filter disease_status != IMMUNE;
```

```
order RandomStream();

actor_set Person asEligible[city][sex] // Persons eligible for marriage
filter marital_status != MARRIED && curtate_age >= 16;

actor_set Person asEligibleOrdered[city][sex] // Persons eligible for marriage,
ordered
filter marital_status != MARRIED && curtate_age >= 16
order income;
```

Following are the functions that can be called on an actor set at run time. These are implemented as member functions of the actor set. If the actor set is split into subsets, a given function call applies to a specified sub-group.

int Count( )

The current number of actors in the actor set.

actor * Item(*index*)

Retrieves the actor located at position ***index*** in the group. Valid values for ***index*** range from **0** to **Count( ) - 1**. If a floating point value is supplied for ***index***, the fractional part is discarded. If an invalid value for ***index*** is specified, **Item( )** returns the value NULL. The value of ***index*** associated with a given actor may change as actors enter and leave the actor set. Actors are retrieved in the order specified in the **order** clause. If no **order** clause is specified, actors are returned in an unspecified order. To retrieve actors in a random order, call the **Scramble()** function before iterating over the actor set using **Item()**.

actor * GetRandom(double *dRandom*)

Returns a random actor from the actor set. The argument ***dRandom*** is a uniform random number, as returned by the Modgen function **RandUniform()**. If the actor set contains no members, **NULL** is returned. This function is exactly equivalent to the expression

```
as->Item((int)(dRandom * as->Count())
```

Scramble( )

Results in the actors in the group being returned in a new random order when **Item()** is called. Scramble is available

only for actor sets whose definition includes an **order RandomStream()** clause. When called it assigns a new random order to each actor in the actor set (or in the specified cell of the actor set for multi-dimensional actor sets).

Following are code fragments illustrating the actor set member functions.

```
// Select a random susceptible
Person *prSelected = asSusceptible->GetRandom(RandUniform());

// Number of susceptibles in the population
int nSusceptibles = asSusceptible->Count();

// Iterate over all susceptibles
 for (int nJ=0; nJ<nSusceptibles; nJ++) {
  Person *prSusceptible = asSusceptibles->Item(nJ);
  // ...
  // Do stuff with prSusceptible
  // ...
}

// Select 10 different random susceptible
// this code works only on randomly ordered actor sets
  asSusceptibles->Scramble();
 for (int nJ=0; nJ<10; nJ++) {
  Person *prSusceptible = asSusceptibles->Item(nJ);
  // ...
  // Do stuff with prSusceptible
  // ...
}
// Number of males eligible for marriage in Ottawa
int nEligible = asEligible[OTTAWA][MALE]->Count();
```

Notes:

If an actor set is used in case-based models, it is initialized to empty at the beginning of each case.

Continuously updated states are not allowed as filters, order by states or dimensions of an actor set. This also includes states derived from continuously updated states or states having the same member name as a continuously updated state (of another actor).

If code is iterating through an actor set using Item() and the user changes a state affecting the actor set (e.g. a state referenced in the actor set filter or the order clause) inside the loop, the loop may miss some actors, or process the same actor twice. That's because actor sets are modified instantaneously when associated states change. If the developer wishes to change such states inside a loop, an auxiliary actor set can be created to "hold" the actors to be changed. Here's a sketch showing how…

```
actor Person {
      logical in_holding_area = { FALSE };
};

actor_set Person asHoldingArea
filter in_holding_area;

// Move all susceptibles to holding area
int nSusceptibles = asSusceptible->Count();
for (int nJ=0; nJ<nSusceptibles; nJ++) {
  Person *prSusceptible = asSusceptibles->Item(nJ);
  prSusceptible->in_holding_area = TRUE;
}

// Iterate over the holding area,
// and empty it at the same time.
Person *prTemp = asHoldingArea->Item(0);
while ( NULL != prTemp) {
      // Do stuff to Person
      // ...
      // Remove Person from the holding area
      prTemp->in_holding_area = FALSE;
      prTemp = asHoldingArea->Item(0);
```

```
}
```

Here are some additional examples of actor set

```
=======================================

// Find richest eligible guy using ordered array
int   nCount = 0;
Person      *prRichGuy = NULL;

// get number of eligible males in city
nCount = asEligibleOrdered[city][MALE]->Count();

// find the richest
prRichGuy = asEligibleOrdered[city][MALE]->Item(nCount - 1);



// Find richest eligible guy using not-ordered array

int         nCount = 0;
int         nIndex = 0;
Person      *prRichGuy = NULL;
Person      *prTemp = NULL;

// get number of eligible males in city
nCount = asEligibleOrdered[city][MALE]->Count();

// loop through to find richest
for (nIndex = 0; nIndex < nCount; nIndex++)
{
      prTemp = asEligibleOrdered[city][MALE]->Item(nIndex);
      if (prRichGuy == NULL || prTemp->income > prRichGuy->income) prRichGuy =
prTemp;
}

=======================================

// Infect the whole susceptible population
```

```
Person *prTemp = NULL;

prTemp = asSusceptible->Item(0);
while (prTemp != NULL)
{
      // change disease status.  That will also remove the person from the actor
set.
      prTemp->disease_status = INFECTED;
      prTemp = asSusceptible->Item(0);
}

=======================================
// List the 100 companies with the biggest profits

actor_set Company asEveryone; // All Companies
order profits;

...
int         nCount = 0;
int         nIndex = 0;
Company     *prTemp = 0;

// get the number of company
nCount = asEveryone->Count();

//get the 100 companies with the biggest profits
for (nIndex = 1; nIndex <= 100; nIndex++)
{
      prTemp = asEveryone->Item(nCount - nIndex);
      // ...
      // do stuff with prTemp
      // (except changing profits!)
      // ...
}
```

```
// NOTE : Anytime you want the x first or x last, you'd need an ordered actor
set.  Same thing for median:

================================================

// Find the province with the biggest median income for individuals

classification PROVINCE {        //EN Province
      //EN Newfoundland
      NFLD,
      //EN P. E. I.
      PEI,
      //EN Nova Scotia
      NS,
      //EN New Brunswick
      NB,
      //EN Quebec
      QUE,
      //EN Ontario
      ONT,
      //EN Manitoba
      MAN,
      //EN Saskatchewan
      SASK,
      //EN Alberta
      ALTA,
      //EN British Columbia
      BC,
      //EN Yukon
      YUK,
      //EN N. W. T.
      NWT
};
```

```
actor Person {
      PROVINCE    prov_of_res;
};

actor_set Person asEveryoneByProvince[prov_of_res]; // All Persons, in subsets
by province
order income;

...

int         nCount = 0;
int         nIndex = 0;
int         nMedianIndex= 0 ;
double      dPreviousMedian = -1;
Person            *prTemp  = NULL;
PROVINCE    eProvince = NFLD;
PROVINCE    eLastProvince;

for (nIndex = 0; nIndex < SIZE(PROVINCE); nIndex++ )
{
      eProvince = (PROVINCE) nIndex;

      nCount = asEveryoneByProvince[eProvince]->Count();

      nMedianIndex = nCount / 2;

      prTemp = asEveryoneByProvince[eProvince]->Item(nMedianIndex);
      if (prTemp->income > dPreviousMedian)
      {
            eLastProvince = eProvince;
            dPreviousMedian = prTemp->income;
      }

      // ...
      // do stuff with eProvince
      // ...
```

```
}
```

The Simulation Engine

Modgen looks for the following three C++ functions for controlling the simulation:

PreSimulation – this function is optional.
Simulation – this function is required.
PostSimulaton - this function is optional.

The UserTables function, if present, is also run after the Simulation function.

Modgen allows multiple definitions of the special global functions PreSimulation, PostSimulation and UserTables. Different versions may be defined in different mpp files if different modules need to work on a different set of model-generated parameters and/or user tables. Modgen will mark up the code appropriately and call different definitions of the special functions in the order in which they were parsed (the order specified on the command line or name sort order in case of a wildcard group like *.mpp). Care should be taken if different versions of the special functions modify the same model-generated parameter and/or user table.

PreSimulation function

The PreSimulation function is optional and is run before the Simulation function. It is most often used to compute model generated parameters and initialize global variables (e.g. SetMaxTime). The prototype is:

```
void PreSimulation();
```

Simulation function

The main function in a Modgen model is the Simulation() function. It is here, for example, that the number of cases is passed to the simulation engine. This function must also call the TimeReport() function to specify the time at the beginning of the simulation (for time-based models), before the loop that actually does the simulation.

The Simulation() function calls the StartCase(), the CaseSimulation(), and SignalCase() routines respectively within the simulation loop. The StartCase() and SignalCase() functions are internal to Modgen and are not found in the Modgen model code.

Example

```
// ====================================================================
// Main function of the simple example model.
// Contains code required to run each thread of execution independently.

void Simulation()
{
    long        lCase;

    // case loop
    for (lCase=0;lCase < CASES() && !gbInterrupted && !gbCancelled &&
!gbErrors;lCase++) {
        // simulate the next case
        StartCase();
        CaseSimulation();
        SignalCase();
    }
}
```

The CaseSimulation() function controls the simulation of a case. It is a standard C++ function and has no special meaning to Modgen. This function creates the next actor by calling the New and Start() functions, process the sequence of events through the event loop. It concludes by freeing all current actors in the case. Users wishing to learn more about the event loop are free to contact Statistics Canada at the contact source provided in this document.

Example

```
// ====================================================================
// Simulation Control Functions for the simpex model.
// Users should not modify code beyond this point.

// Simulation of one case, which is a dwelling in this model.
```

```
// An actor for the new case is created together with related person actor,
// an event loop is then run until the death of the person in the Dwelling
// the actors are destroyed in their respective event functions.

void CaseSimulation()
{
    //  initialize the pointer to the dwelling
    Dwelling    *prDwelling;

    // initialize the next case ( dwelling )
    prDwelling = new Dwelling();
    prDwelling->Start();

    // event loop for current case
    while ( !gpoEventQueue->Empty() ) {

        if ( gbCancelled || gbErrors || !prDwelling->lPerson->life_status ==
ALIVE ) {
            // close the case, destroy all remaining actors
            gpoEventQueue->FinishAllActors();
        }
        else {
            // age all actors to the time of next event
            gpoEventQueue->WaitUntil( gpoEventQueue->NextEvent() );

            // implement the next event
            gpoEventQueue->Implement();
```

```
        }

    }

    // release the memory occupied by the actors used in this case
    DeleteAllDwellingActors();
    DeleteAllPersonActors();
}
```

PostSimulation function

A PostSimulation function is also available to model developers. It is called after the simulation is finished but before the tables are generated. It may be used to perform some clean up of global variables and any other activities required after the simulation ends. The use of PostSimulation is optional, Modgen will supply the default version if none was supplied by the model. The prototype is:

```
void PostSimulation();
```

Actor construction and initialization

Successful model specifications require actors to be managed in an intelligent way. Each actor in a model must have an appropriate amount of memory allocated to it in which the states, model links, and other critical information are to be stored. The actors in a Modgen model are C++ objects and are created and destroyed in the following manner.

Actors must be constructed and initialized at the onset of simulation. This involves allocating the appropriate amount of memory to the actor and the appropriate variables initialized. These two tasks are handled by the C++ operator new and the special Modgen function Start() respectively. Note that the time for a new actor is initialized to the current time of the simulation, unless specified otherwise in the Start() function.

The Start() function has special meaning in that Modgen always initializes it as a member function (often called a method) of an actor object. The model developer has access to its prototype within the actor definition and may choose to include arguments there. This is similar to the Finish() function used for actor termination.

Note that an actor should never be referenced by model code after Finish has been called. If model code calls Start on an actor after Finish has been called, Modgen will issue an error message and attempt to stop the simulation.

Example

```
    // initialize the next case ( dwelling )
    prDwelling = new Dwelling();
    prDwelling->Start();
```

where the Start() function is as follows

```
// Initialize a new dwelling
void Dwelling::Start()
{
    Person *prPerson;

    // initialize the next person
    prPerson = new Person();
    prPerson->Start();
    lPerson = prPerson;
}
```

Actor termination and destruction

The termination and destruction of all actors are handled by the special Modgen functions FinishAllActors() and DeleteAllActor_nameActors() respectively. The former function can be specified by the user while the latter should never be modified by the application developer.

Note also that when the just_in_time algorithm is activated in the model, the time for all actors has to be brought to the current time before all actors are finished; this is accomplished automatically via the FinishAllActors function.

Controlling Precision in Case-based Continuous Time Models

Some anomolies can occur within continuous time models due to the precision of time and rounding errors. Modgen gives the modeler two ways to control the precision of floating point calculations.

Using SetMaxTime to control precision

The modeler may estimate the maximal value for the simulated time (the "time" state) and then Modgen will take care of rounding all event times and the "age" state to the appropriate precision. This step is required for both case-based models and time-based models. This will eliminate certain observed anomalies e.g. the value of "age" at person's 61st birthday will be exactly 61 and not something like 60.999999999997.

The ideal place for estimating the maximal time value is the PreSimulation function, which is called only once before the simulation starts. The Modgen-supplied function "SetMaxTime" must be used. Here is the prototype:

```
void SetMaxTime( double dMaxValue );
```

For case-based models, the following line could be added to the PreSimulation function:

```
SetMaxTime( MAX( YEAR ) + MaxLife );
```

Using CoarsenMantissa to control precision

There are still other cases where the precision control of floating point variables may be desired to ensure that arithmetic operations on time and age are performed consistently. To handle those situations another Modgen-supplied function called "CoarsenMantissa" may be used. Here is the prototype:

```
double CoarsenMantissa( double dValue, double dMaxValue );
```

The function returns the rounded value based on the maximal absolute value for the variable. The second argument is optional; if it is not supplied, the value set by the SetMaxTime function is used.

CoarsenMantissa uses the value specified in a previous call to SetMaxTime to deliberately limit the precision of all waiting times, as well as the starting values of time and age. In effect, internal operations related to time and age are performed at fixed precision rather than floating precision, even if time and age are floating-point quantities.

Associated with this fixed precision is a minimum indivisible atomic unit of time. The value of this minimum indivisible atomic unit of time can be retrieved by calling the function GetMinimumTimeIncrement. The value depends on the argument supplied to a previous call to SetMaxTime, and also depends on whether time_type is float or double.

CoarsenMantissa and GetMinimumTimeIncrement return a double in models where time_type is declared as double, whereas they each return a float when time_type is declared as float in the model.

Here is an example of CoarsenMantissa from the LifePaths model:

```
birth = CoarsenMantissa( birth );
```

Independent Random Number Streams in Modgen Models

Modgen uses independent random number streams in order to eliminate spurious 'noise' and reduce the variance of inter-run differences.

When Modgen scans the mpp files it looks for occurrences of RandUniform() and RandNormal() function calls without arguments and assigns a unique index as an argument to those functions. The changes are written back to the mpp files, before the corresponding cpp files are generated. The arguments added by Modgen to RandUniform() and RandNormal() calls are used internally to ensure the independence of all random number streams used by the model.

For example, the following piece of code:

```
sex = ( RandUniform() <= FemaleProp ) ? FEMALE : MALE;
```

gets changed to:

```
sex = ( RandUniform(3) <= FemaleProp ) ? FEMALE : MALE;
```

When the modeler writes new code (e.g. adds a new equation or a new event) all calls to RandUniform() and/or RandNormal() should have no arguments, Modgen will assign them at parse time. Modgen does not modify RandUniform() and RandNormal() calls that already have a numeric argument unless duplicates are detected (e.g. as a result of the user copying a block of code containing indexed RandUniform calls). In the case of duplicates, the function arguments will be changed to unique numbers.

The random number generator state needs to remember the seeds and deviates of all random number streams used in a model. To simplify the use of random number generator state a Modgen-defined class called CRandState should be used. Variables of type CRandState can be declared in any function of the model and passed as arguments to GetRandState and SetRandState functions. The class also supports the assignment of one CRandState to another one.

In the unlikely event that changes are required to seeds of selected random streams only, plSeeds and pdDeviates members can be manipulated directly (e.g. plSeeds[5] = lNewSeed;).

Here is the declaration of the CRandState class:

```
class CRandState {
    public:
        CRandState();
        ~CRandState();
        void operator=( const CRandState &Other );
```

```
        long    *plSeeds;
        double  *pdDeviates;
};
```

Here are the prototypes of GetRandState and SetRandState functions:

```
void GetRandState( CRandState &rRandState );
void SetRandState( CRandState &rRandState );
void SetRandState( long lSeed, bool bResetDeviates = true );
```

The first 2 versions of the functions get and set the complete set of seeds and deviates. The last version is used to modify the random number generator state based on a single seed argument, which may be useful for modifying clones and/or at case initialization. The second argument is optional in that version, by default all random deviates will be reset i.e. no leftover deviates will be used.

The following code fragment illustrates the use of the GetRandState and SetRandState functions:

```
void CaseSimulation()
{
    CRandState    rRandState;
    int           CloneNumber = 0;
    int           nClonesRequested = 0;

    while ( nCloneNumber <= nClonesRequested ) {
        if ( nCloneNumber == 0 ) {
            GetRandState( rRandState );
        }
        else {
            SetRandState( rRandState );
        }
…
    }
…
}
```

Distributed execution

Either the cases from case-based models or the replicates from time-based models can be distributed across machines on the network. In any given scenario of a distributed execution run, Modgen models automatically use the pool from the instance of Executor running on the machine.

Distributed runs, slave and master jobs

The roles of master and slave jobs in a distributed run are as follows. When a distributed job is started, the master job will create the slave jobs. Once those jobs are created, the master job becomes a regular "slave" job. Like all other slave jobs, it will run then exit. Before exiting, a slave job checks in to see if it is the last job to be completed. If all the other slaves have been successfully completed, it becomes the new master. The new master reassembles the data and removes the slave jobs from the request database.

As a result, a slave can be resubmitted independently. Should all the slaves have been successful but one, the user can resubmit that particular slave. If it is successful, it will become the new master and reassemble the data.

Expected and maximum elapsed time in distributed runs

The maximum and expected elapsed time of a distributed run is considered to be the maximum and expected time for one slave to complete. Example: if the user sets the maximum elapsed time to 20 hours, then each slave can take up to 20 hours to run.

Log files of slaves merged

At the end of a distributed simulation, the log files of slaves will be merged into one log file with the name <scenario_name>(log).txt. The log files for individual slaves will be deleted.

Assemble Slave Runs

Under normal circumstances, the slave run results of a distributed run are assembled automatically by Modgen. In case

of error, the results of slave runs can be assembled manually using a command line option. This only works if all slave output files (.tbl) are present. The syntax is as follows:

```
model_name -sc scenario_file -SLAVE ASSEMBLE
```

By design, this option does not access the Executor database, so any distributed job entries remain unchanged. All of the actions normally performed by the final slave (apart from the simulation itself) are performed. When this option is used, the log file is named <scenario>_assemble(log).txt.

Cloning Observations: A Special Technique

The purpose of cloning is to obtain a bigger count of an interesting population and then apply different algorithms (e.g. interventions for lung cancer patients in POHEM) on different clones of the same object. Cloning is not part of Modgen but it can be easily implemented in Modgen-generated models.

Cloning can also be used for retrospective tabulations (e.g. tabulating loan defaulters by the year of their first entry into post-secondary education) which may not always be achieved by the use of filters or tabulation at death. In such cases, typically 2 clones of the same object are generated and the tabulation is performed only on the second clone (using some information obtained during the life of the first clone).

Here is the code required to implement cloning in the LifePaths and Pohem models. The following two states are added to the Person actor:

```
int CloneNumber;        // clone number
int ClonesRequested;    // requested number of clones
```

CloneNumber is assigned only once (at the beginning of the Person actor's life, shortly after the Start function is called). ClonesRequested state is initialized to 0 and incremented during the simulation if some event of interest occurs.

The following part of CaseSimulation takes care of cloning:

```
void CaseSimulation()
{
…
    double   dDeviate;
    int      nCloneNumber = 0;
    int      nClonesRequested = 0;
```

```
    long     lSeed;
    Person   *prDominant;
…
    while ( nCloneNumber <= nClonesRequested ) {
    if ( nCloneNumber == 0 ) {
        GetRandState( &lSeed, &dDeviate );
    }
    else {
        SetRandState( lSeed, dDeviate );
    }
    …
    prDominant = new Person();
    prDominant->Start( … );
    prDominant->CloneNumber = nCloneNumber;
    prDominant->ClonesRequested = nClonesRequested;
    …
    // event loop for current case
    …

    nCloneNumber = prDominant->CloneNumber + 1;
    nClonesRequested = prDominant->ClonesRequested;
   }
…
}
```

At the beginning of each simulated case CloneNumber is set to 0 and ClonesRequested is set to 0. If model code sets ClonesRequested, for the dominant person, to a non-zero value then the current case is re-simulated using the same starting seed. This re-simulation increments the CloneNumber. Model code and tables can be made sensitive to

CloneNumber to generate different case-specific trials and report on their effects.

Controlling Pre-compiled Models from Outside Applications

Pre-compiled Modgen models can be controlled, and their inputs and outputs manipulated, by outside applications. This controllability can be used, for example, to implement tracking techniques to perform model calibration. In this case, a controlling application runs the model in a loop, searching the model's parameter space in order to match historical control totals.

Such controllability also allows a model developer to program a simplified visual interface to a model making it more accessible to a wider audience. Although the Modgen interactive option provides a generic visual interface into a model, a simpler visual interface can provide customized multi-parameter input screens and perform further manipulation on model outputs.

Modgen as an In-process Automation Server

Since one of the most difficult tasks for such an application would be the reading (parsing) and writing of the .dat files, this functionality has been provided to application developers as an in-process automation server within the ModAuto10.dll library. This library must be registered before the server can be used. The Modgen setup program will perform the necessary registration. To re-register later use Microsoft's utility called regsvr32.exe. Only one Modgen server can be registered at any time. If you have multiple versions of ModAuto10.dll ensure that the right version is registered last.

The programmatic identifier of the server is "Modgen10.AutoParser". Applications should use this identifier to create the server object. For more information, refer to the OLE documentation within your development environment.

Use this identifier for late binding of the server e.g.

```
Set objParser = CreateObject("Modgen10.AutoParser")  'late binding
```

For early binding, use autolib2.AutoParser e.g.

```
Public gObjParser As autolib2.AutoParser  'early binding
```

The type library file ModAuto10.tlb is also distributed with the Modgen setup. Applications may use this file with object browsers to access the information about the exposed methods.

Methods exposed by the parsing server and their syntax

All methods return a result code (0 means success). Error codes and their descriptions are found in the documentation

database generated by the model. For more information on the documentation database, see “Modgen Documentation Database”.

[id(1)] short **Init**(BSTR szDocDb);
Initializes the automation server for a specific model using information stored in the documentation database file "szDocDb".

[id(2)] short **ResetData**();
Resets all parameters. This method should be used if the data files need to be re-parsed.

[id(3)] short **ReadFiles**(BSTR szFileSpec);
Parses file(s) indicated by "szFileSpec" (wildcards allowed).

[id(4)] short **SaveFiles**(BSTR szFileSpec);
Saves the contents of files indicated by "szFileSpec".

[id(5)] short **GetParameterCell**(BSTR szName, long lCell, VARIANT *pVariant);
Obtains the data value (as a string) for cell “lCell” (ravelled index in display order) for parameter indicated by "szName". The value is returned as a string in “pVariant”.

[id(6)] short **SetParameterCell**(BSTR szName, long lCell, BSTR szData);
Sets the data value (as a string) for cell “lCell” (raveled index in display order) for parameter indicated by "szName". “szData” contains the new value.

[id(7)] short **GetStoredParameterSlices**(BSTR szName, long *plSlices);
Get the number of first dimension slices stored for an extendible parameter indicated by “szName”.

[id(8)] short **SetStoredParameterSlices**(BSTR szName, long lSlices);
Update the number of first dimension slices stored for an extendible parameter indicated by “szName”.

[id(9)] short **ParameterUpdateRequired**(BSTR szName, short *pnUpdate);
Checks if an extendible parameter needs an update. The result is returned in the variable pointed to by “pnUpdate” (1 - if update is required, 0 - otherwise).

[id(10)] short **GetFileNote**(BSTR szFile, short nLang, VARIANT *pVariant);
Get the note associated with the file indicated by “szFile” in language “nLang” (0-based language index).

[id(11)] short **SetFileNote**(BSTR szFile, short nLang, BSTR szNote);
Set the note associated with the file indicated by “szFile” in language “nLang” (0-based language index).

[id(12)] short **GetParameterNote**(BSTR szName, short nLang, VARIANT *pVariant);
Get the note associated with the parameter indicated by “szName” in language “nLang” (0-based language index).

[id(13)] short **SetParameterNote**(BSTR szName, short nLang, BSTR szNote);

Set the note associated with the parameter indicated by “szName” in language “nLang” (0-based language index).

[id(14)] short **ChangeFileReference**(BSTR szOldFile, BSTR szNewFile);
This method should be used if a data file is parsed and then it needs to be saved under a different name. The server can only save files that were encountered by the parser. Each object (parameter, table exclusion, file note etc.) has a file reference attached to it, so the server knows which objects to save when saving a data file. If "SaveFiles" method was used with a new file name, a blank file would be created."ChangeFileReference" method solves this problem. It should be called just before "SaveFiles" method. It creates an empty file as a side-effect, "SaveFiles" then adds contents to it. "szOldFile" is the original file parsed by the server, "szNewFile" is the new file name.

[id(15)] short **GetParameterDecimals**(BSTR szName, short *pnDecimals);
Gets the number of decimals stored for the parameter indicated by “szName” (negative value in “pnDecimals” or “nDecimals” means no limit).

[id(16)] short **SetParameterDecimals**(BSTR szName, short nDecimals);
This methods obtain and update the number of decimals stored for the parameter indicated by “szName” (negative value in “pnDecimals” or “nDecimals” means no limit).

Sample VB code showing use of the Modgen Server for parsing .dat files

```
Private Sub cmdParser_Click()
   Dim objParser As Object, intRet As Integer, pVariant As Variant, i As Long
   Dim strArray() As String
   'start the OLE server (requires Modgen setup to register the server).
   '  Also add reference using Tools/References in VB pointing to auto.tlb)
   Set objParser = CreateObject("Modgen.AutoParser2")  'late binding
   'initialize the server for your model
   ‘ (must first create the doc DB
   intRet = objParser.Init("e:\OLETest\lpaths_doc.mdb")
   'parse one or more .dat type files
   'check return code intRet if you like....
   intRet = objParser.ReadFiles("e:\OLETest\loans.dat")
```

```
   ReDim strArray(17984) 'read in a very big parm (one cell at a time, in
display order)
   For i = 0 To 17984 'you can get the number of datapoints for any parm from
the doc db
      intRet = objParser.GetParameterCell("UniTuitionFees", i, pVariant)
      strArray(i) = CStr(pVariant) 'or convert to any numeric representation as
appropiate...
   Next i
   'change the value of a parameter (this one has only 1 datapoint)
   intRet = objParser.SetParameterCell("LoanDisbOption", 0, "LDO_NA")
   'save the .dat type file elsewhere
   intRet = objParser.ChangeFileReference("e:\OLETest\loans.dat",
"e:\OLETest\MyScenarios\scenario1.dat")
   intRet = objParser.SaveFiles("e:\OLETest\MyScenarios\scenario1.slf")
   Set objParser = Nothing 'shut down the server
End Sub
```

Running the Model Executable and Waiting for its Completion

Once you have made the parameter changes within your container application and saved them to file, launch the pre-compiled model from your application and wait for the task to complete. The sample VB function funExecCmd shown below was largely taken from the MS TechNet Article, entitled

HOWTO: 32-Bit App Can Determine When a Shelled Process Ends, PSS ID Number: Q129796.

This article will give you step-by-step instructions on how to invoke an executable program asynchronously, wait for its completion, and return to the controlling application. Refer to that article for the *type* and *API* declarations required for this function to work. Make note however of the additional *API* call to **GetExitCodeProcess**. Modgen will return a exit code which can be used to determine how the simulation finished. In MS VB 6.0, declare this *API* as follows:

```
Private Declare Function GetExitCodeProcess Lib "kernel32" _
      (ByVal hProcess As Long, lpExitCode As Long) As Long
```

The following codes are returned by Modgen:

0 - application ended with Task Manager

1 - successful completion

2 - errors detected

3 - cancelled, no outputs

4 - interrupted, partial outputs generated

negative - undetected run-time error

The sample code:

```
Private Function funExecCmd(CMDLINE$) As Long
   Dim proc As PROCESS_INFORMATION
   Dim start As STARTUPINFO
   Dim ret As Long
   Dim lngExitCode As Long
   ' Initialize the STARTUPINFO structure:
   start.cb = Len(start)
   ' Start the shelled application:
   ret = CreateProcessA(0&, CMDLINE$, 0&, 0&, 1&, _
      NORMAL_PRIORITY_CLASS, 0&, 0&, start, proc)
   ' Wait for the shelled application to finish:
   ret = WaitForSingleObject(proc.hProcess, INFINITE)
   If GetExitCodeProcess(proc.hProcess, lngExitCode) Then
      funExecCmd = lngExitCode
   Else
      funExecCmd = -1 'untrapped error
   End If
   ret = CloseHandle(proc.hProcess)
End Function
```

For example, call the above function from VB as follows:

```
   frmMDI.Hide 'hide the container application
   lngExitCode = funExecCmd(strCmdLine)
   frmMDI.Show
   Select Case lngExitCode 'as coded by Modgen
      Case 0
         lblStatus.Caption = "Application ended with task manager."
      Case 1
         lblStatus.Caption = "The simulation completed successfully."
      Case 2
         lblStatus.Caption = "The simulation ended in an error."
      Case 3
         lblStatus.Caption = "The simulation was cancelled and no partial
results were saved."
      Case 4
         lblStatus.Caption = "The simulation was cancelled and partial results
were saved."
      Case Else
         lblStatus.Caption = "The simulation ended with a fatal run-time error."
  End Select
```

The string strCmdLine above contains the model executable filename followed by the –sc command line option and a saved scenario_file. A word of caution: any fully qualified filename containing embedded blanks (this includes the executable name in the first position) must be enclosed in double quotes. An example:

```
 strCmdLine = """c:\Statistics Canada\simpex\simpex.exe""" & _
   strCmdLine = strCmdLine & " -sc """ & strFileSce & """"
```

Debugging and Optimizing Tools and Techniques

Introduction to Debugging

This discussion assumes that the model developer has a good knowledge of the C++ debugging environment. One of most powerful aspects of the Modgen architecture is that it enables the developer to debug the model code within its .mpp files. Although you should rarely be required to open a .cpp file, open one and take a quick look. You will see that this was accomplished firstly through #line directives to the compiler. Secondly, with few exceptions after the #line 1 directive is given, Modgen maintains a 1-to-1 correspondence between a line of .mpp code and a line of .cpp code.

Debugging is a three-phase process.

The first phase involves correcting Modgen syntactic errors. These are generated by the Modgen compiler after running Modgen.exe on your .mpp files. For example, a syntax error in an actor definition will be caught here. Only Modgen syntactic errors in .mpp files are detected at this stage. Syntactic errors in .dat files are detected and reported by Modgen at run-time.

The second phase involves correcting C++ compiler and linker errors during the build process. These errors are usually C++ syntactic errors or mispelled C++ keywords and are relatively easy to fix.

The third debugging phase occurs after successful C++ compilation. These errors could either be run-time errors causing model execution to fail or more serious logic errors i.e. the model works but produces incorrect results. The most common run-time error in Modgen is a reference to a NULL pointer. The most common logic error in Modgen is an incorrect table specification. The remainder of this section covers this third debugging phase.

Creating a debug version

Create a debug version of your executable and set your active configuration to Debug. When running the debugger,

breakpoints may be set on the appropriate lines of code (not the Modgen definitions) in the .mpp files. It is also possible to set breakpoints in the ACTORS.cpp and TABINIT.cpp files (generated by Modgen) however this requires a detailed knowledge of Modgen internals and is not very useful in most cases.

When the run-time error occurs, the call stack should be examined. Sometimes the error occurs inside the Modgen DLL or inside ACTORS.cpp or TABINIT.cpp. If this is the case, then the function from the .mpp file closest to the top of the stack should be examined. In most cases, this will be enough to identify the problem.

To display the value of the variable, type its name in the Watch window. Special handling is only required for parameter names which must be prefixed by "mm::gprParam->" (e.g. mm::gprParam->ModelCohortOnly). In these examples, "mm" is the C++ namespace used by Modgen to protect all symbols declared in model code.

Correction of logic errors and table validation are the most difficult tasks for developers. Use of the tracking database in conjunction with the BioBrowser is appropriate for small runs. Use different filters to observe small subsets of the population of interest, track as many states as you can and always be sure to track the case_seed. For large runs, the expression min_value_in(case_seed) can be added to the tables of interest. It will display the smallest seed of the case that resulted in an entry to a cell that was expected to be blank. That seed can then be used to debug the problem case.

In rare cases, the Debug environment can behave differently than the Release environment. In this case, to simplify the debugging process, re-compile with all compiler optimizations disabled.

Debugging Tools and Techniques

Bounds checking

Parameters: Modgen models compiled in debug mode check, when run, that each index used in an array parameter does not exceed the correct bounds of the associated dimension. If a parameter array bound is violated, a message box describing the problem will be displayed. The same information will be written to the scenario log file. After the message box is displayed, a second dialog box will allow the user to exit the model, to ignore the error, or to invoke the debugger.

Bounds checking is not active in models compiled in release mode, and has no effect on performance in that mode. Bounds checking can be de-activated in debug mode by defining the manifest constant MG_NCHKBND in Visual Studio using "Preprocessor definitions" in the "C/C++" tab in Project Settings. Developers may wish to de-activate bounds checking in debug mode if they know that their model does not violate array bounds and if improving performance in debug mode is essential for troubleshooting a specific problem.

Modgen developers are strongly encouraged to use debug mode to identify array bounds errors before releasing models to users or other developers. Otherwise many error dialog boxes generated due to array bounds violations could make it

difficult to use the model in debug mode – not to mention that a serious error in logic is present!

Actor data member (state arrays) : Modgen verifies that array bounds of actor data members are not violated at run-time for models compiled in debug mode. This feature operates in the same way as for array parameters. Bounds checking decreases the simulation speed of models compiled in debug mode by about 30%

Classification state: Modgen will produce a run-time message box in debug mode when an attempt is made to assign an invalid value to a classification state. The message indicates the state name and the value to be assigned. In release mode such values are silently coerced to the first and last value of the classification.

Verification of memory integrity

In debug mode, Modgen will automatically verify the integrity of certain internal memory structures and will display a message box if user code has compromised these structures. This verification is no guarantee that errors in user code are not present, but rather provides a possible means of identifying if user code is responsible for anomalous behaviour such as crashes, etc. Note that user code is still eminently capable of compromising the stability of the model executable.

Check Sum

Modgen can generate a "unique" number for each case simulated, from the events that occur during the case and the time when each event occurs. This functionality can be activated both in release and in debug mode by defining the manifest constant MG_CHKSUM in Visual Studio using “Preprocessor definitions” in the “C/C++” tab in Project Settings. The case seed and check sum for each case will then be written in the debug file (ex. Base(debug).txt).

This can be used to identify one case which is different between two simulations which should be identical, for instance.

Tracing events

Modgen models can trace all the events occurring in a case, or in a portion of a case. This functionality can be activated in either debug or release mode by defining the manifest constant MG_TRCEVNT in Visual Studio using “Preprocessor definitions” in the “C/C++” tab in Project Settings. The model developer can also specify in the code when to start tracing events as well as when to stop tracing events. To do this, Modgen uses the two functions StartEventTrace() and StopEventTrace().

void **StartEventTrace**( );

void **StopEventTrace**( );

Tracing events does make the simulation extremely slow, so it is wise to limit as much as possible the use of this functionality. When the model traces events, the debug file will contain an entry for each event which will include the case seed, the actor identifier, the name of the actor (ex. Person), the name of the event and the time when the event occurred. For scheduled events, instead of having the name of the event, the entry will contain « scheduled – » followed by the number identifying which scheduled event occurred. It can be used, for instance, after a case has been identified with the check sum method, to identify the first event that differs.

SkipCases function

This function allows you to skip the simulation of a given number of cases. SkipCases starts cases and ends them so that the next seed is the same as if the cases had been simulated.

Supposing you had a series of three cases with the following case seeds :

```
6589593
25768594
183475
```

Skipping one case would have as a result the simulation of a series of two cases with the following case seeds :

```
25768594
183475
```

SkipCases also modifies the distribution of cases among the subsamples so that it is the same as it would have been if the cases skipped had been simulated. Supposing 5000 cases are requested and another 5000 cases are skipped, Modgen will distribute the cases as if 10000 cases had been requested, but will simulate only 5000 cases.

Of course, cases skipped will not be included in tracking or in the number of cases processed. The number of cases skipped does not affect the population size either.

This function facilitates debugging because its use makes it possible to simulate, for instance, only the second half of a simulation, or the last ten cases.

SkipCases has to be called before cases are simulated. To use it, create a parameter containing the number of cases to be skipped and call the function before CaseSimulation in each thread. If a case has been simulated in the thread before SkipCases is called, an error message will be shown and the simulation will continue without skipping cases in that thread. Here is an example from Simpex:

```
void Simulation()
{
	long		lCase;

	SkipCases(CasesToSkip);
	// case loop
	for ( lCase = 0; lCase < CASES() && !gbInterrupted && !gbCancelled &&
!gbErrors; lCase++ )
{
	// simulate the next case
	StartCase();
	CaseSimulation();
	SignalCase();
}
}
```

where CasesToSkip is a parameter created to contain the number of cases that should be skipped.

Note that the number of cases specified in the scenario does not include the cases skipped. Modgen will skip the number of cases specified as arguments of SkipCases then will simulate the number of cases asked in the scenario. As an example, if you ask for 10 cases in the scenario and put CasesToSkip = 1000, the model will skip 1000 cases but will

still simulate 10 cases as was asked. Here's a second example. To simulate the second half of a 1000000 cases simulation, ask (in the scenario) for 500000 cases and request to skip 500000 cases.

WriteDebugLogEntry

The function WriteDebugLogEntry writes in the file *scenario_name*(debug).txt. It can be used in release mode as well as in debug mode. It will not work with distributed simulations. The easiest way to use it is to pass it a string. Here's an example:

```
CString szDebugExample = "";
szDebugExample = "Now in BirthdayEvent";
WriteDebugLogEntry(szDebugExample);
```

Please note that using this function, or tracing the events, will make the debug file huge. This function should thus be used only when all else fails.

How to selectively execute a single replicate

A single case can be simulated in a case-based model by specifying a starting seed. Time-based models have no corresponding built-in ability to simulate a single replicate if multiple replicates were requested. Simulating a single replicate can sometimes be useful for debugging purposes. To simulate a single replicate, insert a line such as the following as the first line of the Simulation() function:

```
// simulate only replicate #4
if ( 4 != GetReplicate() ) return;
```

Run-time error handling

Certain critical "run-time" errors can occur when executing model code during a simulation. Some examples are division by zero, numeric overflow, or lack of memory.

For Release versions of models, a run-time error produces an informative message in the "Errors and Warnings" area at the bottom of the model progress window. The message is also written to the log file of the scenario. The model will produce no output tables but will produce a tracking file if tracking was enabled. If the model was running in interactive

mode, the simulation will end immediately but the program interface will continue to function. If the model was running in batch mode, the model will exit with a return code of 2. If the machine is participating in an Executor pool, a run-time error in a Modgen model will allow the machine to execute other jobs afterwards.

When a run-time error occurs in a Release version of a model, different information is presented depending on the nature of the error, the type of model (case-based, time-based, cell-based) and the phase of execution (PreSimulation, Simulation, Postsimulation, UserTables). The following example was extracted from a scenario log file for a case-based model where the error occurred in the simulation phase.

```
Error:
      Type: invalid argument to sqrt()
      Phase: Simulation
      Subsample: 1
      Event: Person.AgeOneYearEvent(implement)
      case_seed = 2147500455
      time = 18.000000
      actor_id = 33
```

As the example illustrates, the name of the current event is displayed as well as an indication of whether the error occurred in the 'time' or 'implement' function of the event. Note that a run-time error that occurs during the implementation of an event might be only indirectly associated with the code in the implement function of the event. For example, if 'a' is a simple state and 'b' is a derived state defined as 'b = exp(a);', an event implement function that sets 'a=1000;' will generate a run-time error associated with the event, even though the call to 'exp' that generated the error is not located in the implement function of the event.

The error information may include the value of the state actor_id. This can be useful in identifying which actor was responsible for the error. Note that actor_id is an arbitrary unique identifier assigned to each actor in a simulation. To produce reproducible values for actor_id, the scenario starting seed must be identical, and the number of simulation threads in the scenario should be set explicitly to 1.

Virtually all run-time errors (including out of memory and reading or writing invalid memory locations) are caught and reported in the Release version of models. There are specific measures to identify errors that result from invalid

arguments to the frequently used math functions exp(), log(), pow() and sqrt(). Most errors will be reported with comprehensible information under 'Type:', but some rarer errors may require recourse to system documentation to understand. For example, the error EXCEPTION_STACK_OVERFLOW may result from model code that attempts to create an excessively large array in a temporary variable in a function. The error EXCEPTION_IN_PAGE_ERROR may arise as a result of the model being run over a network and not having been linked with the /SWAPRUN:NET option.

When a run-time error occurs in a Debug version of a model, the error is not caught by Modgen. Instead, a dialog box is presented that permits entering the Visual Studio debugger to identify the model code that caused the problem. Depending on the location and nature of the problem, it may be necessary to first use the call stack window in Visual Studio to select the model function that was executing when the error occurred

Cleaner application

The installation of Modgen includes another application: Cleaner.exe. This small application deletes all the Modgen temporary files in your temp folder MDGNxxx.xxx), without removing files that were not created by Modgen. The installation of Modgen adds a shortcut for Cleaner in your startup folder. As a result, when you restart your computer, Modgen temporary files are deleted from your temp folder (in silent mode).

The Cleaner application was created to avoid the situation of simulations crashing due to a lack of disk space. Unless there is a simulation running on your machine, you can use it any time to remove Modgen files in your temp folder, which you might wish to do, for example, after some debugging sessions. However, do NOT use this application while a simulation is running on your machine or else the simulation will crash.

Optimizing Techniques

Running Modgen models reliably over a network

Modgen models often have long running times. If a model is run across a network (e.g. the executable file is located on a server and is run on a workstation), stale connections or other network curiosities can result in obscure run-time errors.

The approach to solve this problem involves a one-time modification to the Visual Studio properties of each Modgen project. Open the solution file for the project in Visual Studio, and pick Project / Properties. Under Configuration Properties on the left pane, click "Linker" and then select "System". Then change the setting "Swap Run From Network" from "No" to "Yes (/SWAPRUN:NET)" and recompile the model.

It's also possible to set this option in an existing executable without re-compiling it. This can be useful for running old models (perhaps compiled with an earlier version of Modgen or Visual Studio) reliably across a network. To set the option in an existing executable without recompiling, open a command prompt and run the EDITBIN utility (supplied with Visual Studio) setting the SWAPRUN option as in the following example:

EDITBIN /SWAPRUN:NET simpex.exe

These changes have been applied to the Visual Studio project file for the Simpex model that is provided with Modgen.

Increasing available memory to Modgen models on Windows x64 operating systems

The memory available to a Modgen model can be doubled if the model is run on a 64-bit Windows operating system such as Windows XP x64 Edition. This can be particularly significant for time-based models where memory is at a premium.

To enable this functionality, open the solution file for the project in Visual Studio, and pick Project / Properties. In the left pane, click "Linker" under "Configuration Properties" and select "System". Change the setting "Enable Large Addresses" to "Support Addresses Larger Than 2 Gigabytes (/LARGEADDRESSAWARE)". Recompile the model.

It's also possible to set this option in existing executables without re-compiling the model. Use the EDITBIN utility (supplied with Visual Studio) as in the following example:

EDITBIN /LARGEADDRESSAWARE simpex.exe

Using the *options* statement

The *options* keyword is followed by one or more options, separated by white space. Each option consists of a keyword followed by =, followed by the option value. Multiple *options* statements are permitted, each separated by a white space, and are cumulative in effect. If the same option is specified multiple times in the model source code, the last option encountered by Modgen takes precedence. Certain Modgen capabilities that previously required changing C++ compiler options can instead be specified using the *options* statement.

Example:

```
options bounds_errors=off;
```

The following table lists the options available in the current version of Modgen and, the equivalent C++ manifest constant (if any).

Option	Default value	Description	Equivalent compiler option
bounds_errors = on | off	on	Controls run-time errors on out-of-bounds range or classification assignment (Debug version only)	MG_NCHKLMT (undocumented)
index_errors = on | off	on	Controls run-time errors on out-of-bounds array index (Debug version only)	MG_NCHKBND
case_checksum = on | off	off	Controls the production of a checksum for each case (Debug and Release versions)	MG_CHKSUM
event_trace = on | off	off	Controls the trace event capability (Debug version, also requires use of helper functions).	MG_TRCEVNT
fp_consistency = on | off	off	Controls whether the C++ compiler generates more consistent but less efficient floating point code	none
packing_level = 0 | 1 | 2	0	Controls memory optimization of actor states	none
permit_all_cus = on | off	off	Controls whether continuously updated states are allowed to be used in event time functions or as filters in tables (the <off> setting means they cannot be so used)	*none (to be confirmed)*

Option bounds_error:

Modgen always ensures that assignments to classification or range actor states fall within the allowed limits of the

classification or *range* by truncating to the minimum or maximum allowed value. The behaviour of deliberately generating run-time errors if an invalid value is assigned to a *range* or *classification* state is enabled or disabled by using the *bounds_errors* option. This allows the developer to use small ranges in *time_based* models to improve memory efficiency (in conjunction with the *packing_level* option).

Option fp_consistency:

In Release versions of models, the C++ compiler normally optimizes the speed of floating point operations. A slight change in model code that does not affect the logic of the model (such as those produced by *packing_level*) can result in differences in results when floating point code is optimized. Setting *fp_consistency* to *on* will reduce or eliminate such effects, but the resulting model will be slower. The option has no effect for normally configured Debug versions of models. The *fp_consistency* option can be useful when testing for differences between model versions.

Option packing_level:

The *packing_level* option controls how Modgen defines and orders actor states to reduce memory use. This option can be useful in time-based models with many actors. Higher values of *packing_level* reduce memory use, but at the expense of increased computation time. The default value of *packing_level* is 0, which means no packing is performed apart from efficient ordering of C++ members of actor-related structures and classes. *packing_level*=1 means that Modgen will use the smallest possible simple C++ integral type to store *range*, *classification*, *partition*, and *logical* states. *packing_level*=2 means that Modgen will pack variables of type *range*, *classification*, *partition*, and *logical* into the minimum number of bits possible. The available packing levels are summarized in the following table.

packing_level	Description
0	Use the C++ type int to store states of type *range*, *classification*, *partition*, and *logical*.
1	Use the smallest possible simple C++ type to store states of type *range*, *classification*, *partition*, and *logical*.
2	Use the smallest possible number of bits to store states of type *range*, *classification*, *partition*, and *logical*.

For Release versions of models, differences in floating point optimized code generated by the C++ compiler can result in different model results depending on the value of *packing_level*. This does not affect Debug versions of models. In situations where consistency is more important than efficiency, the model developer can use the *fp_consistency* option to

turn off the optimization of floating point code in the Release version of a model.

A *packing_level* of 1 or 2 will cause a C++ compiler error in the commonly used Clock.mpp module. To eliminate the error, change the lines

```
logical     bDayBoundary = {FALSE};
logical     bWeekBoundary = {FALSE};
logical     bMonthBoundary = {FALSE};
logical     bYearBoundary = {FALSE};
```

to

```
BOOL  bDayBoundary = {FALSE};
BOOL  bWeekBoundary = {FALSE};
BOOL  bMonthBoundary = {FALSE};
BOOL  bYearBoundary = {FALSE};
```

in the two functions ClockEvent and FinishClock.

The Modgen memory usage report does not include the effect of *packing_level*. This report will be updated in a future version of Modgen.

Option permit_all_cus:

Unless specifically set to “on”, continuously updated states are not allowed to be used in event time fuctions, or as a filter or classificatory dimension of a table. In such situations, Modgen will return an error indicating the illegal use of a continuously updated state and the model will not compile.

When Modgen returns such an error and you wish to have more information, you can use the help file. To be able to produce the help file, the model must have been compiled. If necessary, you can temporarily set the option permit_all_cus = on, compile the model and produce the help file, then later set it back to off.

The help file provides a list of all continuously updated states and their uses. For each continuously updated state, other states derived from it (which are also continuously updated) are additionally indicated, which makes it easy to understand why a given state is continuously updated. States which are continuously updated are:

- age
- time
- duration(…)
- weighted_duration(…
- active_spell_duration(…)
- active_spell_weighted_duration(…)
- any state derived from one of the above

States which have the same member name (without the name of the actor) as a continuously updated state of another actor are considered to be continuously updated and are also prohibited in event time functions. However, you can still use such states in table filters and table classificatory dimensions.

How to improve memory efficiency in time-based models

Time-based models with many actors make heavy demands on memory. The maximum number of actors that can be simulated is limited by available memory and the efficiency of use of that memory. This section contains a number of recommendations for improving memory efficiency in time-based Modgen models. These recommendations are subject to the particularities of a given model.

- Declare all floating point actor states as *real* rather than as *double* or *float*, and specify

  ```
  real_type float;
  ```

- Declare all integer actor states as ranges rather than as *int*, and specify

  ```
  options bounds_errors=off;
  ```

  and

  ```
  options packing_level=2;
  ```

- Specify either

  ```
  options packing_level=1;
  ```

  or

```
options packing_level=2;
```

- Set the number of threads to 1 in Scenario / Settings, if the model might execute on a multi-processor or hyper thread enabled machine.
- Specify

  ```
  time_type float;
  ```

- If the model permits, specify

  ```
  counter_type ushort;
  ```

- If the model permits, specify

  ```
  integer_type short;
  ```

- If the number of cells in any table is 65535 or less (the number of replicates requested in a scenario affects this), specify

  ```
  index_type ushort;
  ```

- Examine the memory usage report to better understand what parts of the model consume the most memory. The use of distinct modules improves the usefulness of the report.
- Comment unneeded or rarely-used tables in model source code.
- Place rarely used tables and associated actor states in separate modules so that they can be excluded easily from the production version of the model.
- Consider logically-equivalent designs that reduce the number of events declared in the model.
- Consider logically equivalent designs that reduce the number of actor states.

Reference Material

Modgen Command Summary

This section lists the Modgen definitions, keywords , expressions and special functions by the nature of their usage. Its purpose is a syntactic summary of the Modgen language. The only Modgen functionality available at run-time for parameter inputs, is the ability to change parameter values and their display order. Any other alterations imply a re-compilation of the model. For example, changing the rank (number of dimensions), shape and type of a parameter will involve a re-compilation. For the BioBrowser tracking file, changing the track statement (e.g. altering the filter or adding another track state for an actor) will also involve re-compilation. On the output side, you can not change tables without re-compilation. However, execution control options allow you to set which tables and parameters are sent to output.

Definitions and Keywords

actor actor_name { elements};	Defines an actor	.mpp
aggregation aggregated_classification, detailed_classification {…	Defines an aggregation	.mpp
classification classification_name { elements };	Defines a classification type	.mpp
cumrate parameter_name …	Defines a cumrate model parameter	.mpp, .dat
event time_function,implement_function<,priority_code>	Defines an event within an actor definition	.mpp
extend_parameter parameter1<,parameter2>;	Extends the stored parameter	.mpp

	data	
FALSE	Possible value for model logical	.mpp, .dat
hook from_function_name, to_function_name;	Defines a function hook into a member function or an event implement function	.mpp
languages …	Defines the languages used by the model	.mpp
link actor_name1 actor_name2;	Establishes a link between actors	.mpp
logical variable_name…	Defines a logical state or parameter	.mpp, .dat
model_type case_based/time_based/cell_based;	.Defines the model type	.mpp
model_generated parameter_name…	Defines a model parameter which is generated	.mpp
NULL	Special keyword for null pointer	.mpp
parameter_group group_name {name,…,name};	Defines a parameter group	.mpp
parameters { model_parameters };	Defines a set of model parameters	.mpp, .dat
partition partition_name { elements };	Defines a model partition	.mpp
piece_linear parameter_name…	Defines a piece_linear model parameter	.mpp, .dat
range range_name { elements };	Defines a model range	.mpp, .dat
table <sparse> actor_name <table_name> <[filter]> { elements };	Defines a table specification	.mpp
table_group group_name {name,…,name};	Defines a table group	.mpp
TIME state_name…	Defines a state to be type	.mpp

	TIME	
TIME_INFINITE	Special waiting time until next event	.mpp
time_type value;	Defines the type of microsimulation model	.mpp
TIME_UNDEF	Special waiting time until next event	.mpp
track actor_name [intial_filter<,final_filter>] { elements };	Define states to be tracked for a given actor	.mpp
TRUE	Possible value for model logical	.mpp, .dat
UNDEF_VALUE	Sets null values in user tables	.mpp
user_table <sparse> table_name {elements};	Defines a user table	.mpp
version value;	Assigns a version number to the model	.mpp

Note 1: If descriptive labels are required, you must use multiple lines.

Note 2: < > denotes optional arguments, … indicates an incomplete statement.

Derived State Expressions Used in Table and Actor Definitions

changes(state)	Counts changes to a state
active_spell_delta(variable, state, analysis variable)	Change in analysis variable over a state's current state
active_spell_duration(variable, state)	Length of time a state has been in its current state
active_spell_weighted_duration(variable, state, weighting variable)	Accumulate a rate variable over a state's current state
completed_spell_delta(variable, state, analysis variable)	Change in analysis variable during last completed spell of a specified state
completed_spell_duration(variable, state)	Length of time a state was in last completed spell of a specified state
completed_spell_weighted_duration(variable, state, weighting variable)	Accumulate a rate variable over the last completed spell of a specified state
duration(variable,state)	Length of time a state has ever been in a specified state
duration_counter(state, value ,size of time interval, maximum value)	Counts the number of intervals of the specified size that have elapsed since the state reached the value, up to the optional maximum value
duration_trigger(state,value ,elapsed_time)	1 if length of time a state is in a specified state for a given length of time
entrances(variable, state)	Counts the number of times a state has entered a specified state
exits(variable, state)	Counts the number of times a state has left a specified state
undergone_change(variable)	Value of analysis variable when state first changed states
undergone_entrance(variable, state)	Value of analysis variable when state first entered a specified state
	Change in analysis variable during last completed

	spell of a specified state
split(state, parition_name)	Partition analysis variable into classifications for table dimension
transitions(variable, from state, to state)	Count the number of times a state has entered a state from another state
value_at_changes(variable, recorded variable)	Sum of the values of an analysis variable when a specified state changes
value_at_entrances(variable, state, recorded variable)	Sum of the values of an analysis variable when a specified state is entered
value_at_exits(variable, state, recorded variable)	Sum of the values of an analysis variable when a specified state is left
value_at_first_change(variable, recorded variable)	Value of an analysis variable when a specified state first changes
value_at_first_entrance(variable, state, recorded variable)	Value of analysis variable when a specified state first enters a specified state
value_at_latest_change(variable, recorded variable)	Value of an analysis variable when a specified state last changed
value_at_latest_entrance(variable, state, recorded variable)	Value of an analysis variable when a specified state last entered a specified state
value_at_transitions(variable, from state, to state, recorded variable)	Sum of the values of an analysis variable when a state goes from one state to another
weighted_cumulation(state1,state2 **)**	When state1 changes multiply by state2 before cumulation
weighted_duration(<variable, state,> weighting variable)	Accumulate a rate variable over the time a state is in a specified state

Expressions Used in Table Definitions Only

delta(***state*** **)**	**Change in value of an analysis variable**
delta2(*state*)	Squared change in value of an analysis variable
event(*state)*	Report simultaneous change in analysis and classification vars under new classification

interval(*state)*	Report simultaneous change in analysis and classification vars under old classification.
max_delta(*state* **)**	Maximum change
max_value_in(*state* **)**	Maximum value when first entering a table cell
max_value_out(*state* **)**	Maximum value when exiting a table cell
min_delta(*state* **)**	Minimum change
min_value_in(*state* **)**	Minimum value when first entering a table cell
min_value_out(*state* **)**	Minimum value when exiting a table cell
nz_delta(*state*)	Equals 1 if there is change in the value of an analysis variable
nz_value_in(*state*)	Equals 1 if value of analysis variable in entrance to a table cell is non zero
nz_value_out(*state*)	Equals 1 if value of analysis variable in exit of a table cell is non zero
sqrt(*state*)	Square root of an analysis variable
unit	Number of actors which entered each cell in a table
value_in(*state*)	Value of analysis variable when first entering a table cell
value_in2(*state*)	Squared value of analysis variable when first entering a table cell
value_out(*state*)	Value of analysis variable when exiting a table cell
value_out2(*state*)	Squared value of analysis variable when exiting a table cell

Commands and Functions Used with Links

Add(actor_pointer)	**Adds an actor of the right type to the link**
count(link_name)	Returns the number of linked actors of a certain type
IMPLEMENT_HOOK	Location of the function hook
FinishAll( )	Call the finish function
***GetNext**(int nInitPos, int *pnPos)	Returns the position of the next actor in a multiple link
max_over(link_name, state_name)	Maximum of the specified state over all linked actors
min_over(link_name, state_name)	Minimum of the specified state over all linked actors
Remove(int nPosition)	Removes the actor at the specified position from the link
RemoveAll( )	Removes all the actors from the link
sum_over(link_name, state_name)	Sum of the requested state over all linked actors

Modgen Supplied C++ Functions

The Modgen supplied C++ functions have been summarized in the section entitled “Run-time Functions and Macros”. They can be used anywhere within Modgen code.

Modgen Documentation Database

The Modgen model designer has the ability to document, through a short descripitive label as well as through more thorough descriptive text, every element (actors, states, parameters, events, tables, etc.) of a particular model. If the designer provides this descriptive text in multiple languages, the resulting model can produce outputs in the language selected by the user a run-time. The documentation database captures all such information as an MS Access database consisting of the following tables:

Table: *VersionInfo*
Description: File version.

Fields: *Version*

Table: *LanguageDic*
Description: Dictionary of supported languages. Selected run-time language is indicated in *Selected* field. Translations of a few important Modgen keywords are also included here (in fields *All, Min, Max, Contents, Parameters*).
Fields: *LanguageID* (numeric) , *LanguageCode* (e.g. "EN" for English), *LanguageName, All, Min, Max, Contents, Parameters*

Table: *AutoParserErrorDic*
Description: Descriptions of errors returned by Modgen Autoparser OLE server.
Fields: *ErrorID, ErrorDescription, LanguageID*

Table: *TypeDic*
Description: Dictionary of all variable types used in a model. Includes simple types, classifications, ranges and partitions. For each type more information can be obtained from the more specific dictionary (identified by *DicID* field) by using *TypeID* field.
Fields: *TypeID, DicID* (0 - *SimpleTypeDic*, 1 - *LogicalDic*, 2 - *ClassificationDic*, 3 - *RangeDic*, 4 - *PartitionDic*, 5 - *LinkTypeDic*)

Table: *SimpleTypeDic*
Description: Dictionary of simple data types: int, long, float, double, TIME, cumrate, piece_linear.
Fields: *TypeID* (same as in *TypeDic*), *Name*

Table: *LogicalDic*
Description: Contains the values and descriptions associated with “logical” type.
Fields: *TypeID, Name, Value, ValueName, ValueDescription, LanguageID*

Table: *ClassificationDic*
Description: Dictionary of all classifications.
Fields: *TypeID, Name, Description, Note, SourceFileID, NumberOfValues, LanguageID, ContextID*

Table: *AggregationDic*
Description: Dictionary of all aggregations.

Fields: *Name, AggregatedClassificationID, DetailedClassificationID*

Aggregation tables (named according to *AggregationDic*)

Description: Detailed mapping of a specific aggregation.

Fields: *DetailedValue, AggregatedValue*

Table: *RangeDic*

Description: Dictionary of all ranges.

Fields: *TypeID, Name, Description, Note, SourceFileID, Min, Max, LanguageID, ContextID*

Table: *PartitionDic*

Description: Dictionary of all partitions.

Fields:*TypeID, Name, Description, Note, SourceFileID, NumberOfValues, LanguageID, ContextID*

Table: *LinkTypeDic*

Description: Dictionary of all link types.

Fields:*TypeID, Name, LinkedActorID*

Table: *ClassificationValueDic*

Description: Dictionary of all members of all classifications.

Fields: *TypeID, EnumValue, Name, Description, Note, LanguageID, ContextID*

Table: *PartitionValueDic*

Description: Dictionary of all breakpoints of all partitions. Each breakpoint is stored in numeric and string format.

Fields: *TypeID, Position, Value, StringValue*

Table: *ParameterDic*

Description: Dictionary of all parameters.

Fields: *ParameterID*, *Name*, *Description*, *Note, SourceFileID*, *TypeID*, *Rank*, *NumberOfDatapoints*, *NumberOfCumulatedDimensions*, *ModelGenerated* (Yes/ No), *Extendible* (Yes/ No), *RelatedParameterID, LanguageID, ContextID*

Table: *ParameterDimensionDic*

Description: Dictionary of all dimensions of all parameters

Fields: *ParameterID*, *Position*, *TypeID, DisplayPosition*

Table: *ParameterDependencyDic*

Description: Dictionary of parameter dependencies in parameter extensions

Fields: *ParameterID, DependentParameterID*

Table: *ParameterGroupDic*

Description: Dictionary of all parameter groups.

Fields: *ParameterGroupID*, *Name*, *Description*, *Note, LanguageID, ContextID*

Table: *ParameterGroupMemberDic*

Description: Dictionary of all members of all parameter groups.

Fields: *ParameterGroupID, Position, ParameterID, MemberGroupID*

Table: *ActorDic*

Description: Dictionary of all actors.

Fields: *ActorID*, *Name*, *Description*, *Note, LanguageID, ContextID*

Table: *ActorStateDic*

Description: Dictionary of all states of all actors.

Fields: *StateID* (unique across all actors), *Name, Description, Note, SourceFileID, ActorID, TypeID, KindOfState* (0 - simple, 1 - derived, 2 – identity expression, 3 - linked, 4 – generated expression for tabulation or cumulative operators), LanguageID, *ContextID*

Table: *ActorStateDependencyDic*

Description: Dictionary of state dependencies.

Fields: *StateID, DependentStateID*

Table: *ActorUnknownStateDic*

Description: Dictionary of unknown actor states encountered while generating the list of states read/set by a function or an event. This could be due to complicated pointer expressions involving actor states.

Fields: *StateID, Name*

Table: *ActorEventDic*

Description: Dictionary of all events of all actors.

Fields: *ActorID, EventID, WaitFnName, ImplementFnName, Note, SourceFileID* (of declaration), *LanguageID, ContextID*

Table: ActorEventStateDic

Description: Dictionary of input states of all actor events.

Fields: *ActorID, EventID, StateID*

Table: ActorEventStateReadDic

Description: Dictionary of all states read by the implement function of all actor events

Fields: *ActorID, EventID, StateReadID*

Table: ActorEventStateSetDic

Description: Dictionary of all states set by the implement function of all actor events.

Fields: *ActorID, EventID, StateSetID*

Table: *ActorFunctionDic*

Description: Dictionary of all actors functions (excluding events).

Fields: *ActorID, FunctionID, Name, Description, Note, SourceFileID* (of declaration), *Declaration, LanguageID, ContextID*

Table: *ActorFunctionStateReadDic*

Description: Dictionary of all states read by all actor functions.

Fields: *ActorID, FunctionID, StateReadID*

Table: *ActorFunctionStateSetDic*

Description: Dictionary of all states set by all actor functions.

Fields: *ActorID, FunctionID, StateSetID*

Table: *ActorFunctionHookedFunctionDic*

Description: Dictionary of all function hooks to actor functions.

Fields: *ActorID, FunctionID, HookedFunctionID*

Table: *ActorEventHookedFunctionDic*

Description: Dictionary of all function hooks to actor events.

Fields: *ActorID, EventID, HookedFunctionID*

Table: *ActorLinkDic*

Description: Dictionary of links between the actors.

Fields: *ActorID, LinkID, TypeID, Name, Description, Note, SourceFileID, Single* (Yes/ No), *LinkedActorID, LinkedLinkID, LanguageID, ContextID*

Table: *TableDic*

Description: Dictionary of all output tables.

Fields: *TableID, Name, Description, Note, SourceFileID, ActorID, Rank, AnalysisDimensionPosition, AnalysisDimensionName, AnalysisDimensionDescription, AnalysisDimensionNote, Sparse, LanguageID, ContextID*

Table: *TableClassDimDic*

Description: Dictionary of all classificatory dimensions of all tables.

Fields: *TableID, Position, StateID, Name, Description, Note, TypeID, Totals* (Yes/ No), *LanguageID*

Table: *TableExpressionDic*

Description: Dictionary of all expressions of all tables.

Fields: *TableID, ExpressionID, Name, Description, Note, ExpressionString, Scale, Decimals, LanguageID, DescGenerated*

Here are the values of DescGenerated and their meanings. (Note that this information is primarily of use for tools that use the documentation database to facilitate the translation of a model into a different language.)

0 = the label was specified in the code (in the same language)

1 = the label was generated by Modgen (e.g. for a simple state, the label is the name of the state)

2 = the label was generated by Modgen by taking the label specified in the code for another language

3 = the label was generated by Modgen by taking the label of a state in the same language

4 = the label was generated by Modgen by taking the label of a state in another language

5 = the label was generated by Modgen by taking the Modgen-generated label of another state

Modgen creates descriptive labels according to the following logic:

- If there is a label for the dimension, then DescGenerated is set to 0.
- If there is no label for the dimension in the required language, but there is a label for the dimension in the default language, then this label is shown and DescGenerated is to 2.
- If there is no label in the required language nor in the default language then:
  - if there is a label for the state in the required language, that label is used and DescGenerated is set to 3 if it's not a generated label :
  - If the label is the generated label, then DescGenerated is set to 5.
- If there is no label for the state in the required language, then the label for the default language is used:
  - DescGenerated is set to 4 if that's not a generated label
  - DescGenerated is set to 5 if it is a generated label

Table: *TableRefStateDic*
Description: Dictionary of all states referenced in all tables.
Fields: *TableID, StateID*

Table: *TableGroupDic*
Description: Dictionary of all table groups.
Fields: *TableGroupID, Name, Description, Note, LanguageID, ContextID*

Table: *TableGroupMemberDic*
Description: Dictionary of all members of all table groups.
Fields: *TableGroupID, Position, TableID, MemberGroupID*

Table: *ModuleDic*

Description: Dictionary of all source files (usually mpp files) with file notes attached. This table was formerly known as SourceFileDic, but for backward compatibility purposes, the table SourceFileDic still also exists in the Documentation Database.
Fields: *SourceFileID, FileName, FileNote, LanguageID, ContextID*

Table: *UserTableDic*
Description: Dictionary of all user tables.
Fields: *TableID* (starts at 1000), *Name*, *Description*, *Note*, *SourceFileID*, *Rank*, *AnalysisDimensionPosition*, *AnalysisDimensionName, AnalysisDimensionDescription, AnalysisDimensionNote, Sparse, LanguageID, ContextID*

Table: *UserTableClassDimDic*
Description: Dictionary of all classificatory dimensions of user tables.
Fields: *TableID, Position, Name, Description, Note, TypeID, Totals* (Yes/ No), *LanguageID*

Table: *UserTableExpressionDic*
Description: Dictionary of all expressions of user tables.
Fields: *TableID, ExpressionID, Name, Description, Note, Scale, Decimals, LanguageID, DescGenerated*

Here are the values of DescGenerated and their meanings. (Note that this information is primarily of use for tools that use the documentation database to facilitate the translation of a model into a different language.)

0 = the label was specified in the code (in the same language)

1 = the label was generated by Modgen (e.g. for a simple state, the label is the name of the state)

2 = the label was generated by Modgen by taking the label specified in the code for another language

3 = the label was generated by Modgen by taking the label of a state in the same language

4 = the label was generated by Modgen by taking the label of a state in another language

5 = the label was generated by Modgen by taking the Modgen-generated label of another state

Modgen creates descriptive labels according to the following logic:

- If there is a label for the dimension, then DescGenerated is set to 0.
- If there is no label for the dimension in the required language, but there is a label for the dimension in the default language, then this label is

shown and DescGenerated is to 2.

- If there is no label in the required language nor in the default language then:
  - if there is a label for the state in the required language, that label is used and DescGenerated is set to 3 if it's not a generated label :
  - If the label is the generated label, then DescGenerated is set to 5.
- If there is no label for the state in the required language, then the label for the default language is used:
  - DescGenerated is set to 4 if that's not a generated label
  - DescGenerated is set to 5 if it is a generated label

Modgen Output Database

The Modgen output database is an MS Access database which contains the following tables:

Table: *VersionInfo*
Description: File version.
Fields: *Version*

Table: *LanguageDic*
Description: Language dictionary, same as the one in documentation database. Contains the list of supported languages and translations of some Modgen-generated words encountered in outputs.
Fields: *LanguageID*, *LanguageCode*, *LanguageName*, *All*, *Min*, *Max*

Table: *ModelInfoDic*
Description: Various fields currently found in *ModelInfo* field in *Contents* sheet. LanguageID field is required for multilingual *CompletionStatus*.
Fields: *Time*, *Directory*, *CommandLine*, *CompletionStatus*, *Subsamples* (number) , *CV* (Y/N), *SE*(Y/N), *LanguageID*

Table: *TableDic*

Description: List of non-excluded output tables

Fields: *TableID, Name, Description, Note, Rank, AnalysisDimensionPosition, AnalysisDimensionName, AnalysisDimensionDescription, AnalysisDimensionNote, Sparse, LanguageID*

Table: *TableClassDimDic*

Description: List of classificatory dimensions for all output tables.

Fields: *TableID, Position, Name, Description, Note, TypeID, Totals* (Y/N), *LanguageID*

Table: *TableExpressionDic*

Description: List of expressions used in all output tables.

Fields: *TableID, ExpressionID, Name, Description, Note, Decimals, LanguageID*

Table: UserTableDic

Description: Dictionary of all user tables.

Fields: *TableID* (starts at 1000), *Name, Description, Note, Rank, AnalysisDimensionPosition, AnalysisDimensionName, AnalysisDimensionDescription, AnalysisDimensionNote, Sparse, LanguageID*

Table: *UserTableClassDimDic*

Description: Dictionary of all classificatory dimensions of user tables.

Fields: *TableID, Position, Name, Description, Note, TypeID, Totals* (Yes/ No), *LanguageID*

Table: *UserTableExpressionDic*

Description: Dictionary of all expressions of user tables.

Fields*: TableID, ExpressionID, Name, Description, Note, Decimals, LanguageID*

Table: <*table_name*>

Description: The actual data for the output table indicated by <*table_name*> (regular or user table).

Fields: *Dim0, ..., DimN, Value, CV, SE*

If the simulation was run with more than 1 sub-sample and the model type is not cell-based then additional fields containing sub-sample values are present: *Value0, Value1, …, ValueM* where *M* is one less than the number of sub-samples.

Table: *ParameterDic*

Description: List of output parameters.

Fields: *ParameterID, Name, Description, Note, ValueNote, DataFileID,, TypeID, Rank, LanguageID*

Table: *ParameterDimensionDic*

Description: List of dimensions for all output parameters.

Fields: *ParameterID*, *DisplayPosition*, *TypeID*

Table: <*parameter_name* >

Description: The actual data for the output parameter indicated by <*parameter_name*>

Fields: *Dim0*, ..., *DimN*, *Value*

Table: *TypeDic*

Description: List of all types used by output tables and/or output parameters.

Fields: *TypeID, DicID* (0 - *SimpleTypeDic*, 1 - *LogicalDic*, 2 - *ClassificationDic*, 3 - *RangeDic*, 4 - *PartitionDic*, 5 - *LinkTypeDic*)

Table: *SimpleTypeDic*

Description: List of simple data types used by output tables and/or output parameters.

Fields: *TypeID, Name*

Table: *LogicalDic*

Description: Contains the values and descriptions associated with "logical" type.

Fields: *TypeID, Name, Value, ValueName, ValueDescription, LanguageID*

Table: *ClassificationDic*

Description: List of classifications used by output tables and/or output parameters.

Fields: *TypeID, Name, Description, Note*, *NumberOfValues, LanguageID*

Table: *AggregationDic*

Description: Dictionary of all aggregations.

Fields: *Name, AggregatedClassificationID, DetailedClassificationID*

Aggregation tables (named according to *AggregationDic*)

Description: Detailed mapping of a specific aggregation.
Fields: *DetailedValue, AggregatedValue*

Table: *ClassificationValueDic*
Description: List of all classification members.
Fields: *TypeID, EnumValue, Name, Description, Note, LanguageID*

Table: *RangeDic*
Description: List of ranges used by output tables and/or output parameters.
Fields: *TypeID, Name, Description, Note, Min, Max, LanguageID*

Table: *PartitionDic*
Description: List of partitions used by output tables and/or output parameters.
Fields: *TypeID, Name, Description, Note, NumberOfValues, LanguageID*

Table: *PartitionValueDic*
Description: List of breakpoints of all partitions.
Fields: *TypeID, Position, Value, StringValue*

Table: *LinkTypeDic*
Description: List of link types used by output tables and/or output parameters.
Fields:*TypeID, Name, LinkedActorID*

Table: *DataFileDic*
Description: File notes for all data files.
Fields: *DataFileID, FileName, FileNote, LanguageID*

Format of parameters in output database

Note--if parameters are copied to the output database they are stored in the sparse format (i.e. zeroes are not stored) thus significantly reducing the size of output database for models containing large parameters. In case of the LifePaths model the database size went down from 79 MB to 36 MB. Large savings should occur for the POHEM model as well.

When parameters are exported from the output database to MS Excel format all missing zeroes are added (except in the

sparse format of the pivot table style).

Modgen Tracking Database

This appendix describes release 1.2.0.0 of the tracking database file. All tables except the **History** table within this database are identical in design to the tables in the Modgen Documentation database (refer to Modgen Documentation Database). Within the tracking database, the tables may have fewer fields and fewer records depending on the Modgen track statement used for database creation.

The **History** table contains the actual data for all tracked states. Its fields are:

ObjectID – a long integer
Time – a double
StateID – a long integer
Value – a double
SequenceCnt – a long integer (no repeats over all records).

The Modgen BioBrowser uses SQL queries against this History table to build its graphic displays. To optimize the speed of BioBrowser queries, Modgen indexes this table by ObjectId/ StateId as a combined index and by SequenceCnt as a separate index.

Modgen uses the following rules when creating the tracking file:

- An instance of an actor is uniquely identified by an ObjectID in the History table.
- For each Object, a record is written to the History table for every tracked state only when its value changes. For simultaneous events (same ObjectID, Time & StateID, different Value field), the SequenceCnt field determines the order of display by the BioBrowser.
- Each actor must have a logical state whose Name is "tracking" in the state dictionary ActorStateDic. This is a Modgen generated state (a user generated state can not be called "tracking"). Note that the "Name" field in ActorStateDic is a programmatic identifier for a state and is language independent (unlike the Description and Note fields in that table).
- Every actor will have an internally generated state whose Name is "case_id". This state is not available for display by the BioBrowser (i.e.

it is ignored as an available state in the **Add/insert states** dialog window). A user generated state can not be called "case_id".

- For simple state types, the Value field contains the value of that state at that Time. For classifications, it contains the EnumValue in the ClassificationValueDic. For logical states, it contains the Value field in the LogicalDic. For ranges, it contains an integer value between the Min and Max (inclusive) in the RangeDic. The PartitionDic and PartitionValueDic are always empty in the tracking file (states based on partitions are always written as int's to the tracking file, the integer being the position in the partition). The Value field for Link states is described in detail below.
- Every tracked state (except link states) must have a value at the earliest tracking time. When ordered by SequenceCnt the tracking state will be first and last for all states for that instance of the actor. Its value will be 1 when tracking begins and 0 when tracking ends. Each BioBrowser query contains the tracking state as well as the requested state (this allows the BioBrowser to break up the graphic as the tracking state is shut on and off). The BioBrowser sorts its History table queries by SequenceCnt rather than Time since there are too many ties in the Time field.
- Each StateID in the actor state dictionary and LinkID in the actor link dictionary must be unique across all actors e.g. if a person actor and a child actor both have a state called Gender they will each have their own record and unique StateID in the state dictionary. In addition, the LinkID's in ActorLinkDic, can not be equal to any of the StateID's in the ActorStateDic.
- All dictionaries with language dependent text contain 1 record for each language specified in the LanguageDic.
- Link states are special states: the StateID field in the History table contains the LinkID value for that linked state in ActorLinkDic. In addition, their value field contains a pointer (ObjectID) to the linked object. A link state's Value field contains only -/+ObjectID. This is the only state type that does not contain a value in the History table at the earliest tracking time. The Value field contains ObjectID when that

linked object was first added and –ObjectID when that link state was removed.

Memory Usage Report

During the simulation phase, significant memory use would normally come from four sources: parameters, memory for storing actors, the event queue, and tables.

Parameters

Parameters do not grow in size during a simulation, but may nevertheless consume significant memory if they contain many cells. This is only a problem during the simulation phase if the cells are referenced repeatedly. For example, a large microdata file represented as a parameter may not be a problem if it is read only once at the beginning of the simulation, because the operating system would "page out" the unreferenced memory to disk. If the machine has 2GB or more of physical memory, parameter memory use can reduce the memory available during the simulation phase, because paging cannot increase available memory beyond the 2GB addressing limit of 32-bit processors.

Actor memory

The memory used is the product of the number of actors and (roughly) the number of states. The type of state is also important, with floating point states requiring 8 bytes and other states requiring 4 bytes. Hidden states associated with tables are also present in actors.

Event queue

The memory used is the product of the number of actors and the number of events (for each type of actor) multiplied by the size of the internal structure used for events (a model-independent fixed quantity).

Tables

Tables take up space due to internal states created for each actor, as well as for the common area used to cumulate results. The common area used for tabulation may become large if there are many cells, and if these cells are repeatedly updated during the simulation phase. The table cell area is allocated only for tables that are selected by the user at run-time, whereas the internal table states created in actors are present whether the table is selected or not. The space required for the table cell area also depends on the number of replicates or sub-samples.

Actor sets

The memory used is (roughly) the product of the number of actors in the actor set and the size of the internal structure used to store references to actors in the actor set. Hidden internal states can also be created for actor sets for each actor, whether or not it is a member of the actor set.

The memory usage report identifies areas that are susceptible to change on the part of the model developer or user. Tables are emphasized because eliminating a table is an easy way for a developer to free up memory with no change to

the model structure.

A new checkbox has been added to the Scenario Settings dialog, titled “Memory Usage Report” for case-based and time-based models only, in the Visual Interface to Modgen. If checked, a separate report on memory usage will be generated in a file named scenario_name(usage).htm. The report is generated at the end of the simulation, in the language of the model at the time of report generation. Note that the name is the same in English and in French. For distributed runs, only the statistics for the final “master” job (the one that merges the others), will be saved under the “master” scenario name.

Each part of the memory usage report is explained below.

General information

The first part of the report contains general information about the simulation: date, executable, version, scenario, subsamples or replicates, table copies and simulation threads. The number of simulation threads corresponds to the real number of threads used for the simulation. For instance, if the user specifies a number of threads greater than the number of subsamples, the number of real threads created will be equal to the number of subsamples. The number of table copies is indicated because Modgen keeps copy of the table cells for each subsamples. For case-based models, a copy of the table cells is also kept for the whole simulation. The number of table copies is used in the table memory usage.

Table 1: Maximum instances

Following the general information is a table containing the maximum instances of actors for each type of actor declared in the model (e.g. Person). For practical reasons, that is in fact the sum of the maximum “active” actors in each thread during the simulation. That is used later in the actor memory usage table.

Table 2: Memory use: Parameters

The next table reports the memory usage of parameters. Each parameter is listed in alphabetical order in the table. For each parameter, the memory used in bytes is given. The memory used by a parameter is the number of cells in the parameter multiplied by the memory used for one cell. The memory used for one cell depends on the type of the parameter. Information about the memory used for each type of parameter is given at the end of the report.

Table 3: Memory use: Actors and events

The third table is entitled “Table 3 : Memory use : Actors and events”. There is in fact one table per type of actor in this part of the report. In each of these tables, the memory used is reported by module. There is one row for each module in which memory is used for actors and/or events. Each row contains the following information:

Declared states (bytes per actor)

This is the memory used by states declared by the model developer in that module. It is calculated for one actor only. For instance, if the file PersonCore.mpp includes the following declaration:

```
actor Person
{
      SEX   sex;
};
```

The memory used by the state 'sex' will be counted in the row for module "PersonCore".

Internal states (bytes per actor)

This is the memory used by states created by Modgen for derived states declared in that module. It is calculated for one actor only. For instance, if the file PersonCore.mpp includes the following declaration:

```
actor Person
{
      int   person_year = duration(life_status,ALIVE);
};
```

The memory used for the state "person_year" would be counted in the declared states of the module "PersonCore". However, if a model developer uses the derived state duration(…), Modgen automatically creates one or more "internal states" which also use memory. That memory is counted in the internal states of the module "PersonCore". Information on the maximum memory used by each kind of derived state is provided at the end of the report.

Links (average bytes per actor)

This is the memory used by links declared in the module. For single link, the memory used is the same for all actors. For multiple links, however, the memory used depends on the number of actors linked. For each different link, the maximum actors linked is kept in memory and an average is calculating by taking the memory used by the link at the time it was taking the most memory and dividing by the maximum number of active actors as given in the table "Maximum instances".

Total actor memory (bytes)

The total memory used for actors is the sum of the first three columns, multiplied by the maximum number of active actors in the simulation.

Event functions (number)

This is the number of events declared for that actor in the module. For instance, if the file PersonCore.mpp contains the declaration:

```
actor Person
{
      event timeSchoolYearEvent,SchoolYearEvent, 2;
      event timeSchoolEntryEvent,SchoolEntryEvent, 1;
      event timeElSecDropoutEvent,ElSecDropoutEvent, 1;
      event timeElSecReturnEvent,ElSecReturnEvent, 1;
};
```

then that column would report that 4 event functions are declared in the module “PersonCore”.

Total event memory (bytes)

For one actor, the memory used in bytes corresponds to the memory used for one event multiplied by the number of event functions declared in the module. This column gives the total memory used for events for all actors, which is the memory used for events for one actor multiplied by the maximum actor instances.

Total memory (bytes)

This is the sum of the total event memory and the total actor memory.

Table 4: Memory use: Tables

The next table reports the memory used for tables. Each table is listed in alphabetical order. For each table, the row contains the following information:

Table selected

Memory is used for table cells only if the table is selected in the simulation. This column contains an “X” if the table has been included in this simulation and is blank otherwise.

Table cells (bytes)

This is the memory used by one copy of the table cells, if the table is selected. (Remember that multiple copies of the table cells are kept). This information is provided even if the table was not included in the simulation.

Internal states (bytes per actor) for tables

This is the memory used by states created by Modgen for derived states declared in the table. It is calculated for one actor only. For instance, if the table's declaration is:

```
table Person X03_SeverelyDisabled
{
      {
      entrances( dfle, SEVERE_DIS )
      }
};
```

Modgen will create an internal state for the derived state "entrances(dfle, SEVERE_DIS)". This memory would be reported in the corresponding row of the table.

Modgen also creates internal states for the table itself. That memory is also reported in this column. To help in understanding this column, the memory used for internal states created for a table, (excluding all internal states associated with derived states), is given at the end of the report.

This memory will be used even if the table is not selected.

Total internal states memory (bytes) for tables

This corresponds to the memory used for internal states for one actor multiplied by the maximum active actors as given in the table "Maximum instances".

Total (bytes) for tables

This is the total memory used for the table. If the table was not included in the simulation, then this is the same as the total internal states memory. If the table was included in the simulation, then the table cells memory used, multiplied by the number of table copies, is added to the internal states memory.

Table 5: Memory use: Summary by module

After that comes a summary of the memory used, divided by module. Each module is listed in alphabetical order. For each row, the memory used for parameters, actors and events, and tables declared in the module is reported.

(Potential) Table: Memory use: Actor sets

If no actor set is defined in the model, this table is absent. If the model contains one or more actor set, then it reports the memory used for these actor sets. Each actor set is listed in alphabetical order. For each actor set, the following

information is provided:

Maximum members

This is the maximum number of members the actor set contained during the simulation.

Members (bytes)

Some of the memory used by an actor set depends on the membership of actors in the actor set. For each actor member of the set, memory is needed to keep the position of the actor in the set, etc. This is the memory reported here. This column reports the memory used for each member of the set multiplied by the maximum number of members the set has had during the simulation. To help in understanding this column, the memory used for one member of a set is given at the end of the report.

Internal states (bytes per actor) for actor sets

Depending on the declaration of the actor set, some internal states might be created by Modgen for the set. For instance, if the actor set's declaration is:

```
actor_set Host asAllHosts
order RandomStream();
```

then an internal state is created to keep the random number associated with the actor member of the set. It is also possible for derived states to be declared in an actor set, as is the case for tables. If so, that memory is reported here, and is given for one actor only. That memory is used even if the actor is never a member of the set.

Total internal states memory (bytes)

This is the "internal states memory" multiplied by the maximum number of active actors given in the table "Maximum instances". Note that this is multiplied by the maximum number of active actors, and not by the maximum number of members in the set, because that memory is used whether or not the actor is in the set.

Actor set cells

This is the number of cells in the actor set multiplied by the memory taken by one actor set cell. This memory is used even if the actor set is empty. The memory taken by one actor set cell is given at the end of the report.

Total (bytes)

This is the sum of the memory taken by members, the total internal states memory, and the actor set cells.

Table 6: Memory use: Summary

The last table in the report is a summary. It reports the total memory used for parameters, actors and events, actor sets, and tables.

Annex A – parameter and state sizes

Three annexes come after the tables reporting memory use. The first annex is a list of possible data types with the memory used by one state or parameter of this type. For instance, a state declared with the type “int” will use 4 bytes, whereas a state declared with a type “double” will use 8 bytes.

The size of the type “TIME” depends on the declaration of the type “TIME” given by the model developer. For instance, our example model Simpex has the following declaration:

```
time_type int;


That means a state declared with the type “TIME” will use 4 bytes.
```

Annex B – internal states

For each derived state function, the maximum number of bytes used by internal states is given. For instance, Modgen creates one internal state of type “int” for each derived state “changes”. (e.g. changes(dfle)).

In some cases, the number and type of internal states created by Modgen depend on the context. For instance, Modgen creates 2 internal states of type “double” and 1 internal state of type “TIME” for a “weighted_duration” derived state if it used in the declaration of a state. If, however, the “weighted_duration” is only used inside a table, then Modgen only creates 1 internal state of type “double” and one internal state of type “TIME”. That’s why the number of bytes given is a maximum.

Also reported in this annex is the number of internal states created for a table. This only includes internal states created for the state itself, not internal states created for derived states declared in the table.

Annex C – structures

This annex provides a list of different structures referred to in the report and gives the memory used for the structure itself. This contains the actual size of the actor class created by Modgen for each type of actor. This size includes the effect of packing_level, but unlike Table 3 ”Memory use: Actors and events”, it does not include dynamically-allocated memory used within actors, e.g. the memory used to store links to multiple actors.

Model Log File

Changes to model log file

Simulation information

The log file produced by Modgen models contains the time at which the simulation started as well as selected information on workstation characteristics such as computer name, user name, number of processors, and processor model. The presence of this information may depend on the operating system.

If the model should crash or hang, the log file may contain additional diagnostic information that may be useful to the Modgen development team in identifying the cause of the problem. This diagnostic information is automatically removed from the log file on successful model completion. The CPU processor clock speed is also written to the model log file.

Performance information

All the total performance stastistics are written at the end of the log file. Current performance statistics are written periodically in the log file. They are written approximately every time 10% of the simulation has been completed. If a model is very fast, the performance statistics will be written less frequently.

Modgen version and model version

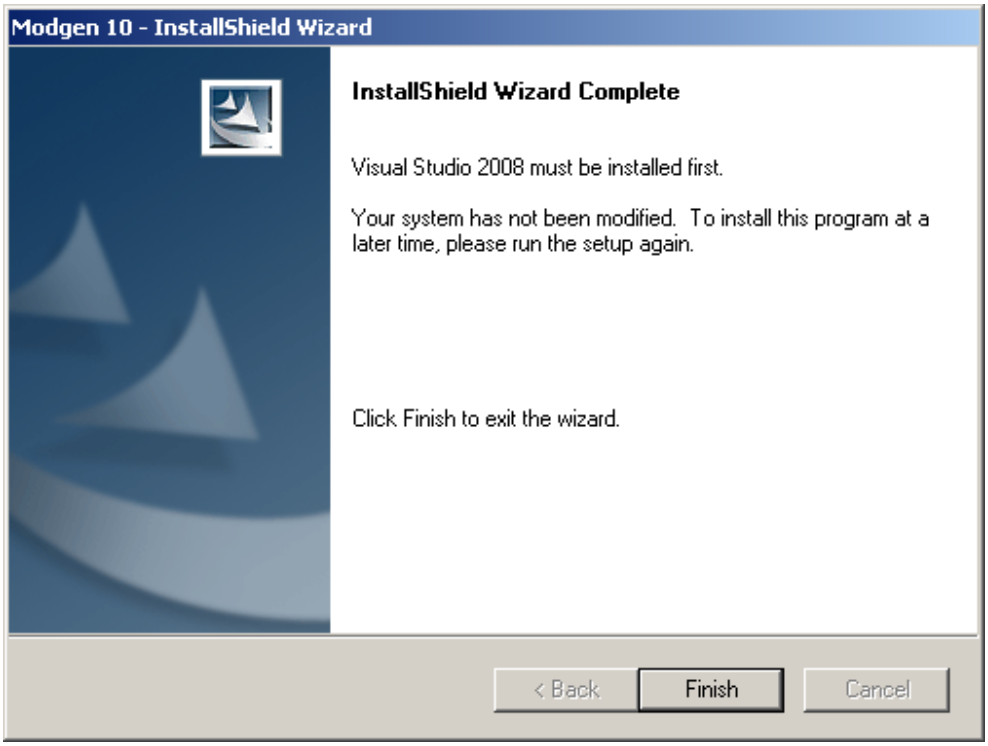
Additional lines are written to the scenario log file, which report the version of Modgen and the name and version of the model, e.g.

- Modgen Version: 10.0.2
- Model Name: simpex
- Model Version: 2.0

Appendix: Modgen and Visual Studio 2008

Configuring Visual Studio 2008 for Modgen 10 Applications

Modgen 10 was developed using Visual Studio 2008, and you need to have Visual Studio 2008 installed on your machine in order to compile your Modgen 10 models. In fact, Visual Studio 2008 must always be installed first before installing Modgen 10. Any attempt to install Modgen 10 on a machine that does not have Visual Studio 2008 will in turn cause the following dialog box to appear:



Once Visual Studio 2008 and Modgen 10 are installed on a machine (in that order), Visual Studio 2008 is configured automatically (by the Modgen 10 setup program) to work with Modgen 10 applications.

Converting Modgen models to Visual Studio 2008

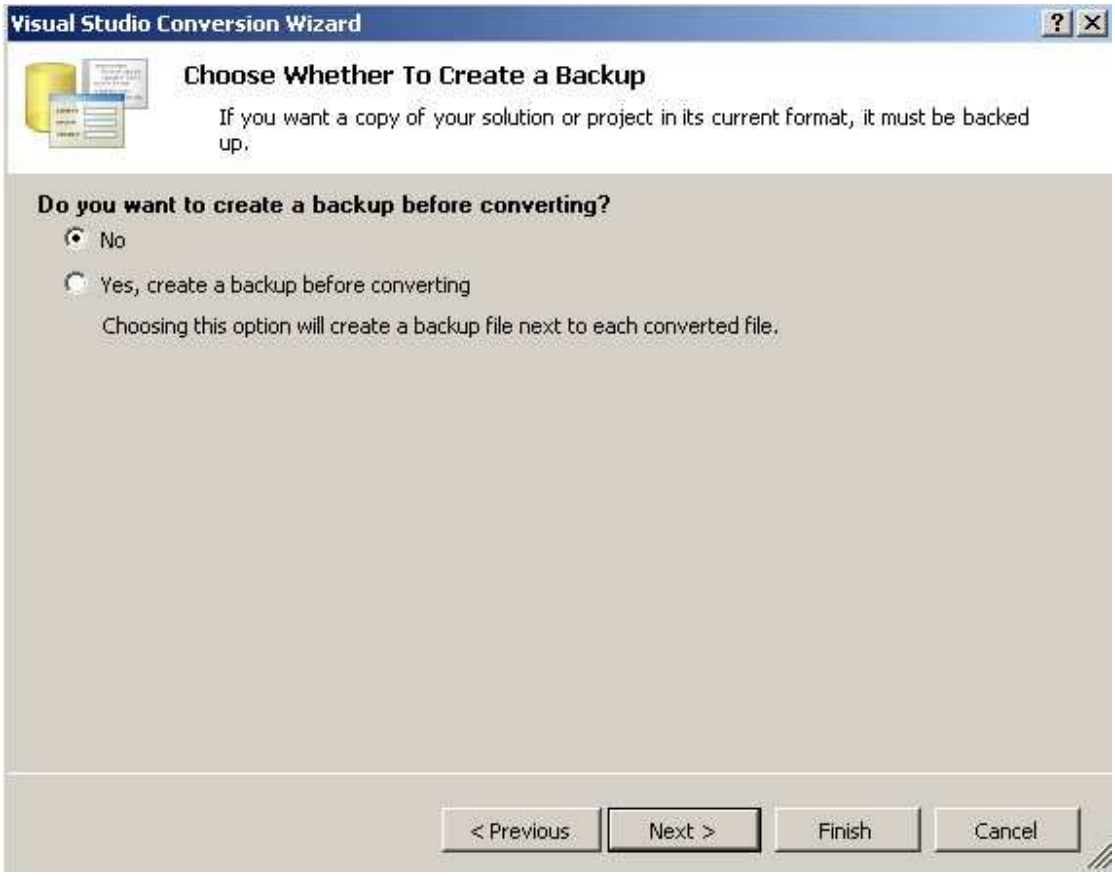
If you have any models created with Modgen 9, their corresponding projects need to be converted to Visual Studio 2008 so that the models can be compiled. This section explains how to convert model projects created or updated with Visual Studio 2005 to Visual Studio 2008.

Before trying to convert, take a backup copy of your model's folder. Once a model has been converted, there's no way (apart from recreating the project from scratch) to convert the project back to a format usable by Visual Studio 2005, and hence no way to use a previous version of Modgen to compile the model.

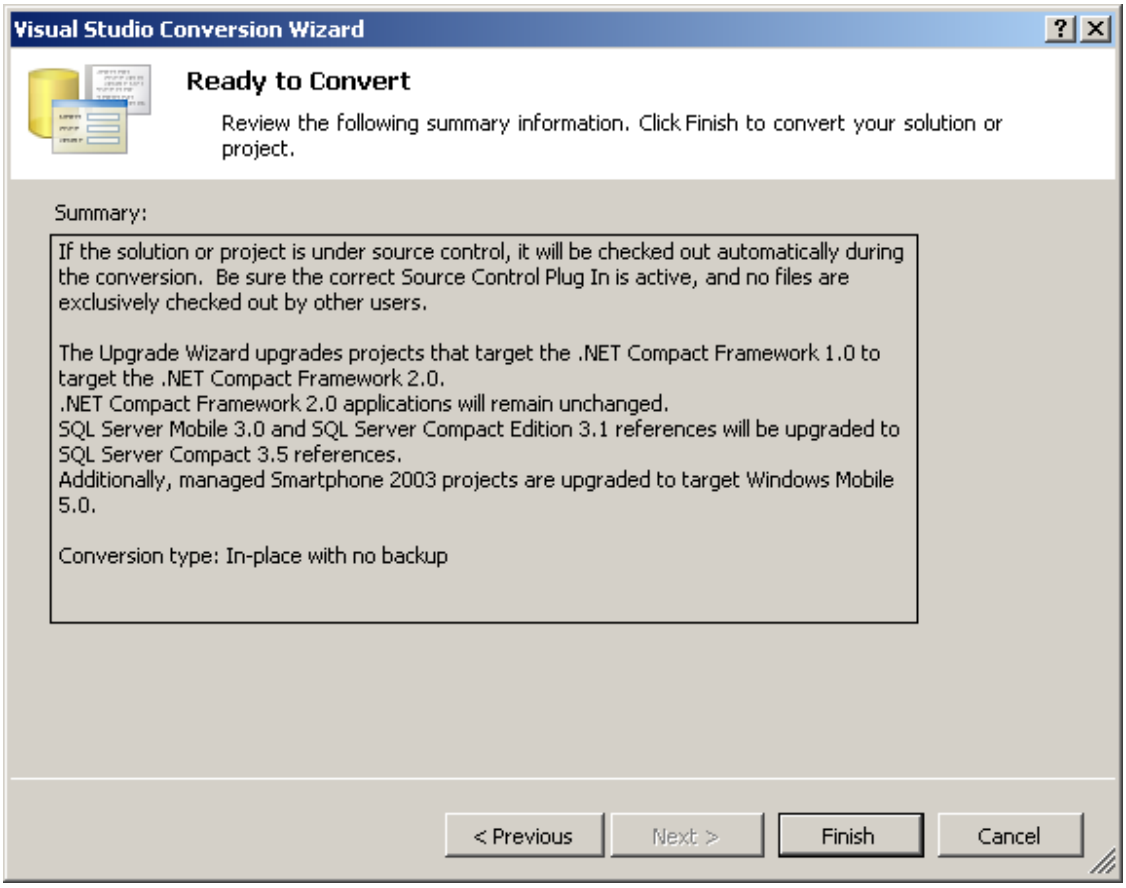
Open Visual Studio 2008. Choose "Open" then "Project/Solution" from the "File" menu. From the "Open project" dialog, navigate to your model's folder and select your model's solution file (.sln). This will start the Visual Studio Conversion Wizard:

Note that you cannot choose to double-click on your solution file instead of opening it this way, at least not if you have both Visual Studio 2005 and Visual Studio 2008 installed on your machine.

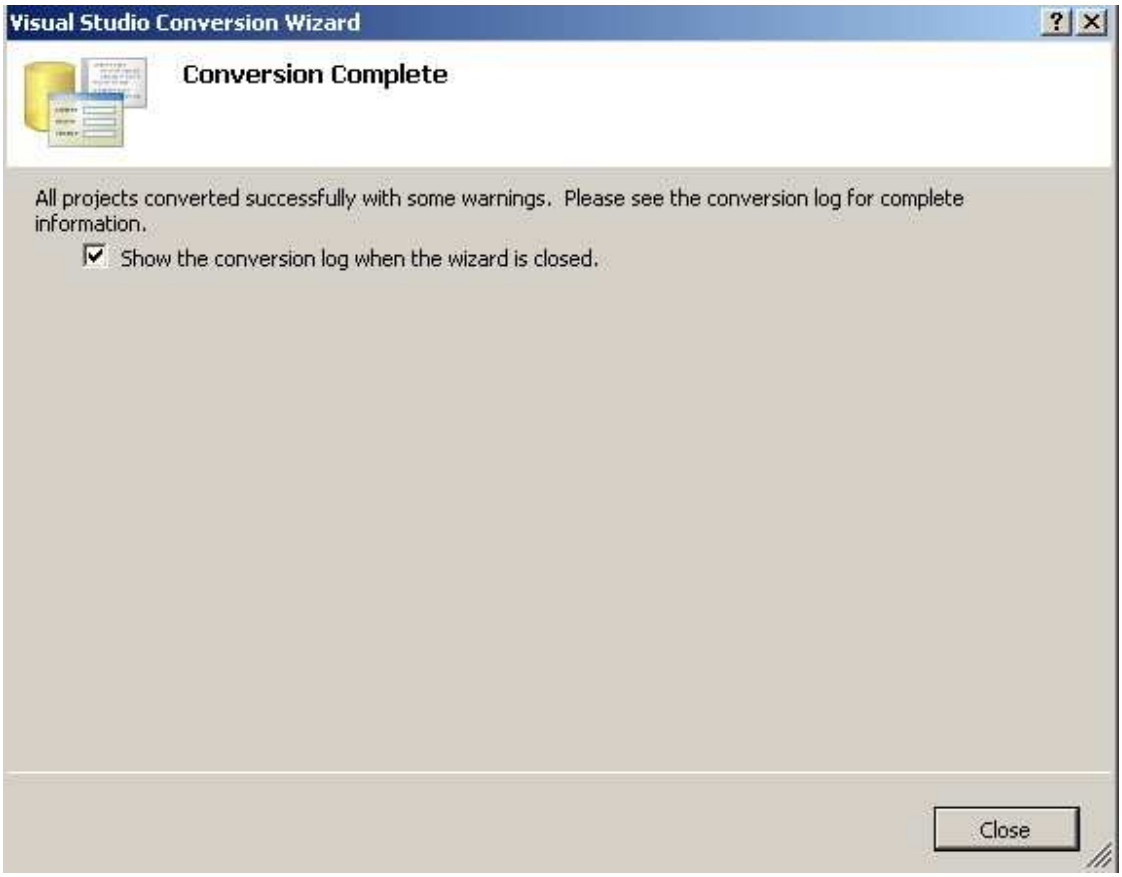
Click “Next” to proceed. You will be asked if you want a backup of your project before converting. If you have copied your model’s folder before starting, you can choose not to have a backup copy.



Click “Next”. You will next be asked to confirm that you are ready to convert.

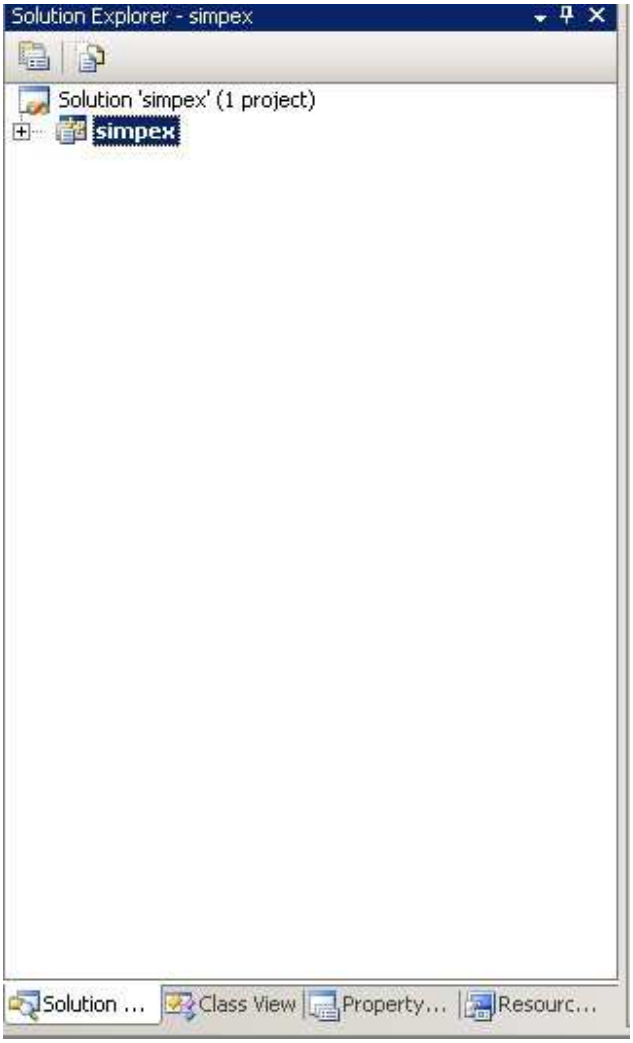


Click "Finish" to convert your model's project. Once the conversion is done, you will be given an option to view the conversion log.



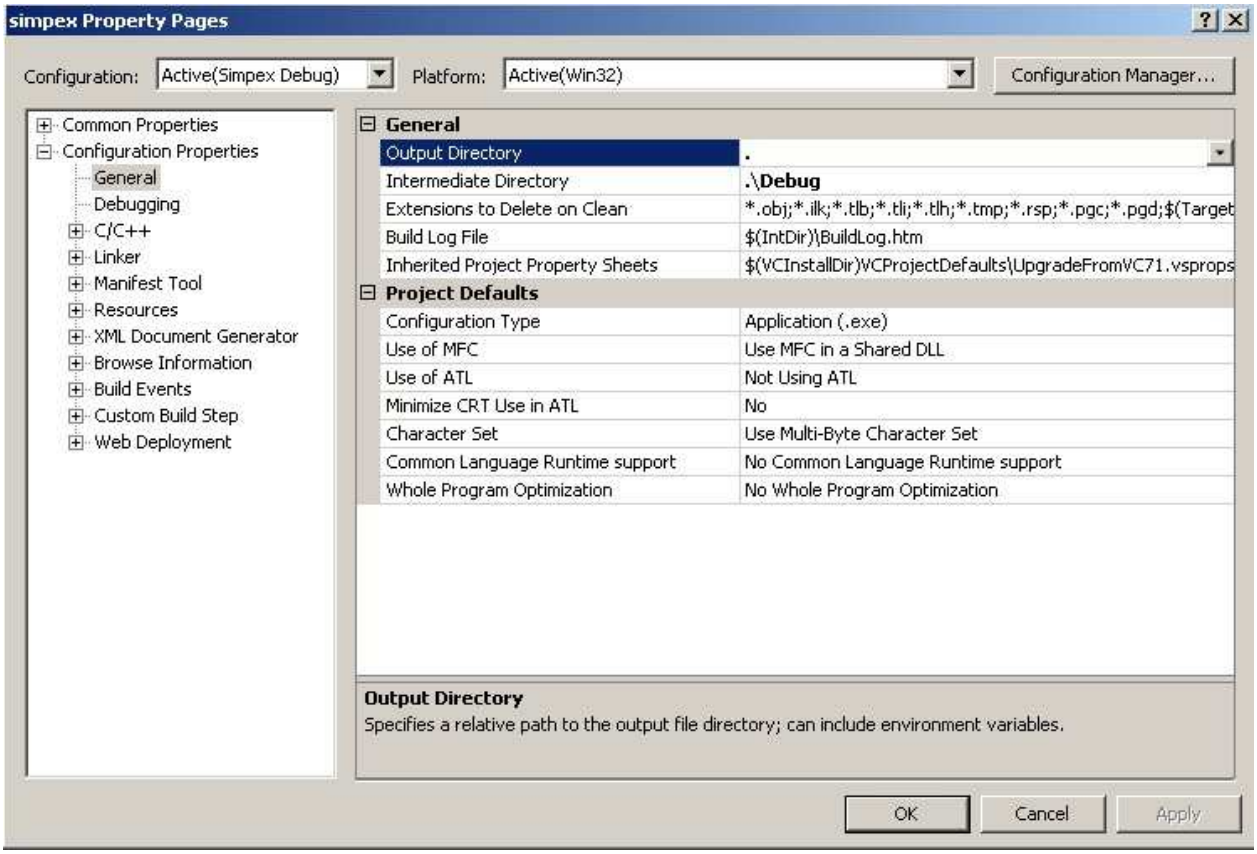
If you decide you want to see it, expect to see some warnings.

After the conversion is complete, you should see the “Solution Explorer” in the left pane of your Visual Studio 2008 screen.



If you don't see the Solution Explorer, choose "Solution Explorer" from the "View" menu.

All of your project settings are preserved when you convert to Visual Studio 2008 from Visual Studio 2005, so you should not need to modify any of them. However, if you nevertheless wish to change some of them, right-click on the project in the Solution Explorer and choose "Properties". You will then see the following screen:



To see the settings common to all configurations (release and debug), choose “All Configurations” in the ‘Configuration’ drop-down list. Note that when you change the configuration in the drop-down list, any changes made to the settings will cause the following message to be displayed:

Assuming you wish to keep the changes you made, click “Yes” to continue and change the configuration.

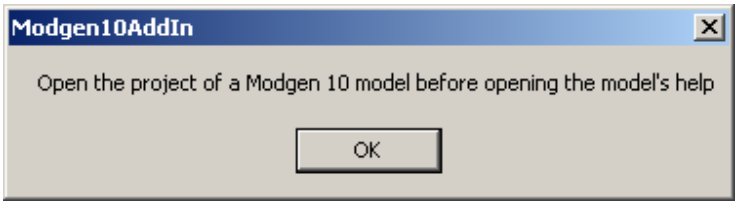
Modgen 10 Toolbar in Visual Studio 2008

The first time Visual Studio 2008 is opened after installing Modgen 10, a toolbar is created for Modgen 10:

From left to right, these icons have the following meanings:

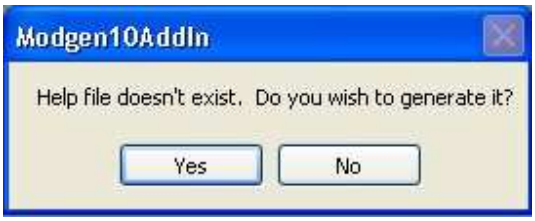
- Run Modgen 10: runs Modgen 10 in the solution's directory. The output is displayed in English. If there is no active solution in Visual Studio 2008, the following message is displayed:

- Modgen 10 Developer's Guide: opens the Modgen 10 Developer's Guide, in English. The guide will automatically close when Visual Studio 2008 is closed.
- Modgen 10 Release Notes: opens the release notes for Modgen 10 in English. The release notes will be closed when Visual Studio 2008 closes.
- Model help: opens the documentation for the model in English. This command assumes the solution's name is the name of the model. As an example, if the solution "Simpex.sln" is opened, the command assumes the name of the model is "Simpex". If your solution doesn't have the same name as the model, this command won't succeed at opening the documentation. If there is no active solution, the following message is displayed:



This command verifies that the documentation (2_EN.chm) exists and is valid. If it is not found, the command will try to look for the file 2.chm which would be created in a model without any languages declared. If neither is found, then

it will ask if the user wishes to create it:



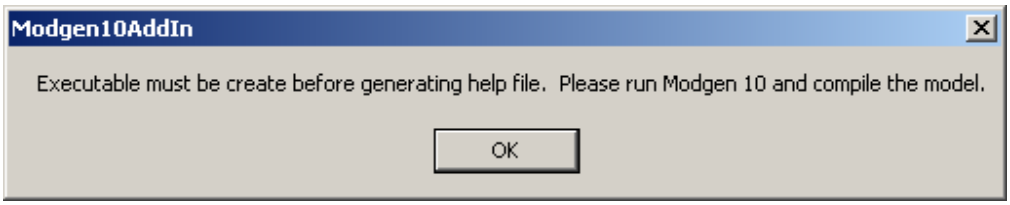
Once the documentation is created, it will once again look for the documentation (2_EN.chm or 2.chm) and if it is found, it will consider that the documentation is valid and can be displayed. Note that in a model where languages other than "EN" are defined, the command will fail to identify a documentation to display. Even if the user agreed to generate the documentation, the files created won't correspond to the files the command is looking for.

If the documentation is found (either 2_EN.chm or 2.chm) but is older than the executable, then it will offer to re-generate the documentation before opening it:

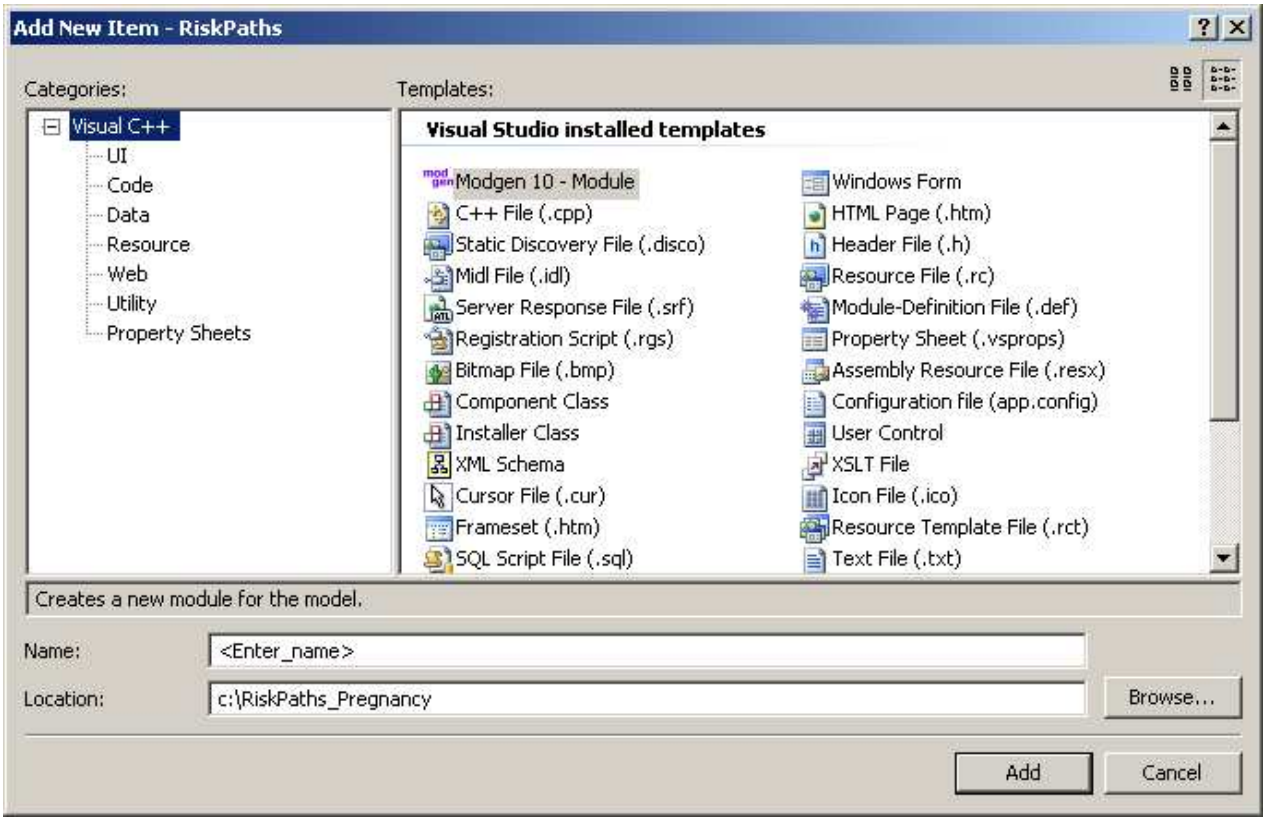
Again, once the model has finished generating the help files, the command will look again to see if either the file 2_EN.chm or 2.chm exist and if so, it will consider that it is valid and can be displayed.

At the end of that, if there is a valid documentation, then this command will open the file 1_EN.chm if it exists and 2_EN.chm if not.

Note that to generate the documentation, the command launches the model with the "-help" argument to create the documentations. That means an executable is required for this to work. If no documentation is found and no executable is found (.exe), then the following message will be displayed:



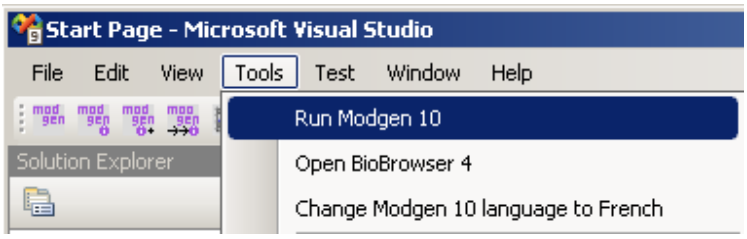
- Open BioBrowser 4: opens BioBrowser 4 but does not open any file within BioBrowser. This command is available only if BioBrowser 4.0.2 (or a later version) is installed on the machine. If it is not, then the command is disabled.
- Change Modgen 10 language to French: changes the language of all the other commands. This command changes the labels for all the other commands to their French versions and changes the behaviour of the other commands as well. After the developer clicks on this command, all the other commands will be executed in French. That means that the output to Modgen 10 will be displayed in French and the guides and release notes will be displayed in French. This command will be changed to "English". The language used is preserved so that the next time Visual Studio 2008 is opened, the commands will be displayed in the last language used.
- Add New Item: starts a wizard to create a new item. This is the equivalent to either going through the menu "Project" and choosing "Add new item" or right-clicking on the project in the "Solution Explorer" and choosing "Add new item". If there is no active project, this command is disabled. When the wizard begins, the following dialog appears:



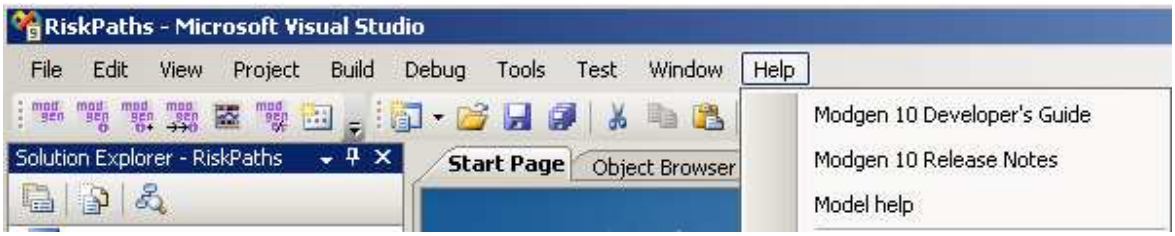
Specify the name of the new module. Note that module names cannot contain spaces or special characters and the wizard will refuse to create a module if the name isn't a valid Modgen name. Do not change the location. Double-

click on the "Template" called "Modgen 10 - Module". The wizard creates an empty mpp and corresponding cpp file and adds both to your project.

Each of the above commands can also be found in the menus of Visual Studio 2008. The commands "Run Modgen 10", “Open BioBrowser 4” and "Change Modgen 10 language to French" are found in the "Tools" menu:



The commands "Modgen 10 Developer's Guide", "Modgen 10 Release Notes" and "Model help" are found in the "Help menu":



Modgen Model Development in C++

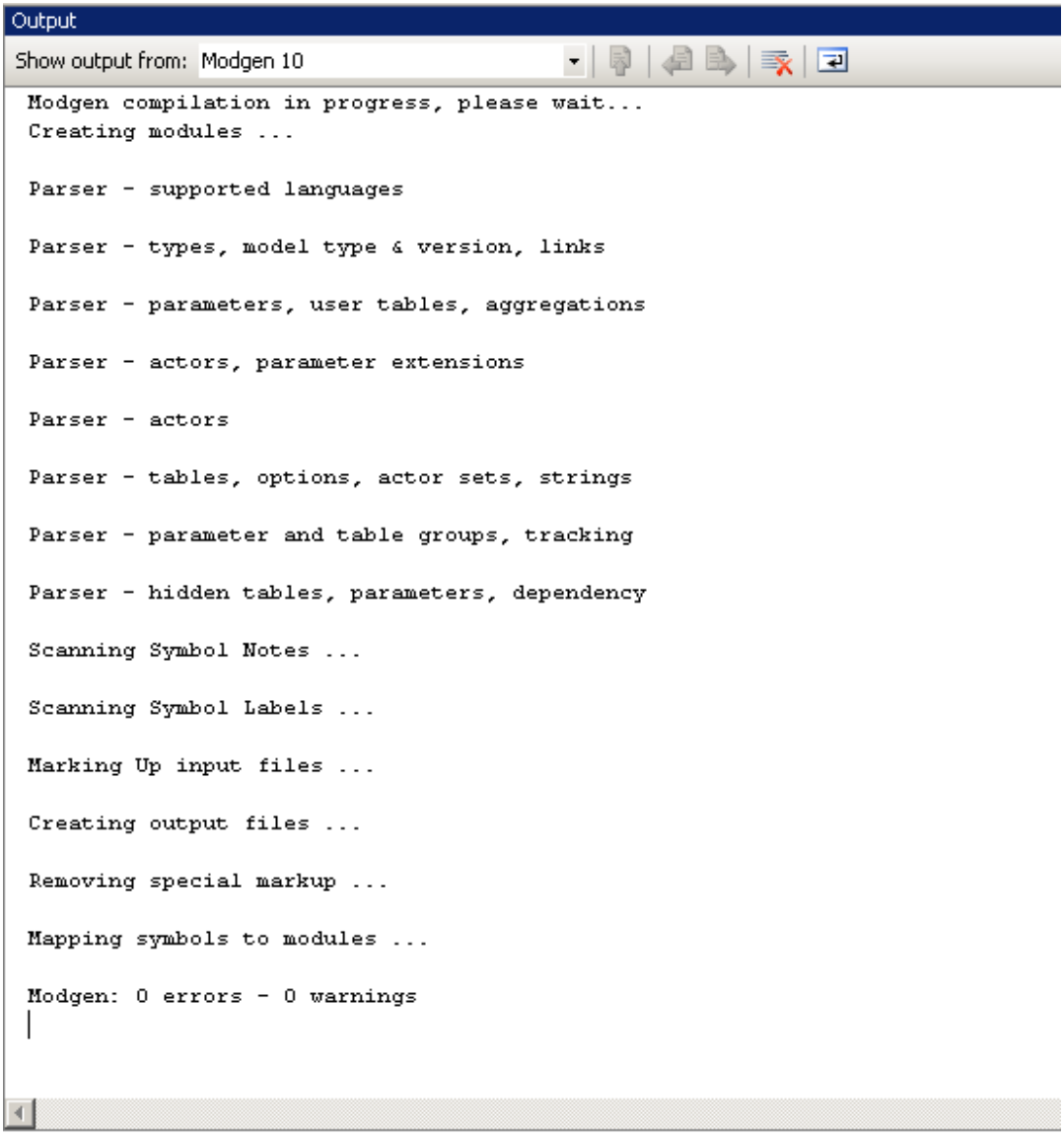
Open the .sln file in Visual Studio 2008. You can do this by navigating to the .sln file using Explorer, and double clicking the file. Ensure that the default configuration is the ‘Release’ version, and not the ‘Debug’ version.

To change a model parameter and run the model, double click the appropriate .dat file in project files. This will open an edit window displaying the contents of the selected parameter file. Modify the file as desired using the Visual Studio 2008 editor. After changing parameter values, run the model by selecting “Start Without Debugging” from the menu “Debug”.

To change model source code and create a new model, open the desired .mpp source file by double clicking its name and then making the code changes. Next, select the ‘Run Modgen 10’ command from the Tools menu, or alternatively click on the “Run Modgen 10” icon from the Modgen 10 toolbar. Examine the output in the output window or in the Modgen

output file (“ModgenLog.htm”) for errors. If there are errors, you can click on the error in the output window to go directly to the place in the code where the error occurred.

Here is an example of an output window:



Next, if there are no errors in the output window, select the ‘Build/Build’ command. If you have any compilation errors in your model code, you can double click the error line to navigate to the offending piece of code, just as you can when there are errors after executing the ‘Run Modgen 10’ command. Once all errors are fixed, run Modgen 10 again, then re-compile the model again. When both steps succeed without error, the new model executable has been created.

Folders in projects in Visual Studio

In Visual Studio 2008, sub-folders in projects are called filters. It may be convenient to use this feature in large models (those with many mpp files) in order to separate them from the C++ files. The sub-folders may also be used to organize the DAT files for easy viewing in the text format. To create a new sub-folder right-click on the parent folder (e.g. “Simpex files”), choose “Add” and “New filter” from the context menu. Enter the name for the new folder, optionally also file extensions if all new files of those types will belong to the new folder (e.g. *.mpp if you’re creating a folder for all mpp files). Click OK. You’ll have to move existing files around by dragging and dropping in the correct folder.

To add (or change) an extension associated with the filter, right-click on the sub-folder and choose “Properties” from the context menu. In the “Properties” window, add the extension (e.g. *.mpp for all mpp files) in the “filter” setting.

The Simpex project uses subfolders and may serve as an example. The existing C++ and mpp files have been organized into 2 sub-folders: “mpp files” and “C++ files”. Since model developers only need to look at mpp files, the use of sub-folders reduces in size the list of files that can be viewed.

Appendix: Table of Old and New Names

As Modgen has continued to be developed and enhanced over the past several years, some features have had their names revised. The body of this document only refers to these items by their revised or current names. However, for those who might be maintaining older models, the following table displays the former and revised names for each changed entity.

Current Name	Previous Name	Where Referenced?
duration_trigger	duration_dummy	Derived states associated with durations of actor states
active_spell_duration	current_spell_duration	Derived states associated with durations of actor states
active_spell_weighted_duration	current_spell_weighted_duration	Derived states associated with durations of actor states
active_spell_delta	current_spell_delta	Derived states associated with durations of actor states
completed_spell_duration	previous_spell_duration	Derived states associated with durations of actor states
completed_spell_weighted_duration	previous_spell_weighted_duration	Derived states associated with durations of actor states
completed_spell_delta	previous_spell_delta	Derived states associated with

		durations of actor states
LABEL(module_name,language_code)	FILE_LABEL(language_code)	Symbol Labels
NOTE(module_name,language_code)	NOTE(language_code)	File and Symbol Notes
ModuleDic	SourceFileDic	Modgen Documentation Database (Appendix)
RandNormal()	RandDeviate()	Functions associated with random number generator
RandUniform()	RATE(LongRand())	Functions associated with random number generator
Undergone_change()	First_changes()	Derived states associated with changes to actor states
Undergone_entrance()	First_entrances()	Derived states associated with changes to actor states
Actor_id	ID (no longer in Modgen)	
mm	modgen_model	Creating and running a new model executable (namespace name)
SPLIT	Split	Run-time Functions and Macros